

Universidad de Valladolid

Escuela de Ingeniería Informática

TRABAJO FIN DE GRADO

Grado en Ingeniería Informática
Mención en Ingeniería de Software

Aplicación móvil para la información y geolocalización de cofradías y procesiones en semana santa.

Autor:
Elena Martínez Vázquez

Tutores:
Juan Alberto Muñoz Cristóbal
Pablo Federico Sanchez Mayoral

Dedicatoria

Agradecimientos

Resumen

Abstract

Índice general

Índice de figuras

Índice de tablas

Acrónimos

- **API**: Application Programming Interface.
- **PWA**: Progressive Web App (ya lo explicas en el texto, pero debería estar en el apéndice)
- **GPS**: Global Positioning System
- **UML**: Unified Modeling Language
- **RF**: Requisito Funcional
- **RI**: Requisito de Información
- **RNF**: Requisito No Funcional
- **HU**: Historia de Usuario
- **TFG**: Trabajo de Fin de Grado

Capítulo 1

Introducción

1.1. Contexto

La Semana Santa es una de las fiestas culturales y religiosas más relevantes del calendario español. Detrás de esta tradición hay siglos de historia, ya que forma parte del patrimonio histórico y artístico del país. En ciudades como Sevilla, Málaga o Valladolid, entre otras, está declarada Fiesta de Interés Turístico Internacional. Esta celebración tiene un gran impacto tanto cultural como económico, debido a los miles de visitantes nacionales e internacionales que atrae cada año.

La estructura organizativa de la Semana Santa está formada por las cofradías o hermandades, las cuales son asociaciones religiosas encargadas de organizar las procesiones y los eventos de estos días. La procesión es el acto central: cada una recorre un itinerario propio por las calles de la ciudad, llevando consigo pasos[1] de gran valor artístico que representan escenas de la Biblia. Junto a las procesiones, las cofradías organizan también otros eventos de carácter más puntual –pregones, traslados u ofrendas florales, entre otros– que completan el programa de la Semana Santa pero, a diferencia de una procesión, no siguen un recorrido propio por la ciudad. Esta organización de horarios, recorridos y actos requiere una gran coordinación por parte de las distintas cofradías y de la Junta de Cofradías de cada ciudad. Durante estos días, la ciudad experimenta imprevistos, como cortes de tráfico, cambios en la movilidad urbana y una gran cantidad de personas en puntos estratégicos del recorrido.

Desde el punto de vista social, la Semana Santa puede implicar dificultades de movilidad debido al desconocimiento de la ubicación de las distintas procesiones, además de los cambios de última hora en cuanto a los horarios de comienzo de cada procesión debido al mal tiempo o incluso a cancelaciones. Como esto se sale de la planificación inicial, los cofrades encuentran dificultades a la hora de comunicar estos cambios y decisiones de última hora a los espectadores y fieles. A día de hoy, son las distintas Juntas de Cofradías las que deben gestionar toda esta información de forma manual o con herramientas anticuadas.

En el contexto tecnológico actual, la mayoría de los turistas y ciudadanos hacen uso de sus teléfonos

[1]Paso: plataforma o andamio, habitualmente de grandes dimensiones y ricamente ornamentado, sobre el que se transportan a hombros o mediante ruedas las imágenes religiosas durante la procesión.

móviles como principal medio para consultar información. Aplicaciones como Google Maps [**?**] permiten conocer en tiempo real la ubicación (geolocalización [**?**]) de calles, monumentos, bares, etc. Esta funcionalidad podría aplicarse a la Semana Santa, de forma que facilitaría a los visitantes y ciudadanos información constante y rápida sobre dónde se encuentra una procesión o si ha habido algún cambio o imprevisto en la misma.

Finalmente, debido a la cantidad de dinero que genera el turismo durante la Semana Santa, podemos concluir que es de gran interés para las ciudades que la experiencia del turista y del ciudadano sea positiva y que existan herramientas para que esto se haga realidad, de forma que una aplicación que mejore dicha experiencia repercutirá positivamente.

1.2. Motivación

La principal motivación de este proyecto surge de la dificultad que tienen tanto los ciudadanos como los visitantes a la hora de buscar y obtener información clara y actualizada durante los días de la Semana Santa. Aunque a día de hoy existan programas oficiales, carteles informativos o cuentas oficiales en redes sociales, estos medios no siempre consiguen dar a conocer en tiempo real la situación de las procesiones, ya sea por retrasos o cambios de última hora debidos a factores meteorológicos o a problemas dentro de la propia cofradía.

Debido a esto, los asistentes, en muchas ocasiones, deben desplazarse a los distintos puntos del recorrido sin tener la certeza de que la procesión no haya pasado ya, cuánto tiempo de espera se estima hasta que pase por ese punto o incluso si llegará a pasar debido a cancelaciones. Esto genera desorganización y aglomeraciones innecesarias, además de una experiencia menos satisfactoria para los observadores.

Gracias a estas observaciones, se ve clara la necesidad de contar con un medio común capaz de organizar y difundir la información necesaria para que la experiencia sea lo más clara y accesible posible para cualquier persona.

Como respuesta a estas limitaciones, se plantea la idea de una aplicación móvil que permita la centralización de la información sobre el estado de las procesiones de forma actualizada, además de incluir otras funciones como la visualización de los recorridos y la geolocalización en tiempo real de las mismas. De esta forma, se mejora no solo la experiencia de los usuarios, sino que también se facilita la gestión de la información durante la Semana Santa.

Además, el desarrollo de esta aplicación permite no solo aplicar los conocimientos adquiridos durante el grado sobre análisis de requisitos, diseño de software, desarrollo de aplicaciones móviles y gestión de proyectos, sino también ampliarlos mediante su aplicación en un proyecto real, enfrentándose a problemas prácticos como la integración de distintas tecnologías, el manejo de la geolocalización en tiempo real o la gestión de múltiples tipos de usuarios dentro de un mismo sistema.

1.3. Estado de la cuestión

En la actualidad existen diversas soluciones tecnológicas relacionadas con la Semana Santa, aunque la mayoría de ellas se encuentran limitadas a ámbitos geográficos concretos o se centran únicamente en la difusión de información estática. A continuación, se analizan las principales herramientas disponibles, así como sus características más relevantes.

1.3.1. Aplicaciones actuales de Semana Santa

A día de hoy, existen diversas aplicaciones móviles orientadas a la Semana Santa para distintas ciudades españolas. Entre ellas encontramos *El Penitente* [**?**], centrada únicamente en las ciudades de Sevilla y Málaga. Esta aplicación ofrece información sobre horarios, itinerarios y una representación del recorrido de las procesiones sobre el mapa, mostrando el trayecto seguido y, en algunos casos, su posición aproximada.

La aplicación presenta una interfaz (Figura 1.1) en la que el usuario puede consultar información sobre las procesiones y sus recorridos, notificaciones, planificaciones para organizar la Semana Santa y horarios de hermandades o cofradías.

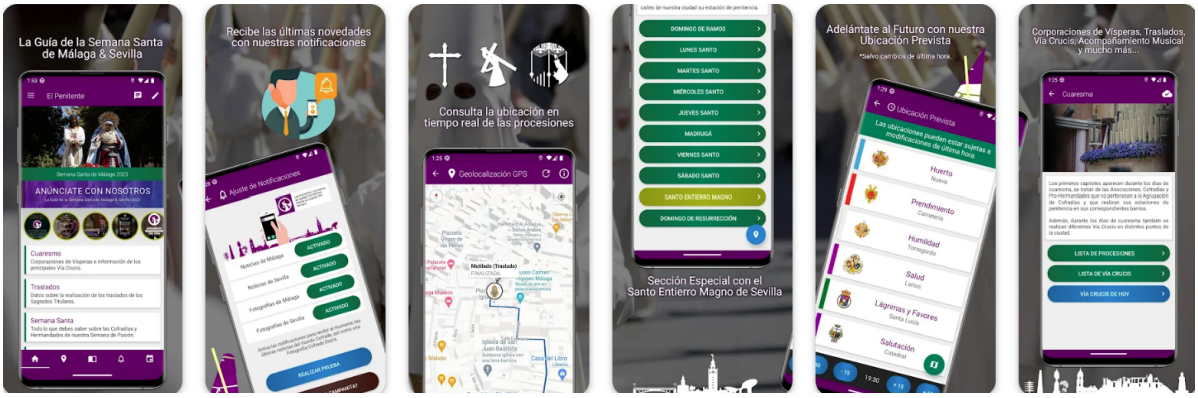


Figura 1.1: Interfaz de la aplicación El Penitente [**?**]

Su mayor limitación es que está diseñada y pensada para ciudades específicas y no constituye una solución unificada a nivel nacional.

Otra aplicación a analizar es *Valladolid Cofrade* [**?**], una herramienta reciente desarrollada como aplicación web progresiva (PWA) [**?**], es decir, una aplicación accesible desde el navegador que puede instalarse en el dispositivo y funcionar de forma similar a una aplicación nativa. Esta permite a los usuarios acceder a información sobre la Semana Santa sin realizar la instalación tradicional desde tiendas oficiales. La aplicación ofrece información sobre programas, recorridos y un sistema de notificaciones en tiempo real ante cambios o incidencias (Figura 1.2).

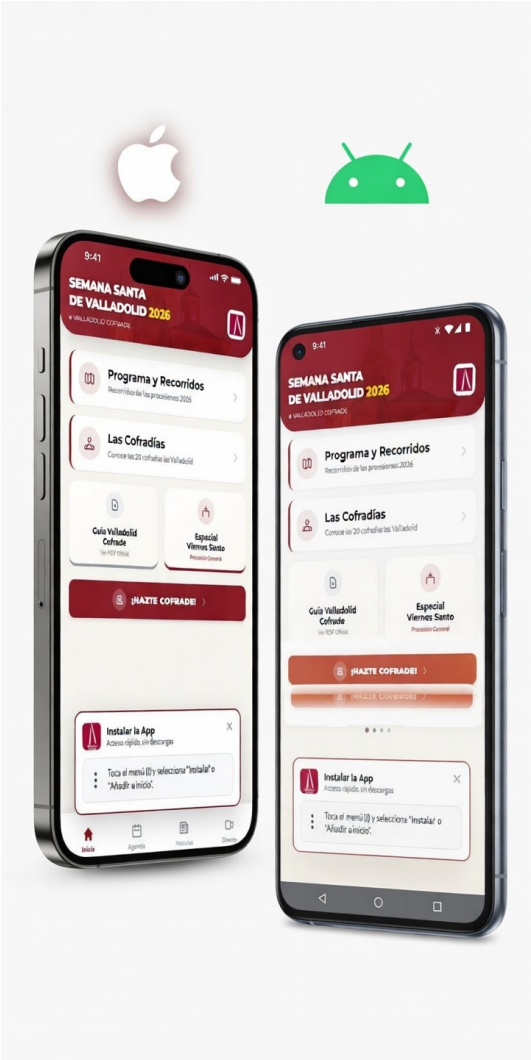


Figura 1.2: Interfaz de la aplicación Valladolid Cofrade [**?**]

En otras ciudades se cuenta con aplicaciones locales específicas o sitios web adaptados, como sSantaZgz en Zaragoza [**?**] o Junta Pro Semana Santa de Zamora [**?**]. Estas aplicaciones muestran el programa oficial, los distintos recorridos e incluso, en algunos casos, como en la de Zamora, incluyen herramientas para conocer la previsión meteorológica.

Más allá de estas limitaciones técnicas, también hay que destacar la diversidad organizativa de la Semana Santa en España, ya que esta depende de la ciudad, de la estructura que se siga en ella y de la terminología utilizada. Por ejemplo, en algunas ciudades, como Valladolid, es común emplear los términos *cofradía* o *cofrade*, mientras que en otras, como Sevilla, se utilizan *hermandad* o *nazareno* para referirse a conceptos equivalentes. Desde el punto de vista organizativo, existen diferentes formas de organizar los eventos, recorridos e itinerarios de las procesiones. Incluso hay diferencias en elementos como pasos, imágenes, homilías, rezos o cánticos que forman parte de estas procesiones o eventos.

Estas diferencias hacen que cada aplicación tenga un diseño distinto, pensado específicamente para una ciudad concreta, dificultando la creación de una solución reutilizable. Por tanto, en la actualidad no existe una plataforma que consiga unificar y generalizar estos conceptos y ofrecer una visión global de la

Semana Santa.

Además de las aplicaciones, los usuarios también hacen uso de las publicaciones oficiales de las cofradías en sus páginas web o de programas impresos. Sin embargo, los medios que realmente resultan útiles para la actualización inmediata de la información son las redes sociales oficiales de las cofradías o de las Juntas de Cofradías, que permiten a los espectadores recibir información sobre cambios o comunicados al instante. Aun así, esta forma de comunicación no siempre resulta fácil de encontrar o de organizar debido a la diversidad de cuentas que gestionan dicha información.

1.3.2. Aplicaciones de seguimiento en tiempo real

En cuanto a la geolocalización y el seguimiento en tiempo real, se trata de una tecnología ya existente que se utiliza en diversas aplicaciones como *Google Maps* [**?**], que permite conocer la ubicación, cómo llegar a distintos destinos o el estado del transporte público (Figura 1.3).

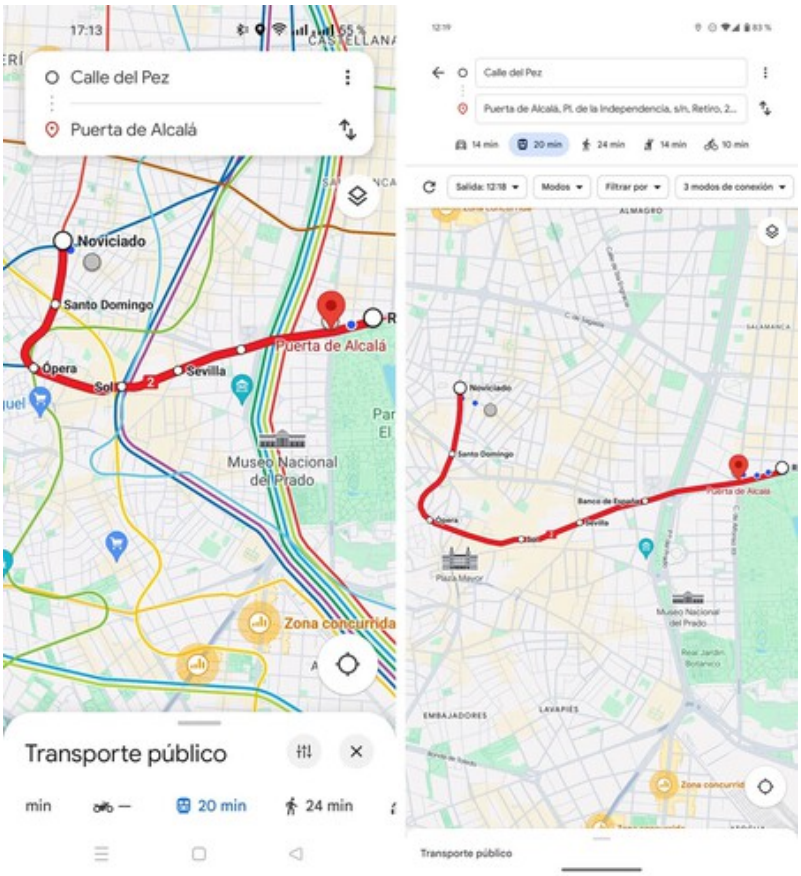


Figura 1.3: Ejemplo de geolocalización en tiempo real en Google Maps

Además, plataformas como *Uber* [**?**] permiten a los usuarios conocer en todo momento la ubicación exacta del vehículo solicitado (Figura 1.4).

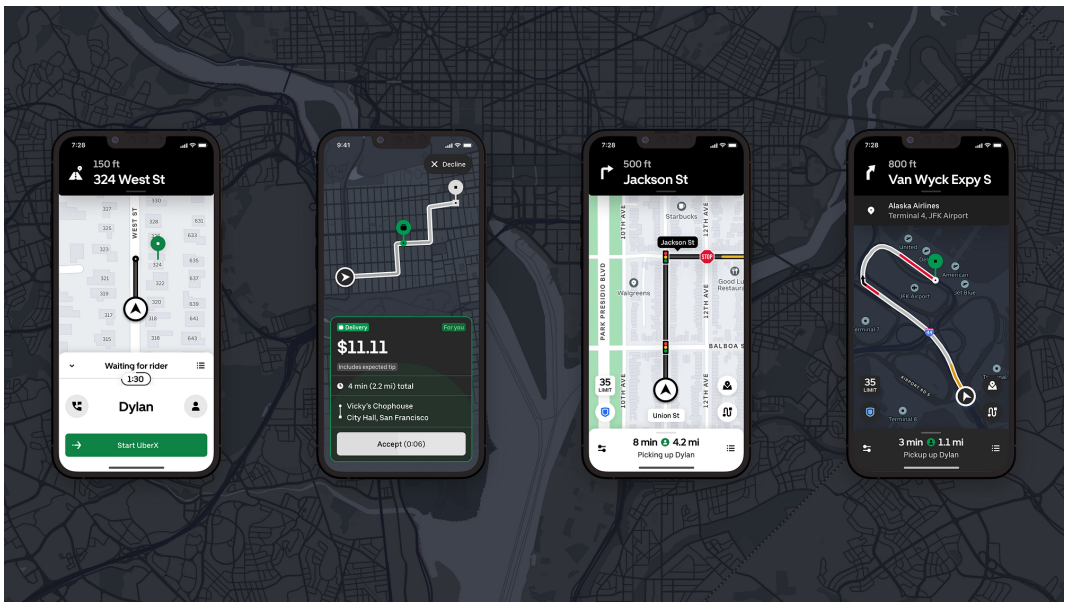


Figura 1.4: Seguimiento en tiempo real de un vehículo en Uber [**?**]

Además, existen diversos Trabajos de Fin de Grado centrados en el desarrollo de aplicaciones móviles y el uso de tecnologías de geolocalización. Un ejemplo es el proyecto realizado por Mario Llorente Pérez [**?**], enfocado en una aplicación móvil relacionada con actividades deportivas y el uso de mapas y geolocalización. En trabajos como este puede observarse el importante papel que juega la localización en tiempo real y la necesidad de diseñar interfaces que permitan al usuario interactuar con la aplicación de forma sencilla.

Todas estas aplicaciones y trabajos constituyen ejemplos claros de que el seguimiento en tiempo real es una funcionalidad ampliamente utilizada, a pesar de que en el contexto de la Semana Santa sigue estando limitada y, cuando se ofrece, suele restringirse a entornos locales o presentar un nivel de precisión reducido.

Por tanto, su incorporación en una solución a nivel nacional conseguiría no solo mejorar la experiencia de los usuarios, sino también superar las limitaciones de las soluciones actuales en cuanto a la oferta de información precisa sobre el estado y la ubicación de las procesiones o eventos.

1.4. Objetivos

El objetivo principal de este Trabajo de Fin de Grado es el diseño y desarrollo de una aplicación móvil orientada a la Semana Santa, capaz de centralizar en una única plataforma la información de las distintas celebraciones de Semana Santa a nivel nacional, con independencia de la ciudad, cofradía o terminología empleada en cada una de ellas. En concreto, se busca ofrecer a los usuarios una herramienta que les permita consultar la información relevante de la Semana Santa de una ciudad y visualizar en tiempo real, mediante geolocalización, la ubicación, el recorrido y el estado de sus procesiones y los eventos.

Para alcanzar este objetivo, se plantean los siguientes objetivos específicos:

- Diseñar dicha aplicación permitiendo a los usuarios elegir entre las distintas Semanas Santas dentro del territorio nacional.
- Proporcionar acceso a la información general relacionada con la Semana Santa, incluyendo historia, cofradías o hermandades y elementos representativos.
- Mostrar información detallada sobre los eventos, como nombre, descripción, fechas, horas e itinerarios.
- Añadir funcionalidades de personalización, como la selección de eventos o elementos de interés y la recepción de notificaciones ante cambios o incidencias.
- Implementar un sistema de visualización sobre mapa que permita representar las procesiones y su evolución en tiempo real.
- Desarrollar una solución accesible desde dispositivos móviles que garantice una experiencia de uso adecuada.
- Diseñar una arquitectura escalable que permita la incorporación de nuevas ciudades y eventos sin necesidad de rediseñar el sistema.

1.5. Estructura de la memoria

La memoria se ha organizado en distintos capítulos en los cuales se puede ver la estructura y el desarrollo de dicho proyecto, de forma que se pueda comprender el proyecto de forma clara y completa.

En primer lugar, en este capítulo de introducción se presenta el contexto del proyecto, la motivación que lo impulsa, el estado actual de las distintas soluciones existentes, así como los objetivos que se llevan a cabo.

Después, en el capítulo de planificación se describe la metodología seguida durante el desarrollo de la aplicación, la organización del proyecto, además de las distintas fases de desarrollo y gestión de tareas.

A continuación, en el capítulo de análisis se identifican los actores del sistema, se especifican los requisitos funcionales, de información y no funcionales, y se detallan los casos de uso y el modelo de dominio sobre los que se sustenta el desarrollo de la aplicación.

Seguido por el capítulo de las iteraciones, en el cual se muestran las distintas etapas en las que se ha dividido y repartido el desarrollo de la aplicación, mostrando la evolución que ha ido teniendo el sistema a lo largo del desarrollo del proyecto, con el aspecto de las pantallas implementadas en cada una de ellas.

A continuación, el capítulo de pruebas describe cómo se ha comprobado que el sistema cumple los requisitos: las pruebas automáticas del backend, las simulaciones del seguimiento en tiempo real, las pruebas en directo con el sistema desplegado y las pruebas manuales de aceptación, junto con los errores detectados y su corrección.

El capítulo de conclusiones valora los objetivos conseguidos, plantea las líneas de trabajo futuro y recoge los puntos débiles del sistema.

Por último, los apéndices recogen los casos de uso adicionales, el esquema completo de la base de datos, el manual de despliegue y el manual de usuario.

Capítulo 2

Planificación

2.1. Metodología

Para el desarrollo de este proyecto se ha decidido hacer uso de una metodología **iterativa incremental** [**?**]. Este modelo se ha seleccionado porque permite organizar el proyecto en ciclos o iteraciones sucesivas, donde cada una añade funcionalidad al sistema de forma progresiva. Cada iteración incluye sus propias fases de análisis, diseño, implementación y pruebas, lo que facilita la validación continua del producto y la detección temprana de errores.

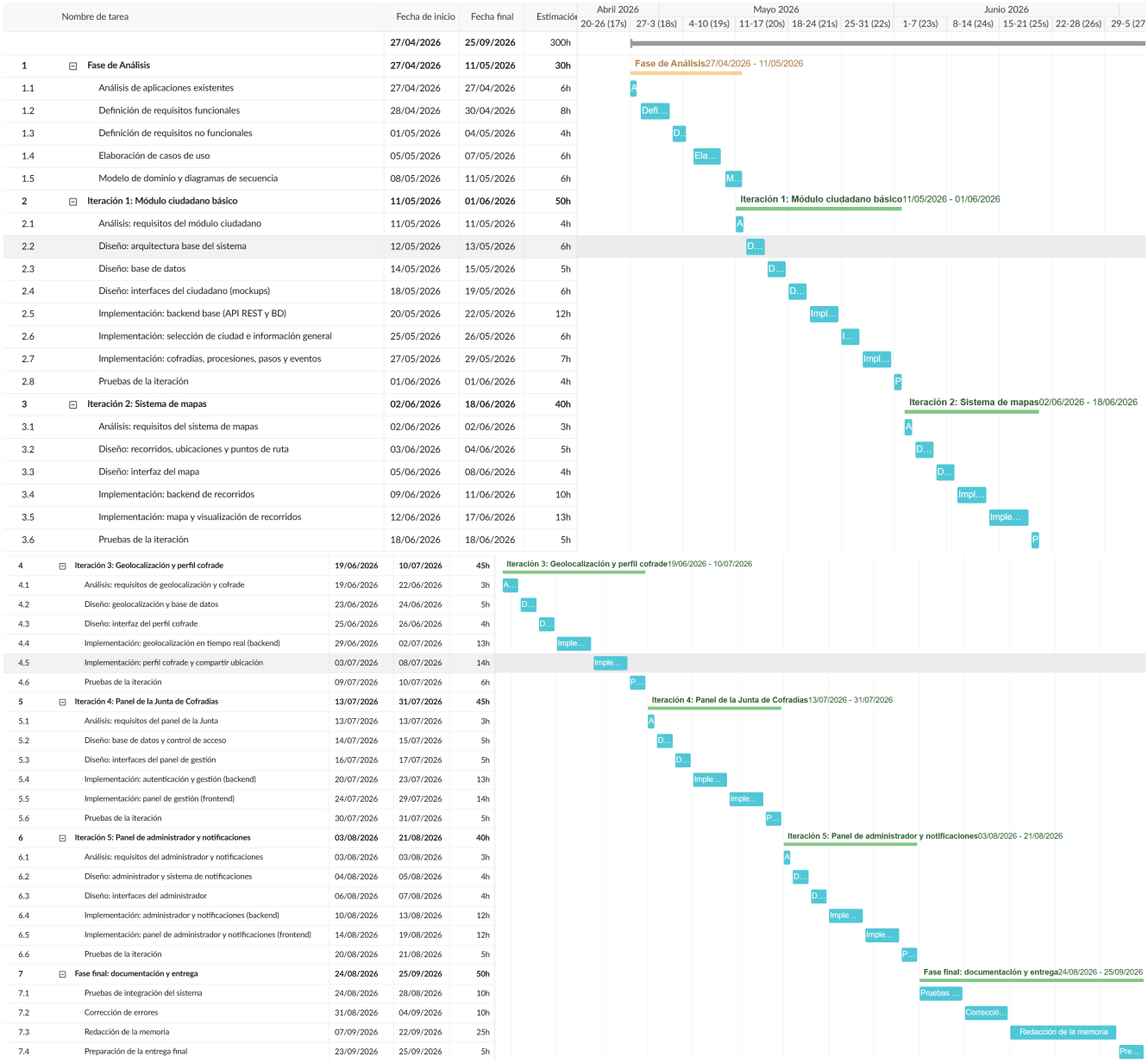
La elección de esta metodología está justificada por la naturaleza del proyecto, ya que permite entregar incrementos funcionales del sistema de manera gradual, comenzando por las funcionalidades básicas del módulo ciudadano y avanzando progresivamente hacia características más complejas como la geolocalización en tiempo real, los paneles de gestión y el sistema de notificaciones. Al ser un proyecto desarrollado por un único desarrollador, este modelo resulta adecuado porque combina la flexibilidad de adaptarse a posibles ajustes durante el desarrollo con una planificación estructurada que facilita el seguimiento del avance.

El proyecto se ha organizado en una **fase inicial de análisis** general, seguida de **cinco iteraciones** que abordan de forma incremental los distintos módulos del sistema, y una **fase final** de documentación, integración y entrega.

2.2. Planificación, Fases e iteraciones

2.2.1. Planificación inicial

Este proyecto consta de tres grandes fases, cuya planificación general se puede observar en el siguiente Diagrama de Gantt [**?**] (Figura 2.1): una primera fase de análisis, una segunda fase de desarrollo iterativo e incremental, organizada en cinco iteraciones, y una tercera y última fase de documentación y entrega.



	Nombre de tarea	Fecha de inicio	Fecha final	Estimación
		27/04/2026	**25/09/2026**	300h
1	**Fase de Análisis**	**27/04/2026**	**11/05/2026**	**30h**
1.1	Análisis de aplicaciones existentes	27/04/2026	27/04/2026	6h
1.2	Definición de requisitos funcionales	28/04/2026	30/04/2026	8h
1.3	Definición de requisitos no funcionales	01/05/2026	04/05/2026	4h
1.4	Elaboración de casos de uso	05/05/2026	07/05/2026	6h
1.5	Modelo de dominio y diagramas de secuencia	08/05/2026	11/05/2026	6h
2	**Iteración 1: Módulo ciudadano básico**	**11/05/2026**	**01/06/2026**	**50h**
2.1	Análisis: requisitos del módulo ciudadano	11/05/2026	11/05/2026	4h
2.2	Diseño: arquitectura base del sistema	12/05/2026	13/05/2026	6h
2.3	Diseño: base de datos	14/05/2026	15/05/2026	5h
2.4	Diseño: interfaces del ciudadano (mockups)	18/05/2026	19/05/2026	6h
2.5	Implementación: backend base (API REST y BD)	20/05/2026	22/05/2026	12h
2.6	Implementación: selección de ciudad e información general	25/05/2026	26/05/2026	6h
2.7	Implementación: cofradías, procesiones, pasos y eventos	27/05/2026	29/05/2026	7h
2.8	Pruebas de la iteración	01/06/2026	01/06/2026	4h
3	**Iteración 2: Sistema de mapas**	**02/06/2026**	**18/06/2026**	**40h**
3.1	Análisis: requisitos del sistema de mapas	02/06/2026	02/06/2026	3h
3.2	Diseño: recorridos, ubicaciones y puntos de ruta	03/06/2026	04/06/2026	5h
3.3	Diseño: interfaz del mapa	05/06/2026	08/06/2026	4h
3.4	Implementación: backend de recorridos	09/06/2026	11/06/2026	10h
3.5	Implementación: mapa y visualización de recorridos	12/06/2026	17/06/2026	13h
3.6	Pruebas de la iteración	18/06/2026	18/06/2026	5h
4	**Iteración 3: Geolocalización y perfil cofrade**	**19/06/2026**	**10/07/2026**	**45h**
4.1	Análisis: requisitos de geolocalización y cofrade	19/06/2026	22/06/2026	3h
4.2	Diseño: geolocalización y base de datos	23/06/2026	24/06/2026	5h
4.3	Diseño: interfaz del perfil cofrade	25/06/2026	26/06/2026	4h
4.4	Implementación: geolocalización en tiempo real (backend)	29/06/2026	02/07/2026	13h
4.5	Implementación: perfil cofrade y compartir ubicación	03/07/2026	08/07/2026	14h
4.6	Pruebas de la iteración	09/07/2026	10/07/2026	6h
5	**Iteración 4: Panel de la Junta de Cofradías**	**13/07/2026**	**31/07/2026**	**45h**
5.1	Análisis: requisitos del panel de la Junta	13/07/2026	13/07/2026	3h
5.2	Diseño: base de datos y control de acceso	14/07/2026	15/07/2026	5h
5.3	Diseño: interfaces del panel de gestión	16/07/2026	17/07/2026	5h
5.4	Implementación: autenticación y gestión (backend)	20/07/2026	23/07/2026	13h
5.5	Implementación: panel de gestión (frontend)	24/07/2026	29/07/2026	14h
5.6	Pruebas de la iteración	30/07/2026	31/07/2026	5h
6	**Iteración 5: Panel de administrador y notificaciones**	**03/08/2026**	**21/08/2026**	**40h**
6.1	Análisis: requisitos del administrador y notificaciones	03/08/2026	03/08/2026	3h
6.2	Diseño: administrador y sistema de notificaciones	04/08/2026	05/08/2026	4h
6.3	Diseño: interfaces del administrador	06/08/2026	07/08/2026	4h
6.4	Implementación: administrador y notificaciones (backend)	10/08/2026	13/08/2026	12h
6.5	Implementación: panel de administrador y notificaciones (frontend)	14/08/2026	19/08/2026	12h
6.6	Pruebas de la iteración	20/08/2026	21/08/2026	5h
7	**Fase final: documentación y entrega**	**24/08/2026**	**25/09/2026**	**50h**
7.1	Pruebas de integración del sistema	24/08/2026	28/08/2026	10h
7.2	Corrección de errores	31/08/2026	04/09/2026	10h
7.3	Redacción de la memoria	07/09/2026	22/09/2026	25h
7.4	Preparación de la entrega final	23/09/2026	25/09/2026	5h

Figura 2.1: Diagrama de Gantt general completo del proyecto

2.2.2. Fase de análisis

Esta primera fase se centra en el análisis de las aplicaciones relacionadas con la Semana Santa existentes, la identificación de sus carencias y áreas de mejora, y en una versión inicial de los requisitos funcionales y no funcionales [**?**] del sistema, además de la elaboración de los casos de uso [**?**] y los distintos diagramas que servirán como base para la realización del desarrollo de la aplicación.

Esta fase comienza con el análisis de las propuestas existentes en el mercado, identificando las características principales de aplicaciones como *El Penitente* [**?**], *Valladolid Cofrade* [**?**] o *sSantaZgz* [**?**].

Seguido de este análisis se realiza la definición de los requisitos funcionales, los cuales describen las funcionalidades concretas que el sistema debe ofrecer, y los requisitos no funcionales, que definen las

condiciones de calidad, rendimiento y seguridad que debe cumplir [**?**], teniendo en cuenta los cuatro perfiles de usuario: ciudadano, cofrade, Junta de Cofradías y administrador. También se elaboran los casos de uso [**?**], representaciones estandarizadas de las interacciones entre los usuarios y el sistema, así como el modelo de dominio y los diagramas de secuencia, que servirán para el desarrollo posterior.

2.2.3. Fase de iteraciones

Esta segunda fase está centrada en el desarrollo de la aplicación siguiendo un enfoque iterativo e incremental [**?**], organizado en cinco iteraciones. En cada una de ellas se repite un mismo ciclo de trabajo formado por las etapas de análisis, diseño, implementación y pruebas, de modo que las funcionalidades correspondientes a esa iteración se analizan, diseñan, implementan y validan antes de darla por finalizada y avanzar a la siguiente. Este enfoque permite ir incorporando las funcionalidades de forma progresiva, obteniendo al final de cada iteración una versión incremental y funcional del sistema. El desglose detallado de tareas y horas de cada iteración puede consultarse en el Diagrama de Gantt (Figura 2.1).

Estas iteraciones abarcan desde las funcionalidades básicas del ciudadano hasta las más complejas, como la geolocalización en tiempo real, es decir, la capacidad de conocer y actualizar la posición geográfica de una procesión de forma continua haciendo uso de la tecnología GPS[1] del móvil, o el sistema de notificaciones (mensajes enviados desde el servidor a los dispositivos de los usuarios sin que estos tengan que abrir la aplicación activamente).

Iteración 1: Módulo ciudadano básico

Esta primera iteración tiene como objetivo establecer la arquitectura base del sistema y desarrollar las funcionalidades fundamentales del perfil de ciudadano, entre las que se incluyen la selección de ciudad, la visualización de la información general de la Semana Santa, el listado de cofradías, procesiones y pasos y la consulta de eventos.

Iteración 2: Sistema de mapas

Esta segunda iteración se centra principalmente en la implementación del sistema de mapas, que permitirá visualizar los distintos recorridos de las procesiones.

Iteración 3: Geolocalización y perfil cofrade

Esta tercera iteración se centra en la implementación completa del sistema de geolocalización en tiempo real, de forma que se pueda conocer y actualizar de manera continua la posición de las procesiones

[1] GPS (Global Positioning System): sistema de navegación por satélite que permite determinar la posición geográfica de un dispositivo en cualquier punto del planeta.

durante su recorrido. Además, se desarrolla el perfil de cofrade con sus correspondientes funcionalidades, incluyendo la posibilidad de compartir la ubicación durante las procesiones.

Iteración 4: Panel de la Junta de Cofradías

Esta cuarta iteración corresponde al desarrollo del panel de gestión para las Juntas de Cofradías, permitiendo administrar la información de la Semana Santa, incluyendo eventos, recorridos, imágenes y participantes. El acceso a este panel está restringido mediante un mecanismo de control de acceso [**?**], que permite diferenciar los permisos y limitaciones de los distintos roles del sistema.

Iteración 5: Panel de administrador y notificaciones

Esta quinta y última iteración se centra en la implementación del panel de administrador global del sistema, así como del sistema de notificaciones, que permitirá a la Junta de Cofradías comunicar a los usuarios cambios o incidencias en las procesiones de forma inmediata, sin necesidad de que estos tengan la aplicación abierta.

2.2.4. Fase final: documentación y entrega

En esta última fase se realizan las pruebas de integración del sistema completo, la corrección de errores detectados, la finalización de la memoria del proyecto y la preparación de la entrega final.

2.3. Análisis de riesgos

Otra parte importante en la planificación de un proyecto de desarrollo de software es el análisis de riesgos. Un riesgo es un evento de incertidumbre que puede afectar de forma positiva o negativa a los objetivos del proyecto [**?**]. Para garantizar el alcance de los objetivos del proyecto, es necesario realizar un análisis de estos riesgos.

Los riesgos identificados se organizan en función de su prioridad, la cual se determina mediante la técnica de análisis de exposición al riesgo (*Risk Exposure*, RE) [**?**], donde el riesgo se calcula mediante un valor numérico que representa la probabilidad e impacto estimados de cada evento. Se define de la siguiente manera:

$$RE = \text{probabilidad} \times \text{impacto} \tag{2.1}$$

La variable probabilidad representa la probabilidad de que el riesgo ocurra en el proyecto (escala de 1 a 10), mientras que el impacto representa la gravedad de sus consecuencias en caso de ocurrir (escala de 1 a 10). En la Tabla 2.1 se muestran los riesgos identificados en el proyecto.

Riesgo	Probabilidad	Impacto	Prioridad
1. Estimaciones de tiempo erróneas	8	7	56
2. Falta de experiencia con las tecnologías del backend propio (Spring Boot)	7	7	49
3. Problemas con la integración de Google Maps	6	8	48
4. Errores durante el desarrollo	7	6	42
5. Retrasos o errores por falta de experiencia con tecnologías móviles	7	6	42
6. Problemas con la geolocalización en tiempo real	5	8	40
7. Cambios en los requisitos	6	6	36
8. Especificaciones ambiguas o poco definidas	5	6	30
9. Problemas con el entorno de desarrollo	4	5	20
10. Indisponibilidad del desarrollador	3	6	18

Tabla 2.1: Riesgos identificados en el proyecto

2.3.1. Planificación de riesgo

Tras realizar la identificación de los riesgos, el siguiente paso es la decisión de qué medidas se deben tomar para reducir la probabilidad de que el riesgo ocurra, de forma que se pueda disminuir su impacto. En las Tablas 2.2 a 2.11 se detallan, para cada uno de los riesgos identificados en la Tabla 2.1, las medidas de reducción propuestas para disminuir su probabilidad y las medidas de contingencia previstas en caso de que finalmente ocurran.

Riesgo	Estimaciones de tiempo erróneas
Reducción	Planificar las tareas con margen por si la estimación ha sido inferior al tiempo necesario. Dividir las tareas grandes en subtareas más pequeñas y manejables.
Contingencia	Cambiar la planificación, reducir el alcance del proyecto priorizando las funcionalidades esenciales.

Tabla 2.2: Riesgo 1: Estimaciones de tiempo erróneas

Riesgo	Problemas con la integración de Google Maps
Reducción	Realizar pruebas de concepto tempranas con la API de Google Maps. Documentarse sobre las limitaciones y cuotas del servicio.
Contingencia	Considerar alternativas como OpenStreetMap [**?**] o Mapbox en caso de problemas persistentes.

Tabla 2.3: Riesgo 2: Problemas con la integración de Google Maps

2.4. Estimación de costes

A continuación se presenta el presupuesto estimado del proyecto, calculado como si este fuese desarrollado por un equipo profesional completo, siguiendo los perfiles habituales en el desarrollo de una

Riesgo	Errores durante el desarrollo
Reducción	Verificar que la funcionalidad está implementada correctamente antes de dar por finalizado el incremento. Realizar pruebas unitarias[2]Pruebas unitarias: pruebas que verifican el correcto funcionamiento de componentes individuales del código de forma aislada.
Contingencia	Volver al último punto correctamente probado y continuar desde ahí. Utilizar control de versiones [**?**] para recuperar estados anteriores del código.

Tabla 2.4: Riesgo 3: Errores durante el desarrollo

Riesgo	Retrasos o errores por falta de experiencia con tecnologías móviles
Reducción	Formarse en las tecnologías clave antes de proceder al desarrollo. Consultar documentación oficial y tutoriales.
Contingencia	Aprovechar la experiencia del tutor para solventar los posibles problemas. Buscar soluciones en comunidades de desarrolladores.

Tabla 2.5: Riesgo 4: Retrasos o errores por falta de experiencia con tecnologías móviles

Riesgo	Problemas con la geolocalización en tiempo real
Reducción	Investigar las mejores prácticas para implementar geolocalización en aplicaciones móviles. Realizar prototipos tempranos para validar la viabilidad técnica.
Contingencia	Implementar un modo de actualización manual de la ubicación como alternativa temporal.

Tabla 2.6: Riesgo 5: Problemas con la geolocalización en tiempo real

Riesgo	Cambios en los requisitos
Reducción	El desarrollo iterativo permite adaptar el sistema a cambios de forma gradual.
Contingencia	Reevaluar las prioridades y ajustar el alcance del proyecto según las nuevas necesidades.

Tabla 2.7: Riesgo 6: Cambios en los requisitos

Riesgo	Especificaciones genéricas
Reducción	Reunir toda la información posible antes de abordar una tarea. Clarificar requisitos con el tutor.
Contingencia	Prototipar en caso de ser necesario para validar la interpretación de los requisitos.

Tabla 2.8: Riesgo 7: Especificaciones genéricas

aplicación móvil de estas características. Se trata de una estimación teórica con fines de planificación y no de los costes reales asumidos por el autor, quien ha desarrollado el TFG de forma individual.

Riesgo	Problemas con el entorno de desarrollo
Reducción	Utilizar un repositorio remoto (GitHub [**?**]) para almacenar copias de seguridad del código.
Contingencia	Utilizar una máquina de respaldo o entorno de desarrollo alternativo.

Tabla 2.9: Riesgo 8: Problemas con el entorno de desarrollo

Riesgo	Indisposición del desarrollador
Reducción	Planificar las tareas con margen por si es necesario extender el tiempo para completar el incremento.
Contingencia	Realizar horas extra en caso necesario y ajustar la planificación.

Tabla 2.10: Riesgo 9: Indisposición del desarrollador

Riesgo	Falta de experiencia con las tecnologías del backend propio (Spring Boot, Spring Security, JPA/Hibernate)
Reducción	Realizar una prueba de concepto acotada (*spike*) antes de comenzar la implementación completa, construyendo un único recurso extremo a extremo (entidad, autenticación JWT y *endpoint* protegido por rol) para validar el ritmo de desarrollo real con la tecnología. Consultar la documentación oficial y limitar el alcance del backend a lo estrictamente necesario para los requisitos, evitando patrones o herramientas adicionales que no aporten valor al proyecto (por ejemplo, no se contemplan microservicios ni colas de mensajería).
Contingencia	Si el retraso es significativo, reducir el alcance del backend priorizando los endpoints imprescindibles para cada iteración y posponiendo funcionalidades secundarias a iteraciones posteriores, tal y como permite la metodología iterativa incremental (Sección 2.1).

Tabla 2.11: Riesgo 10: Falta de experiencia con las tecnologías del backend propio

2.4.1. Costes de personal

Un proyecto de estas características, correspondiente al desarrollo completo de una aplicación móvil sobre la Semana Santa, requeriría en un entorno profesional un equipo multidisciplinar. Se han considerado los siguientes seis perfiles, entre los que se reparten las 300 horas estimadas de duración del proyecto:

- **Jefe de Proyecto**: coordinación general, planificación y seguimiento.
- **Analista Funcional**: recogida y especificación de requisitos, análisis del dominio.
- **Diseñador UX/UI**: diseño de la experiencia e interfaz de usuario.
- **Desarrollador Backend**: implementación del servidor, base de datos y API.

- **Desarrollador Frontend/Mobile**: implementación de la aplicación Android.
- **Tester/QA**: pruebas funcionales, de regresión y control de calidad.

Para cada perfil se ha calculado el coste por hora a partir del sueldo bruto anual medio en España, incrementado en un 30 % en concepto de cotizaciones a la Seguridad Social a cargo de la empresa, y dividido entre 1.800 horas anuales de jornada completa. La Tabla 2.12 recoge el reparto de horas y el coste asociado a cada perfil.

Perfil	Horas	Coste/hora	Coste total
Jefe de Proyecto [?]	45	33€	1.485,00€
Analista Funcional [?]	45	27€	1.215,00€
Diseñador UX/UI [?]	30	27€	810,00€
Desarrollador Backend [?]	75	26€	1.950,00€
Desarrollador Frontend/Mobile [?]	75	19€	1.425,00€
Tester/QA [?]	30	17€	510,00€
Total	**300**		**7.395,00€**

Tabla 2.12: Reparto de horas y coste de personal por perfil

2.4.2. Costes de equipo y amortización

El equipo del proyecto necesitaría un ordenador portátil por persona, además de un dispositivo Android dedicado a las pruebas. Se considera una duración total del proyecto de 5 meses.

- **Ordenadores portátiles**: 6 unidades valoradas en 1.000€ cada una, con una vida útil estimada de 5 años (60 meses). La amortización mensual por unidad es de 16,66€, lo que supone 83,30€ por portátil durante los 5 meses del proyecto, y 500,00€ en total para los 6 puestos de trabajo.
- **Dispositivo Android de pruebas**: 1 unidad valorada en 300€, con una vida útil estimada de 3 años (36 meses). La amortización mensual es de 8,33€, lo que supone 41,65€ para los 5 meses del proyecto.

Ahora bien, estos importes corresponden a la ocupación de los equipos durante los 5 meses completos del proyecto. Dado que el trabajo real asciende a 300 horas, frente a una capacidad disponible de 750 horas en ese periodo (150 horas/mes de jornada completa, según el criterio de 1.800 horas anuales aplicado en el apartado anterior), el resto del tiempo los equipos podrían destinarse a otro uso. Por ello, la amortización se imputa de forma proporcional al uso real del proyecto (300/750 = 40 %).

El coste de amortización así calculado asciende a **216,66€**, tal y como recoge la Tabla 2.13.

2.4.3. Costes de licencias de software

Buena parte de las herramientas empleadas disponen de planes gratuitos o licencias universitarias suficientes para un proyecto de este tamaño. Sin embargo, un equipo profesional de 6 personas necesitaría

Recurso	Valor	Vida útil	Amort. (5 meses)	Amort. en el proyecto (300h, 40 %)
Portátiles (x6)	6.000,00€	5 años	500,00€	200,00€
Dispositivo Android (x1)	300,00€	3 años	41,65€	16,66€
Total amortización			541,65€	**216,66€**

Tabla 2.13: Amortización de los recursos de equipo, prorrateada al uso real del proyecto

además herramientas colaborativas de pago para la gestión del proyecto y el diseño. La Tabla 2.14 recoge el coste y la fuente de cada licencia.

El backend se ha desplegado en un servidor virtual cedido por la Escuela de Ingeniería Informática de la Universidad de Valladolid, sin coste para el proyecto. Para reflejar lo que costaría fuera de ese entorno, se ha valorado un servidor virtual comercial de prestaciones similares (Hetzner Cloud CX23: 2 vCPU, 4 GB de RAM y 40 GB de disco NVMe), a 5,99€ al mes sin IVA con la dirección IPv4 pública incluida [**?**], lo que supone 29,95€ para los 5 meses del proyecto.

Herramienta	Coste	Fuente
Android Studio	0,00€	[**?**]
Visual Studio Code	0,00€	[**?**]
Overleaf (plan gratuito)	0,00€	—
Expo Push Notifications (plan gratuito)	0,00€	[**?**]
Expo Application Services, EAS Build (plan gratuito)	0,00€	[**?**]
Servidor virtual (equivalente: Hetzner Cloud CX23, 5 meses)	29,95€	[**?**]
Google Maps API (crédito gratuito)	0,00€	[**?**]
GitHub (plan gratuito)	0,00€	[**?**]
Cuenta de desarrollador Google Play (pago único)	23,00€	[**?**]
Figma Professional (1 licencia, 5 meses)	69,00€	[**?**]
Jira Software Standard (6 usuarios, 5 meses)	218,40€	[**?**]
Total licencias	**340,35€**	

Tabla 2.14: Coste de licencias de software

2.4.4. Otros costes indirectos

Un equipo de 6 personas trabajando de forma presencial o híbrida necesitaría un espacio de oficina. Se ha tomado como referencia el alquiler de un despacho de 22 m^2 en un espacio de coworking en el centro de Valladolid, con todos los suministros incluidos (calefacción, refrigeración, electricidad, wifi y limpieza), por un importe de 475€ al mes [**?**]. Para los 5 meses de duración del proyecto, el coste estructural asciende a 2.375,00€.

Al igual que con la amortización de equipos, este importe corresponde a la disponibilidad del espacio durante los 5 meses completos, mientras que el uso real del proyecto es de 300 horas frente a una capacidad de 750 horas en ese periodo (40 %); el resto del tiempo el espacio podría destinarse a otro uso. Aplicando este mismo criterio de proporcionalidad, el coste estructural imputado al proyecto asciende a **950,00€** (2.375,00€ × 40 %).

2.4.5. Coste total

La Tabla 2.15 resume el presupuesto completo del proyecto, considerando personal, amortización de equipo, licencias de software y costes estructurales —estos dos últimos prorrateados a las 300 horas reales de uso— para una duración de 300 horas repartidas en 5 meses.

Elemento	**Coste**
Personal (Tabla 2.12)	7.395,00€
Amortización de equipo, prorrateada (Tabla 2.13)	216,66€
Licencias de software (Tabla 2.14)	340,35€
Costes estructurales, prorrateados (oficina y suministros)	950,00€
Total	**8.902,01€**

Tabla 2.15: Costes totales del proyecto

2.5. Seguimiento del proyecto

Nota: esta sección se completará al finalizar el desarrollo del proyecto, comparando la planificación inicial con la duración real del proyecto. Se comentará que ha habido distintas replanificaciones debido a diversos recortes de tiempo que han surgido a lo largo de los meses de desarrollo.

Capítulo 3

Análisis

En este capítulo se recoge el análisis del sistema: los actores, los requisitos, los casos de uso y el modelo de dominio. Como a lo largo del desarrollo algunos de estos elementos se fueron ajustando, para facilitar su comprensión se presenta el análisis final de la aplicación, es decir, tal y como ha quedado en su versión final.

3.1. Actores del sistema

En el desarrollo de cualquier sistema software es fundamental identificar y definir los actores que interactuarán con la plataforma. Los actores representan a las entidades externas, ya sean personas u otros sistemas, que se comunican con la aplicación y hacen uso de sus funcionalidades.

En el caso de esta aplicación, orientada a la gestión y consulta de información sobre la Semana Santa, se han identificado cuatro actores principales: el ciudadano, el cofrade, la Junta de Cofradías y el administrador. Cada uno de ellos dispone de un nivel de acceso y unas funcionalidades diferenciadas, que se detallan a continuación y que sirven de base para la definición de los requisitos funcionales (Sección 3.2).

3.1.1. Ciudadano

El ciudadano es el actor principal de la aplicación y representa a cualquier persona interesada en seguir la Semana Santa, ya sea de forma presencial en la ciudad o de manera remota. No requiere registro previo para utilizar la aplicación.

Puede consultar la información detallada de cofradías, eventos, procesiones y pasos, realizar búsquedas y aplicar filtros, y visualizar en tiempo real la posición de las procesiones mediante geolocalización. Además, puede marcar contenido como favorito y recibir notificaciones sobre la ciudad que tiene seleccionada, activables o no desde el propio permiso de notificaciones del dispositivo.

3.1.2. Cofrade

El cofrade es la persona que comparte su ubicación en tiempo real mientras está procesionando. Accede a la aplicación con el código de acceso de su cofradía, generado por la Junta de Cofradía correspondiente, escaneando su código QR o escribiéndolo a mano, sin necesidad de registrarse.

Su única función es compartir su ubicación durante una procesión en curso de su cofradía: puede empezar a compartirla cuando la procesión comienza y dejar de hacerlo en cualquier momento, y la aplicación deja de compartirla automáticamente cuando la procesión finaliza. Esta ubicación es la que alimenta la geolocalización de la procesión que consulta el resto de usuarios.

3.1.3. Junta de Cofradías

La Junta de Cofradías es el actor responsable de la gestión del contenido asociado a la Semana Santa de su ciudad dentro de la aplicación. Accede a un panel de gestión desde el que puede crear y editar cofradías, eventos y procesiones. El recorrido de cada procesión se importa desde un archivo GPX, preparado previamente con una herramienta externa de trazado de rutas: de ese archivo la aplicación solo toma la secuencia ordenada de coordenadas, que forma el trazado de la procesión, sin ningún nombre ni información adicional. Una vez importado, la Junta revisa ese recorrido desde la propia aplicación y marca a mano cuáles de esos puntos son puntos de interés[1].

Asimismo, se encarga de actualizar el estado de las procesiones y de los eventos, de gestionar la información de ambos y de los pasos de cada cofradía, y de crear las notificaciones que reciben los ciudadanos y cofrades. También es responsable de generar, para cada cofradía, su propio código de acceso, el cual permite a los cofrades compartir su ubicación en la procesión de cofradía correspondiente.

3.1.4. Administrador

El administrador es el actor con mayor nivel de privilegios dentro del sistema y se encarga de su configuración y mantenimiento a nivel global. Su principal responsabilidad es la gestión de las ciudades disponibles en la aplicación, así como de la Junta de Cofradías de cada una, dando de alta, editando o dando de baja aquellas en las que el sistema ofrece cobertura. Además gestiona a los miembros de las distintas Juntas de Cofradías –dando de alta, editando o dando de baja su acceso, y resolviendo las solicitudes de reactivación de los miembros desactivados– y su propio perfil.

La Tabla 3.1 resume los cuatro actores identificados y su rol dentro del sistema.

[1] Punto de interés: lugar concreto del recorrido de una procesión al que se le da un significado propio dentro de la celebración, como un encuentro con otra procesión, la entrada a una iglesia de la que la procesión sale más tarde, o una parada para una lectura o un cántico.

Actor	Rol
Ciudadano	Consulta información y sigue la Semana Santa sin necesidad de registro.
Cofrade	Ciudadano perteneciente a una cofradía; comparte su ubicación durante las procesiones.
Junta de Cofradías	Gestiona todo el contenido de la Semana Santa de su ciudad: cofradías, eventos, procesiones, recorridos y notificaciones.
Administrador	Gestiona la configuración global del sistema, en particular las ciudades disponibles y sus juntas de cofradias.

Tabla 3.1: Resumen de los actores del sistema

3.2. Requisitos

3.2.1. Requisitos funcionales

Los requisitos funcionales definen las diferentes funciones del sistema, además de cómo debe ser su comportamiento en ciertas situaciones. Cada requisito incluye, junto a su descripción, el nivel de prioridad asignado (Alta, Media o Baja), que indica su importancia relativa dentro del proyecto.

Para facilitar su lectura, los requisitos se han agrupado según el perfil de usuario (Sección 3.1) al que van dirigidos –ciudadano, cofrade, Junta de Cofradías o administrador–. Dentro de cada grupo se han ordenado de mayor a menor prioridad, salvo en el del ciudadano: al ser el grupo más numeroso, sus requisitos se listan siguiendo en su lugar el flujo de uso habitual de la aplicación (desde que se accede hasta la geolocalización en tiempo real y las notificaciones), para que el lector pueda seguirlos en el mismo orden en que se encontraría con ellos.

Ciudadano

- **RF-01** *(Prioridad: Alta)* El sistema debe permitir el uso de la aplicación como ciudadano sin la necesidad de realizar un registro previo.
- **RF-02** *(Prioridad: Alta)* El sistema debe permitir al usuario seleccionar la ciudad de la que desea consultar la Semana Santa.
- **RF-03** *(Prioridad: Alta)* El sistema debe solicitar permiso a los usuarios para acceder a su ubicación en tiempo real.
- **RF-04** *(Prioridad: Media)* El sistema debe seleccionar automáticamente la ciudad más cercana en función de la ubicación del usuario.
- **RF-05** *(Prioridad: Alta)* El sistema debe mostrar la información detallada de las cofradías.
- **RF-06** *(Prioridad: Alta)* El sistema debe mostrar la información detallada de los eventos.
- **RF-07** *(Prioridad: Alta)* El sistema debe mostrar la información detallada de las procesiones.

- **RF-08** *(Prioridad: Alta)* El sistema debe mostrar la información detallada de los pasos.
- **RF-09** *(Prioridad: Alta)* El sistema debe mostrar un calendario con los días de la Semana Santa, indicando en cada día si hay eventos o procesiones programadas.
- **RF-10** *(Prioridad: Alta)* El sistema debe permitir a los usuarios seleccionar un día concreto del calendario para consultar el listado de procesiones y eventos programados ese día.
- **RF-11** *(Prioridad: Media)* El sistema debe permitir a los usuarios acceder, desde un menú principal, a los listados completos de procesiones, pasos, eventos y cofradías.
- **RF-12** *(Prioridad: Alta)* El sistema debe permitir a los usuarios realizar búsquedas sobre eventos, procesiones, pasos o cofradías específicas.
- **RF-13** *(Prioridad: Media)* El sistema debe permitir filtrar eventos y procesiones por día y por estado.
- **RF-14** *(Prioridad: Media)* El sistema debe permitir a los usuarios marcar cofradías, procesiones, eventos y pasos como favoritos, y consultarlos reunidos en un listado propio de favoritos.
- **RF-15** *(Prioridad: Media)* El sistema debe permitir al usuario interactuar con el mapa (zoom, desplazamiento, ...).
- **RF-16** *(Prioridad: Alta)* Sin que el usuario tenga que seleccionar ninguna cofradía, evento o procesión, el sistema debe mostrar por defecto en el mapa las procesiones que están actualmente en curso.
- **RF-17** *(Prioridad: Baja)* El sistema debe mostrar en el mapa los puntos de interés del recorrido de las procesiones que se estén mostrando.
- **RF-18** *(Prioridad: Alta)* El sistema debe permitir visualizar la posición de las procesiones en tiempo real mediante geolocalización.
- **RF-19** *(Prioridad: Baja)* El sistema debe poder sugerir rutas hacia las ubicaciones más relevantes de una procesión (inicio, puntos de interés, fin, ...).
- **RF-20** *(Prioridad: Baja)* El sistema debe poder generar rutas entre dos puntos evitando las procesiones activas.
- **RF-21** *(Prioridad: Alta)* El sistema debe enviar a cada usuario las notificaciones de la ciudad que tiene seleccionada en la aplicación (RI-12), siempre que no haya desactivado su recepción (RF-22).
- **RF-22** *(Prioridad: Media)* El sistema debe permitir al usuario desactivar la recepción de notificaciones a través del propio permiso de notificaciones del dispositivo, mostrando en su perfil si las tiene activadas y dando acceso directo a los ajustes del dispositivo para cambiarlo.

Cofrade

- **RF-23** *(Prioridad: Alta)* El sistema debe permitir a los cofrades acceder al modo cofrade, desde el inicio de sesión, escaneando el código QR del código de acceso de su cofradía generado por la Junta de Cofradías o escribiendo el código a mano, sin necesidad de registrarse.

- **RF-24** *(Prioridad: Alta)* El sistema debe permitir a los cofrades compartir su ubicación en tiempo real únicamente mientras una procesión de su cofradía esté en curso –eligiendo cuál si hay varias en curso a la vez–, y dejar de compartirla en cualquier momento. El sistema debe dejar de compartirla automáticamente, informando al cofrade del motivo, cuando la procesión finalice, cuando se revoque su código de acceso o cuando caduque su sesión. Si el cofrade concede el permiso de ubicación permanente, la ubicación se debe seguir enviando aunque la pantalla esté bloqueada; en caso contrario, solo mientras la aplicación esté abierta.
- **RF-25** *(Prioridad: Media)* El sistema debe descartar, sin guardarlas ni representarlas, las posiciones recibidas de los cofrades que se encuentren a más de 50 metros del recorrido de la procesión.

Miembro de la Junta de Cofradías

- **RF-26** *(Prioridad: Media)* El sistema debe permitir al miembro de la Junta de Cofradías iniciar sesión con su correo electrónico y su contraseña. Al ser dado de alta, el miembro recibe por correo electrónico una contraseña provisional generada por el sistema, y el sistema debe obligarle a sustituirla por una propia la primera vez que inicie sesión, sin permitirle acceder al panel de gestión ni realizar ningún cambio hasta hacerlo.
- **RF-27** *(Prioridad: Media)* El sistema debe permitir al miembro de la Junta de Cofradías gestionar la información de su propio perfil.
- **RF-28** *(Prioridad: Alta)* El sistema debe permitir al miembro de la Junta de Cofradías crear y eliminar cofradías, eventos y procesiones.
- **RF-29** *(Prioridad: Alta)* El sistema debe permitir al miembro de la Junta de Cofradías editar información de cofradías, eventos, procesiones y de la Semana Santa de su ciudad.
- **RF-30** *(Prioridad: Media)* El sistema debe permitir al miembro de la Junta de Cofradías gestionar (dar de alta, editar y dar de baja) los pasos que pertenecen a cada una de las cofradías de su ciudad –un paso pertenece siempre a una única cofradía (RI-10), aunque después pueda participar en varios eventos y procesiones de esa misma cofradía–.
- **RF-31** *(Prioridad: Media)* El sistema debe permitir al miembro de la Junta de Cofradías emitir, cuando lo necesite, un código de acceso para una cofradía, para que sus cofrades puedan acceder al modo cofrade; el valor del código lo genera automáticamente el propio sistema, sin que tenga que escribirlo a mano, y el sistema debe mostrarlo también como un código QR, junto al texto del código. Cada código debe caducar 24 horas después de emitirse. También debe permitirle revocar un código emitido, de modo que deje de dar acceso al modo cofrade.
- **RF-32** *(Prioridad: Alta)* El sistema debe permitir al miembro de la Junta de Cofradías importar el recorrido de una procesión desde un archivo GPX, del que se toma la secuencia ordenada de coordenadas que forma el trazado.
- **RF-33** *(Prioridad: Baja)* El sistema debe permitir al miembro de la Junta de Cofradías marcar como punto de interés cualquier punto del recorrido importado de una procesión, indicando su tipo, su nombre y, opcionalmente, una descripción y una imagen, y editar después esa información.

- **RF-34** *(Prioridad: Alta)* El sistema debe actualizar automáticamente el estado de las procesiones y de los eventos según su horario: deben pasar a en curso al llegar su hora de inicio y a finalizado al llegar su hora de fin. Además, el sistema debe permitir al miembro de la Junta de Cofradías corregir ese estado a mano (programado, en curso, finalizado o cancelado), salvo cuando el cambio contradiga el horario –volver a programado una vez pasada la hora de inicio, o a en curso una vez pasada la hora de fin–, en cuyo caso debe indicarle que modifique la hora en lugar del estado.
- **RF-35** *(Prioridad: Alta)* Cuando una procesión o un evento pase a en curso o a finalizado, ya sea automáticamente o porque lo cambie a mano el miembro de la Junta de Cofradías, el sistema debe generar y enviar la notificación correspondiente (RI-08) sin que tenga que crearla. Cuando el cambio sea a cancelado, el sistema no debe avisar por sí solo, sino abrir a continuación la creación de una notificación de cancelación para que el miembro de la Junta indique el motivo (RF-36).
- **RF-36** *(Prioridad: Alta)* El sistema debe permitir al miembro de la Junta de Cofradías crear notificaciones dirigidas a todos los usuarios sobre incidencias o cambios en eventos y procesiones.

Administrador

- **RF-37** *(Prioridad: Alta)* El sistema debe permitir al administrador gestionar (dar de alta, editar y dar de baja) las ciudades disponibles.
- **RF-38** *(Prioridad: Alta)* El sistema debe permitir al administrador gestionar (dar de alta, editar y dar de baja) las Juntas de Cofradías del sistema.
- **RF-39** *(Prioridad: Alta)* El sistema debe permitir al administrador gestionar (dar de alta, editar, desactivar y dar de baja) a los miembros de las Juntas de Cofradías. Al dar de alta a un miembro, el sistema debe generar una contraseña provisional y enviársela por correo electrónico junto a un mensaje de bienvenida. Mientras el miembro no haya cambiado esa contraseña, el administrador debe verlo como invitación pendiente y poder reenviarle la invitación, lo que genera una contraseña provisional nueva e invalida la anterior.
- **RF-40** *(Prioridad: Media)* El sistema debe permitir al administrador gestionar la información de su propio perfil.
- **RF-41** *(Prioridad: Media)* El sistema debe permitir a un miembro desactivado de una Junta de Cofradías solicitar su reactivación, y al administrador aceptarla o rechazarla.

3.2.2. Requisitos de información

Los requisitos de información definen los datos que el sistema debe almacenar y gestionar para su correcto funcionamiento. Al igual que en los requisitos funcionales, se indica la prioridad de cada uno de ellos, ordenados de mayor a menor prioridad. En este caso no se agrupan por perfil de usuario, ya que describen el modelo de datos del sistema en su conjunto y no están asociados a un único actor.

- **RI-01** *(Prioridad: Alta)* El sistema debe almacenar la información sobre las ciudades:

  - Identificador.
  - Nombre.
  - Comunidad autónoma.
  - Provincia.
  - Historia.
  - Patrimonio.
  - Si está activa (visible para el ciudadano) o no.
  - Latitud y longitud (opcionales), para poder preseleccionar la ciudad más cercana al Ciudadano por GPS.
  - Junta de Cofradías que la gestiona.

- **RI-02** *(Prioridad: Alta)* El sistema debe almacenar la información sobre las Juntas de Cofradías:
  - Identificador.
  - Nombre.
  - Email.
  - Teléfono.
  - Si está activa o desactivada (una Junta desactivada no puede hacer cambios).
  - Ciudad que gestiona (relación 1:1; no agrupa directamente a las cofradías de esa ciudad, ver RI-04).
  - Miembros que la componen.

- **RI-03** *(Prioridad: Alta)* El sistema debe almacenar la información sobre los usuarios con credenciales propias –miembros de Junta de Cofradías y administradores; el ciudadano y el cofrade nunca se registran, ver Sección **??**–:
  - Identificador.
  - Email.
  - Contraseña, como *hash* (nunca en texto plano).
  - Fecha de ingreso.
  - Si es miembro de Junta de Cofradías, además: nombre, teléfono, Junta de Cofradías a la que pertenece, si está activo o desactivado, si sigue usando la contraseña provisional que se le generó al darlo de alta, y si tiene una solicitud de reactivación pendiente.
  - Si es administrador, además: nombre y teléfono (opcionales, se completan al editar su perfil). Un administrador no está atado a ninguna ciudad en concreto: cualquier administrador puede gestionar cualquier ciudad.

- **RI-04** *(Prioridad: Alta)* El sistema debe almacenar la información sobre las cofradías/hermandades:
  - Identificador.
  - Nombre.

  - Historia.
  - Web oficial.
  - Imagen.
  - Fecha de creación.
  - Si está activa (visible para el ciudadano) o todavía en preparación.
  - Ciudad a la que pertenece –nunca a la Junta de Cofradías que la gestiona, ver Apéndice **??**–.
  - Pasos que posee, y eventos y procesiones en los que participa.

- **RI-05** *(Prioridad: Alta)* El sistema debe almacenar la información sobre los eventos:
  - Identificador.
  - Nombre.
  - Historia.
  - Tradición.
  - Fecha y hora de inicio.
  - Fecha y hora de fin (opcional; sin ella, el evento no se finaliza automáticamente).
  - Estado (programado, en curso, finalizado o cancelado).
  - Web oficial.
  - Ubicación en la que se celebra.
  - Cofradías que participan en él (puede ser más de una).
  - Pasos que participan en él.
- **RI-06** *(Prioridad: Alta)* El sistema debe almacenar el histórico de posiciones que comparten los cofrades durante una procesión, como lecturas **anónimas** –sin ningún dato que identifique a la persona que las envía, ver Sección **??**–:
  - Identificador.
  - Latitud y longitud.
  - Instante de la lectura.
  - Procesión a la que pertenece.
- **RI-07** *(Prioridad: Alta)* El sistema debe almacenar la información sobre los recorridos:
  - Identificador.
  - Distancia total.
  - Tiempo estimado.
  - Puntos de ruta que lo componen, cada uno con su orden dentro del recorrido y la hora prevista de paso.
- **RI-08** *(Prioridad: Alta)* El sistema debe almacenar la información sobre las notificaciones:
  - Identificador.

  - Título.
  - Mensaje.
  - Fecha de creación.
  - Fecha de expiración (opcional).
  - Tipo (inicio, fin, incidencia, cambio de horario, cancelación o actualización).
  - Prioridad (baja, media o alta; obligatoria salvo en las de tipo inicio y fin, que genera el propio sistema).
  - Ciudad a la que pertenece.

- **RI-09** *(Prioridad: Media)* Al ser una procesión un tipo concreto de evento, el sistema debe almacenar sobre ella, además de toda la información de RI-05 (incluidos los pasos que participan en ella):
  - Recorrido asociado (opcional hasta que se importe).
  - Histórico de posiciones anónimas de sus cofrades (RI-06), a partir del cual se calcula al vuelo su posición actual mientras está en curso.
  - Progreso alcanzado, a lo largo del recorrido, por el cofrade más adelantado y por el más atrasado que están compartiendo ubicación, para pintar la estela en vivo de la procesión.
- **RI-10** *(Prioridad: Media)* El sistema debe almacenar la información sobre los pasos:
  - Identificador.
  - Nombre.
  - Historia.
  - Análisis artístico.
  - Imagen.
  - Cofradía a la que pertenece.
  - Eventos y procesiones en los que participa.
- **RI-11** *(Prioridad: Media)* El sistema debe almacenar la información sobre los puntos de un recorrido y demás ubicaciones de referencia utilizadas en el mapa:
  - Latitud.
  - Longitud.
  - Dirección (opcional).
  - Si el punto se destaca como punto de interés (un encuentro, la entrada a una iglesia, una parada para una lectura u oración, un monumento...), además: tipo (monumento, iglesia, encuentro, oración, entrada de procesión o salida de procesión), nombre, descripción e imagen.
- **RI-12** *(Prioridad: Media)* El sistema debe almacenar la información sobre los dispositivos registrados para recibir notificaciones *push*:
  - Identificador.

  - Token de Expo Push del dispositivo.
  - Ciudad de la que quiere recibir notificaciones.
  - Fecha de registro y de última actualización.

- **RI-13** *(Prioridad: Media)* El sistema debe almacenar la información sobre los códigos de acceso de las cofradías:
  - Identificador.
  - Código, generado automáticamente por el sistema y único.
  - Estado (emitido, validado –ya se ha usado al menos una vez para entrar– o revocado).
  - Fecha de emisión y fecha de caducidad (24 horas después de la emisión).
  - Cofradía a la que da acceso.

3.2.3. Requisitos no funcionales

Los requisitos no funcionales describen las características de calidad del sistema, así como restricciones sobre su funcionamiento. También se indica la prioridad de cada uno de ellos, ordenados de mayor a menor prioridad. Al igual que los requisitos de información, no se agrupan por perfil de usuario, ya que son de aplicación transversal a todo el sistema.

- **RNF-01** *(Prioridad: Alta)* El sistema debe proteger los datos de los usuarios, especialmente la información de ubicación: las posiciones que comparten los cofrades deben guardarse de forma anónima, sin ningún dato que identifique a quien las envía (RI-06), y las contraseñas no deben almacenarse en texto plano (RI-03).
- **RNF-02** *(Prioridad: Alta)* La aplicación debe ser accesible y fácil de usar para usuarios sin experiencia previa.
- **RNF-03** *(Prioridad: Alta)* El sistema debe implementar mecanismos de control de acceso que diferencien entre ciudadanos, cofrades, miembros de la junta de cofradías y administrador.
- **RNF-04** *(Prioridad: Alta)* El sistema debe ser compatible tanto con dispositivos Android como con dispositivos iOS.
- **RNF-05** *(Prioridad: Alta)* El sistema debe actualizar la posición de las procesiones con una frecuencia mínima de 30 segundos[2].
- **RNF-06** *(Prioridad: Media)* El sistema debe funcionar correctamente en dispositivos móviles con conexión a internet. Sin conexión, la aplicación debe seguir mostrando, durante un máximo de 24 horas desde que se consultaron, la lista de ciudades, la información de la ciudad y de las cofradías ya visitadas, y la ubicación de los eventos y el recorrido de las procesiones ya consultados. El resto de funcionalidades, como la geolocalización en tiempo real, las notificaciones o la consulta de procesiones y eventos, requieren conexión.

[2]Este intervalo se ha elegido como equilibrio entre ofrecer una posición suficientemente precisa para el usuario y evitar un consumo excesivo de batería y datos móviles en los dispositivos de los cofrades que comparten su ubicación.

- **RNF-07** *(Prioridad: Media)* El sistema debe gestionar de forma eficiente la carga del mapa, garantizando tiempos de respuesta menores a 10 segundos[3].

- **RNF-08** *(Prioridad: Media)* El sistema debe ser capaz de enviar notificaciones de manera fiable y en tiempo adecuado (menos de 60 segundos de latencia).

3.3. Casos de uso

Los casos de uso son los escenarios detallados que describen cómo interactúan los diferentes usuarios con la aplicación, desde la consulta de información hasta la gestión administrativa de la Semana Santa. Debido a que los usuarios pueden comportarse con diferentes roles (ciudadano, cofrade, miembro de la Junta de Cofradías o administrador), es importante diferenciar los casos de uso entre cada uno de ellos. Para ello se han dividido los mismos en cuatro diagramas (ver figuras 3.1, 3.2, 3.3 y 3.4), uno para cada rol.

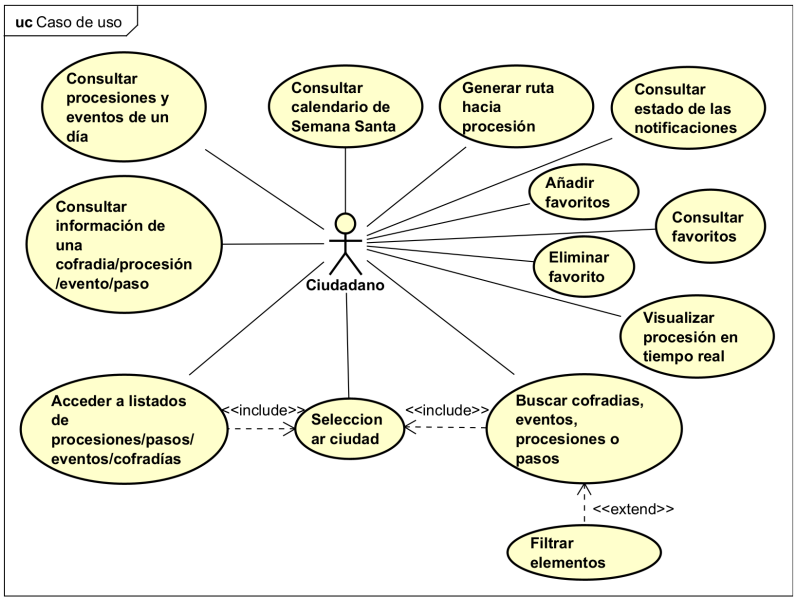


Figura 3.1: Diagrama de caso de uso Ciudadano

[3]Valor tomado como referencia habitual de usabilidad para evitar que el usuario perciba la aplicación como lenta al cargar el mapa.

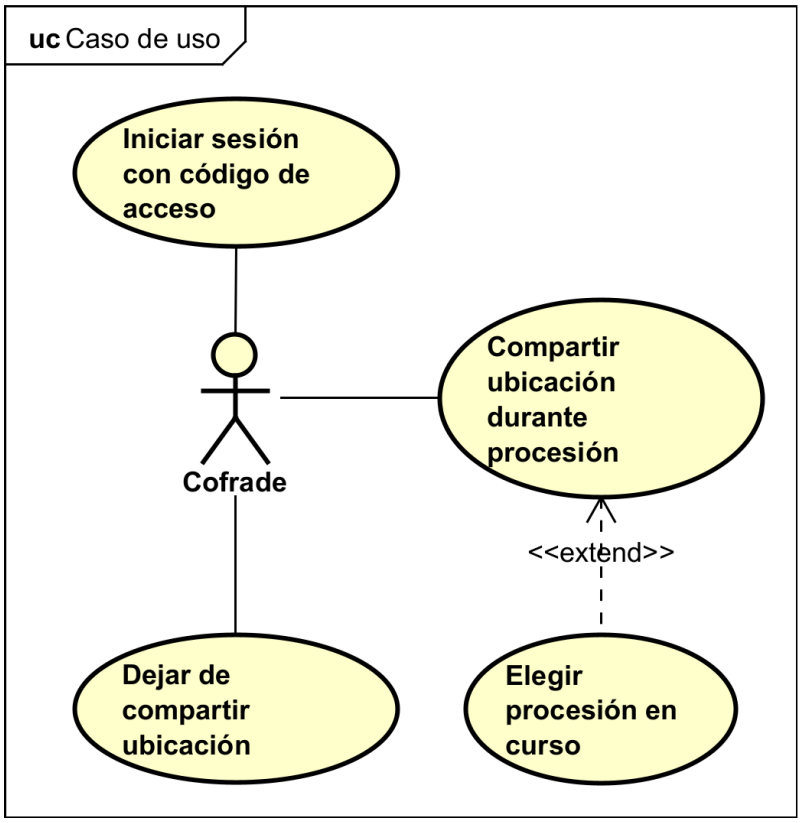


Figura 3.2: Diagrama de caso de uso Cofrade

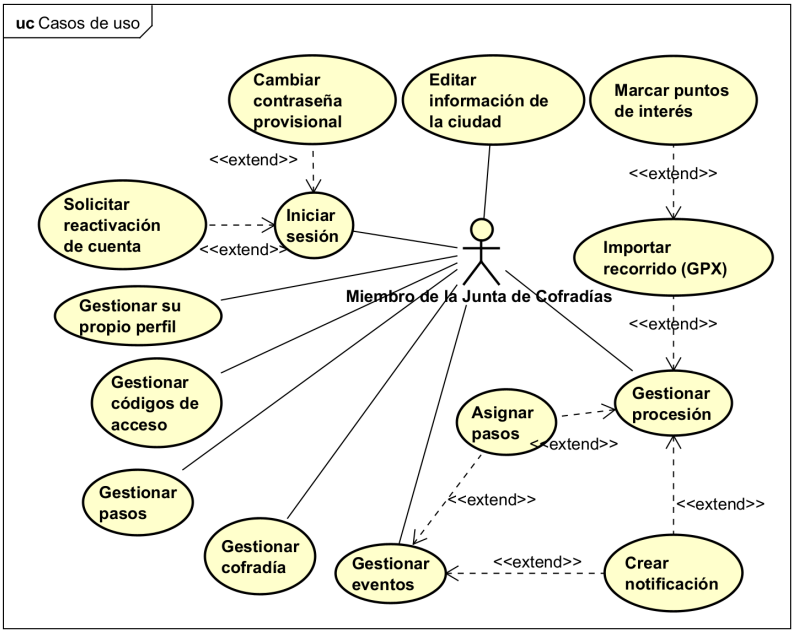


Figura 3.3: Diagrama de caso de uso Miembro de la Junta de Cofradías

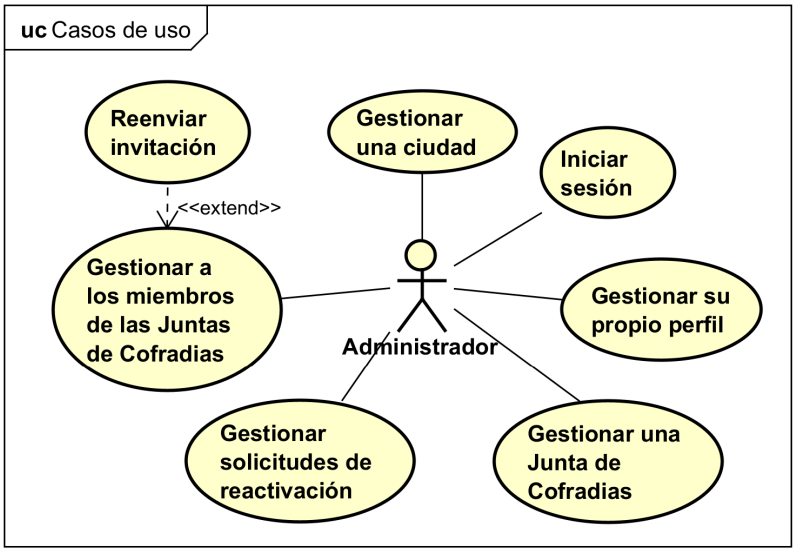


Figura 3.4: Diagrama de caso de uso Administrador

En las siguientes tablas se exponen los cinco casos de uso más representativos del sistema, que cubren a los cuatro roles implicados (ciudadano, cofrade, miembro de la Junta de Cofradías y administrador). En cada uno se detallan: el caso de uso, los actores que lo realizan, las precondiciones necesarias para iniciarlo, las postcondiciones que deben cumplirse al completarse, una secuencia base de pasos, las secuencias alternativas que pueden darse durante su ejecución y los requisitos relacionados. El resto de los casos de uso identificados, de carácter más secundario, se recogen en el Apéndice **??**.

3.4. Modelo de dominio

En esta sección se presenta el modelo de dominio del sistema, es decir, los conceptos principales de la Semana Santa que maneja la aplicación y cómo se relacionan entre sí. El modelo se ha construido a partir de los requisitos y de los casos de uso del análisis, así que no representa cómo está programada la aplicación, sino qué información existe y qué reglas tiene. A continuación se explican las decisiones más importantes que se han tomado al construir el diagrama de la Figura 3.5.

Caso de uso: Visualizar procesión en tiempo real	
Resumen	El usuario consulta en el mapa la posición en tiempo real de las procesiones que están en curso en la ciudad seleccionada.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad y dispone de conexión a internet.
Postcondición	Se muestran sobre el mapa el recorrido de las procesiones en curso y el tramo que ocupa cada una en ese momento.
Secuencia Base	
[1] El usuario solicita consultar las procesiones en tiempo real.	
[2] El sistema muestra el mapa de la ciudad seleccionada con las procesiones que están en curso en ese momento, sin que el usuario tenga que elegir ninguna.	
[3] El sistema dibuja el recorrido de cada procesión en curso y, sobre él, el tramo que ocupa en ese momento, desde el cofrade más atrasado hasta el más adelantado que comparten su ubicación.	
[4] El sistema actualiza periódicamente la posición de cada procesión, al menos cada 30 segundos.	
[5] El usuario interactúa con el mapa (zoom y desplazamiento) o elige una de las procesiones en curso para centrar el mapa en ella.	
Secuencia Alternativa	
[2.a.1] No hay ninguna procesión en curso: el sistema muestra el mapa de la ciudad e informa de que no hay procesiones en movimiento.	
[3.a.1] Ningún cofrade comparte su ubicación todavía: el sistema muestra el recorrido de la procesión sin el tramo ocupado.	
[4.a.1] Pérdida de conexión: el sistema mantiene la última posición conocida hasta que la conexión se recupera.	
Requisitos Relacionados	RF-15, RF-16 y RF-18.

Tabla 3.2: Caso de uso: Visualizar procesión en tiempo real

Caso de uso: Crear notificación	
Resumen	El miembro de la Junta de Cofradías informa a los ciudadanos de su ciudad de una incidencia o un cambio en una procesión o un evento.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión como miembro activo de la Junta de Cofradías y ya ha sustituido su contraseña provisional. Acaba de guardar cambios en una procesión o un evento de su ciudad (casos de uso «Gestionar procesión» y «Gestionar eventos»).
Postcondición	La notificación queda registrada en el sistema y se envía como notificación *push* a los dispositivos registrados en la ciudad.
Secuencia Base	
[1] El usuario indica que quiere notificar el cambio realizado.	
[2] El sistema solicita el tipo de notificación (actualización de información, cambio de horario, incidencia o cancelación), su prioridad (baja, media o alta) y, opcionalmente, un mensaje.	
[3] El usuario introduce los datos de la notificación.	
[4] El usuario confirma el envío.	
[5] El sistema registra la notificación con el nombre de la procesión o del evento como título.	
[6] El sistema envía la notificación *push* a todos los dispositivos registrados en la ciudad.	
Secuencia Alternativa	
[1.a.1] El cambio guardado ha sido una cancelación: el sistema inicia directamente este caso de uso con el tipo cancelación ya elegido, para que el usuario indique el motivo.	
[1.a.2] El usuario no quiere notificar el cambio: el caso de uso termina sin crear ninguna notificación.	
[3.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[4.a.1] Datos incompletos: falta el tipo o la prioridad; el sistema indica qué datos son obligatorios y no la envía.	
[5.a.1] Tipo cancelación: además de registrar la notificación, el sistema cambia el estado de la procesión o del evento a cancelado.	
[6.a.1] Error al enviar la notificación *push*: la notificación queda registrada y consultable en la aplicación igualmente.	
Requisitos Relacionados	RF-35 y RF-36.

Tabla 3.3: Caso de uso: Crear notificación

Caso de uso: Iniciar sesión con código de acceso	
Resumen	Un cofrade accede al modo cofrade con el código de acceso de su cofradía, escaneando su código QR o escribiéndolo, sin necesidad de registrarse.
Actor	Cofrade
Precondición	La Junta de Cofradías ha emitido un código de acceso para la cofradía del cofrade, no lo ha revocado y no han pasado 24 horas desde su emisión.
Postcondición	El cofrade queda identificado como cofrade de esa cofradía y puede compartir su ubicación durante sus procesiones en curso.
Secuencia Base	
[1] El usuario solicita acceder como cofrade.	
[2] El sistema solicita el código de acceso de la cofradía, que el usuario puede escanear como código QR o escribir.	
[3] El usuario escanea el código QR que le muestra la Junta de Cofradías.	
[4] El sistema comprueba que el código existe, que no ha sido revocado ni ha caducado, y que la cofradía está activa.	
[5] El sistema identifica al usuario como cofrade de esa cofradía, sin crear ningún usuario ni guardar ningún dato que lo identifique, y le da acceso al modo cofrade.	
Secuencia Alternativa	
[3.a.1] El usuario no puede o no quiere usar la cámara: escribe el código que aparece junto al QR.	
[3.a.2] El usuario no concede el permiso de cámara: el sistema le informa de que puede escribir el código a mano.	
[3.a.3] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[4.a.1] Código inexistente: el sistema informa de que el código no es válido y no concede el acceso.	
[4.a.2] Código revocado: el sistema informa de que el código ha sido revocado y no concede el acceso.	
[4.a.3] Código caducado: el sistema informa de que el código ha caducado y de que debe pedir uno nuevo a la Junta de Cofradías, y no concede el acceso.	
[4.a.4] Cofradía desactivada: el sistema informa de que la cofradía está desactivada y no concede el acceso.	
Requisitos Relacionados	RF-23.

Tabla 3.4: Caso de uso: Iniciar sesión con código de acceso

Caso de uso: Compartir ubicación durante procesión	
Resumen	Un cofrade comparte su ubicación en tiempo real mientras participa en una procesión en curso de su cofradía, para alimentar la posición en vivo que consultan los ciudadanos.
Actor	Cofrade
Precondición	El cofrade ha accedido al modo cofrade (caso de uso «Iniciar sesión con código de acceso»).
Postcondición	El sistema recibe periódicamente la ubicación del cofrade, de forma anónima, hasta que deja de compartirla o la procesión finaliza.
Secuencia Base	
[1] El cofrade solicita compartir su ubicación.	
[2] El sistema comprueba que la cofradía tiene una procesión en curso en ese momento.	
[3] El sistema solicita permiso para acceder a la ubicación del dispositivo, si todavía no lo tiene concedido.	
[4] El cofrade concede el permiso.	
[5] El sistema envía la ubicación del cofrade de forma periódica, al menos cada 30 segundos, y sigue haciéndolo aunque la pantalla del dispositivo esté bloqueada.	
[6] El sistema descarta las posiciones que se encuentran a más de 50 metros del recorrido de la procesión, y guarda el resto como lecturas anónimas.	
[7] El sistema informa al cofrade de que está compartiendo su ubicación y en qué procesión.	
Secuencia Alternativa	
[2.a.1] La cofradía no tiene ninguna procesión en curso: el sistema informa de ello y no comparte la ubicación.	
[2.a.2] La cofradía tiene varias procesiones en curso a la vez: se realiza el caso de uso «Elegir procesión en curso».	
[3.a.1] Permiso ya concedido anteriormente: el sistema pasa directamente al paso 5.	
[4.a.1] El cofrade deniega el permiso: el sistema informa de que no puede compartir la ubicación sin él, y el caso de uso termina.	
[5.a.1] El cofrade solo concede el permiso mientras la aplicación está abierta: el sistema le avisa de que únicamente compartirá la ubicación con la aplicación abierta.	
[5.a.2] Pérdida de conexión: el sistema vuelve a intentar el envío en el siguiente ciclo.	
[5.a.3] La procesión finaliza, la Junta revoca el código de acceso, el código caduca o caduca la sesión: el sistema deja de compartir la ubicación automáticamente e informa al cofrade del motivo.	
[7.a.1] El cofrade decide dejar de compartir su ubicación: se realiza el caso de uso «Dejar de compartir ubicación».	
Requisitos Relacionados	RF-24 y RF-25.

Tabla 3.5: Caso de uso: Compartir ubicación durante procesión

Caso de uso: Gestionar una ciudad	
Resumen	El administrador da de alta, edita o da de baja las ciudades disponibles en la aplicación.
Actor	Administrador
Precondición	El usuario ha iniciado sesión como administrador.
Postcondición	Las ciudades disponibles quedan actualizadas en el sistema.
Secuencia Base	
[1] El usuario solicita gestionar las ciudades.	
[2] El sistema muestra las ciudades registradas, activas o no.	
[3] El usuario indica si quiere dar de alta una ciudad nueva, editar una existente o darla de baja.	
[4] El sistema solicita los datos de la ciudad: nombre, comunidad autónoma, provincia, historia, patrimonio, si está activa (visible para los ciudadanos) y, opcionalmente, su latitud y longitud.	
[5] El usuario introduce o modifica los datos y confirma la operación.	
[6] El sistema valida los datos y guarda la ciudad.	
Secuencia Alternativa	
[3.a.1] Dar de baja: el sistema pide confirmación y, si el usuario confirma, elimina la ciudad junto con sus notificaciones y sus dispositivos registrados.	
[3.a.2] La ciudad tiene una Junta de Cofradías o cofradías asociadas: el sistema no permite darla de baja e informa de que antes hay que eliminarlas, o bien desactivar la ciudad.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[6.a.1] Datos incompletos o no válidos: el sistema indica qué datos debe corregir el usuario y no guarda la ciudad.	
Requisitos Relacionados	RF-37.

Tabla 3.6: Caso de uso: Gestionar una ciudad

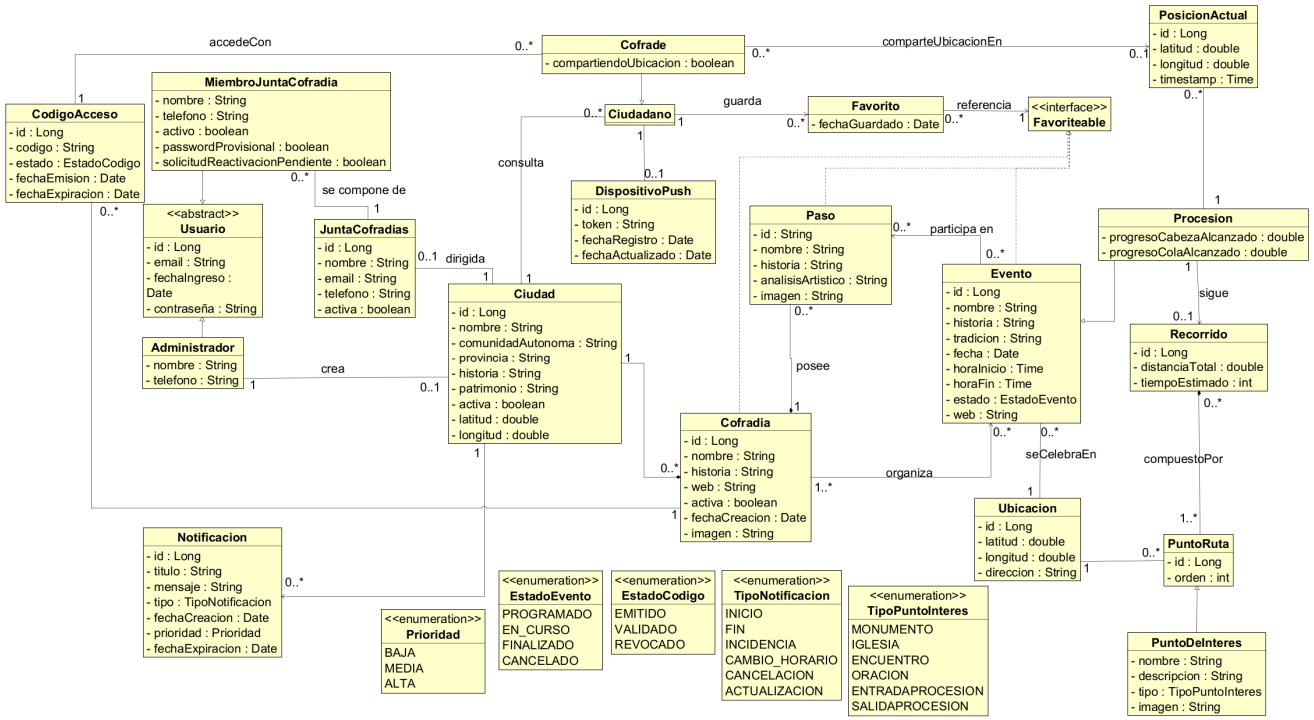


Figura 3.5: Diagrama de dominio del proyecto

Usuarios. Lo primero que se tuvo que decidir fue cómo representar los cuatro perfiles de la aplicación, ya que no todos funcionan igual. El administrador y el miembro de la Junta de Cofradías necesitan entrar con su correo y su contraseña, así que se ha creado una clase abstracta `Usuario` con los datos comunes a los dos (correo, contraseña y fecha de ingreso), de la que heredan `Administrador` y `MiembroJuntaCofradia`. El miembro de la Junta tiene además algunos datos propios, como si está activo, si todavía tiene la contraseña provisional que se le envió al darle de alta o si ha pedido que se le reactive la cuenta. En cambio, el `Ciudadano` no se registra nunca (RF-01), por eso no hereda de `Usuario` y no tiene ningún dato personal. El `Cofrade` se ha modelado como un tipo de `Ciudadano`, porque puede hacer todo lo que hace un ciudadano y además compartir su ubicación, pero tampoco se registra: accede con el `CodigoAcceso` de su cofradía.

Ciudad, Junta y cofradías. La clase central del modelo es `Ciudad`, ya que toda la información de la aplicación pertenece a una ciudad concreta. Cada ciudad está dirigida como mucho por una `JuntaCofradias` (puede que todavía no tenga) y cada junta dirige una sola ciudad, y la junta está compuesta por sus miembros. Una decisión importante fue que las `Cofradia` pertenecen directamente a la ciudad y no a la junta. Así, si la junta se renueva o se da de baja, las cofradías y toda su información siguen existiendo. Por otro lado, el administrador es quien da de alta las ciudades, pero no está atado a ninguna: puede gestionar cualquiera de ellas.

Eventos, procesiones y pasos. Un `Evento` es cualquier acto de la Semana Santa (un pregón, un sermón, un vía crucis...) y puede estar organizado por varias cofradías a la vez, por eso la relación entre `Evento` y `Cofradia` es de muchos a muchos. Todo evento tiene una fecha y una hora de inicio y de fin, que es lo que permite saber cuándo pasa a estar en curso y cuándo termina (RF-34). Como un evento se celebra en un solo día, se ha preferido guardar una única fecha con sus dos horas en lugar de repetir

la fecha. Si alguna procesión termina pasada la medianoche, su hora de fin será menor que la de inicio, lo que indica que acaba al día siguiente. La duración no se guarda, ya que se puede calcular a partir de las dos horas. Se decidió que `Procesion` **herede** de `Evento` en lugar de ser una clase aparte, porque una procesión es un evento con algo más: tiene sus mismos datos, fechas y cofradías, y además el `Recorrido` que sigue y lo que ha avanzado por él la cabeza y la cola de la procesión. Cada `Paso` pertenece a una única cofradía, pero puede salir en varios eventos y procesiones, así que la relación entre `Paso` y `Evento` también es de muchos a muchos. Al principio solo se relacionaban los pasos con las procesiones, pero se vio que en otros actos también aparecen pasos, así que se subió la relación a `Evento` y la procesión la hereda. Además, hay una regla que no se ve en las multiplicidades: un paso solo puede participar en un evento si su cofradía participa en él (RF-30).

Recorridos y ubicaciones. Un `Recorrido` está formado por una serie de `PuntoRuta` ordenados, que son los puntos del trazado de la procesión. Algunos de esos puntos tienen algo especial (una iglesia, un encuentro, una parada para rezar...), y para ellos se ha creado `PuntoDeInteres`, que hereda de `PuntoRuta` y le añade nombre, descripción, imagen y tipo. Las coordenadas se han separado en una clase `Ubicacion` independiente (latitud, longitud y dirección), porque las necesitan tanto los puntos del recorrido como los eventos, que se celebran en un lugar concreto. De esta forma se reutiliza el mismo concepto en vez de repetir los mismos atributos en varias clases.

Seguimiento en tiempo real. Mientras una procesión está en curso, los cofrades pueden compartir su ubicación, y cada lectura que envían se guarda como una `PosicionActual` de esa procesión. Aquí la privacidad era muy importante (RNF-01), así que aunque la posición la genera un cofrade, no se guarda ningún dato que diga quién la ha enviado, solo la procesión a la que pertenece. Además, la posición que ven los demás usuarios no es la de un cofrade concreto, sino una calculada a partir de las lecturas más recientes de la procesión.

Notificaciones. Al principio se pensó en tener dos tipos de notificaciones (avisos y alertas), pero al analizarlo se vio que lo único que las diferenciaba era su importancia, así que se juntaron en una sola clase `Notificacion` con un tipo y una prioridad. Cada notificación pertenece a una ciudad. Las de tipo `INICIO` y `FIN` las crea el sistema solo cuando un evento empieza o termina, y el resto (`INCIDENCIA`, `CAMBIO_HORARIO`, `CANCELACION` y `ACTUALIZACION`) las escribe la Junta de Cofradías. No se guarda qué ciudadano ha leído cada notificación, porque el ciudadano no tiene cuenta. Para que le lleguen al móvil, el ciudadano puede tener un `DispositivoPush`, y recibe las notificaciones de la ciudad que está consultando. No se ha relacionado directamente `DispositivoPush` con `Ciudad` porque esa ciudad siempre es la misma que consulta su ciudadano, y añadir esa relación habría creado un ciclo con información repetida.

Código de acceso y favoritos. El `CodigoAcceso` da acceso a una cofradía concreta, tiene un estado (emitido, validado o revocado) y caduca 24 horas después de emitirse. Lo emite la Junta de Cofradías en nombre de la cofradía, ya que las cofradías no tienen una cuenta propia, y una misma cofradía puede tener varios códigos a lo largo del tiempo, por ejemplo si se revoca uno y se emite otro nuevo. Por último, para los favoritos (RF-14) se ha usado la interfaz `Favoriteable`, que la cumplen `Cofradia`, `Evento` y `Paso` (y `Procesion` a través de `Evento`). Así, la clase `Favorito` puede guardar cualquiera de ellos sin tener que crear una relación distinta para cada tipo.

3.4.1. Modelo de interacción

En esta sección se describen, mediante diagramas de secuencia, los cinco casos de uso más representativos de la aplicación (Sección 3.3), de forma que aparecen los cuatro roles. Los diagramas se han hecho a nivel de análisis: el sistema aparece como una caja negra que se comunica con los objetos del modelo de dominio, sin entrar en pantallas ni en cómo está programado. Para que se lean mejor, no se han incluido las secuencias en las que el usuario cancela la operación. Además, solo se dibuja el estereotipo `«create»` cuando el caso de uso crea un objeto nuevo; cuando se trabaja con uno que ya existe (una procesión, un código de acceso...), el sistema simplemente lo obtiene.

En el caso de uso *Visualizar procesión en tiempo real* (Figura 3.6), el ciudadano pide ver las procesiones en tiempo real y, sin tener que elegir ninguna (RF-16), el sistema busca las `Procesion` de su ciudad que están en curso. Si no hay ninguna, se le muestra el mapa y se le avisa de que no hay procesiones en movimiento. Si las hay, para cada una se obtiene su `Recorrido` y, a partir de las `PosicionActual` más recientes, el tramo que ocupa en ese momento, que va desde lo que ha avanzado la cola hasta lo que ha avanzado la cabeza. Todo esto se repite al menos cada 30 segundos (RNF-05) mientras el ciudadano tiene el mapa abierto, para que vea cómo avanza cada procesión.

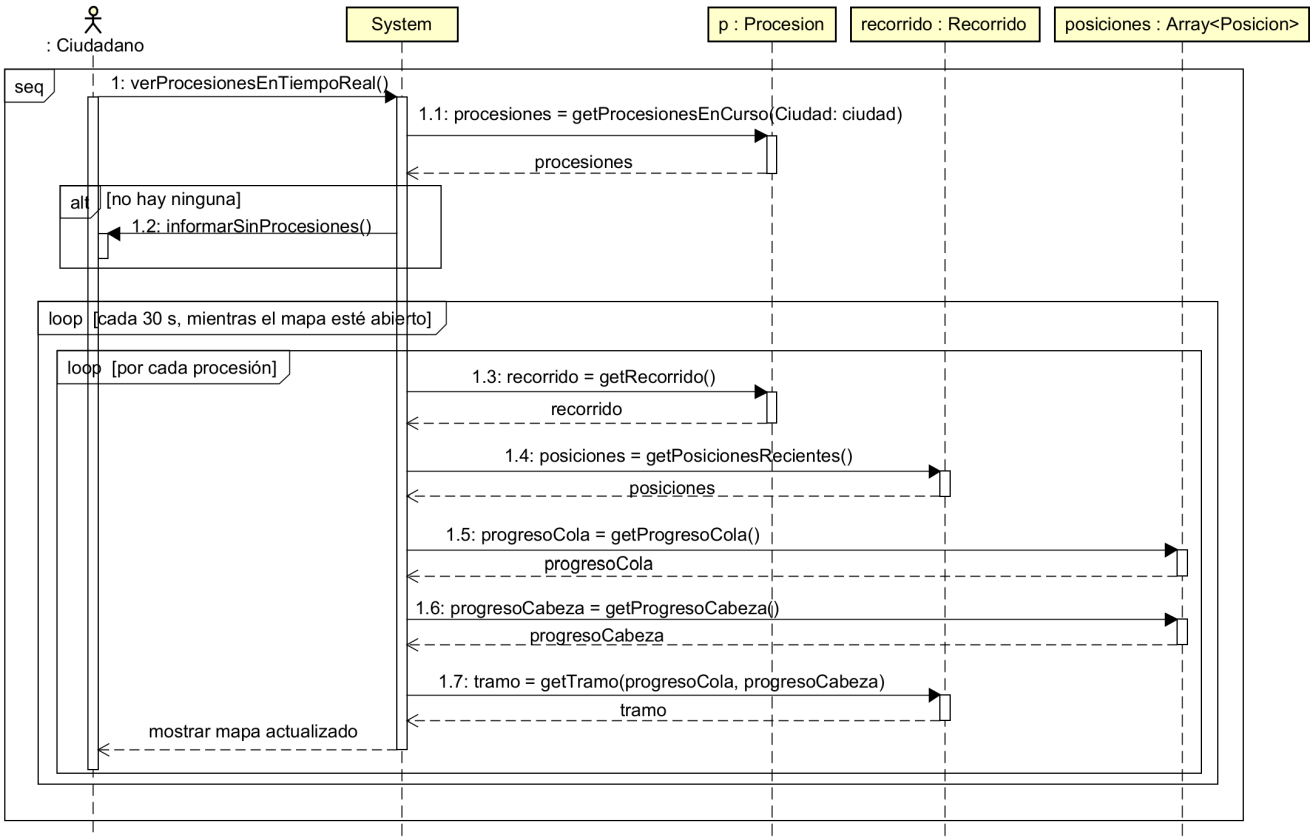


Figura 3.6: Diagrama de secuencia del CU “Visualizar procesión en tiempo real”

En el caso de uso *Iniciar sesión con código de acceso* (Figura 3.7), el usuario pide entrar como cofrade y el sistema le pide el código de su cofradía, que el usuario escanea como código QR o escribe a mano. Con el código obtenido, el sistema busca el `CodigoAcceso` correspondiente y comprueba cuatro cosas: que

existe, que no ha sido revocado, que no ha caducado y que su `Cofradia` está activa. Si falla alguna, se le informa del motivo y no entra. Si todo es correcto, el código pasa a estado validado y el usuario queda identificado como cofrade de esa cofradía. En este caso de uso no se crea ningún `Usuario`: el cofrade sigue siendo anónimo y lo único que se sabe de él es a qué cofradía pertenece.

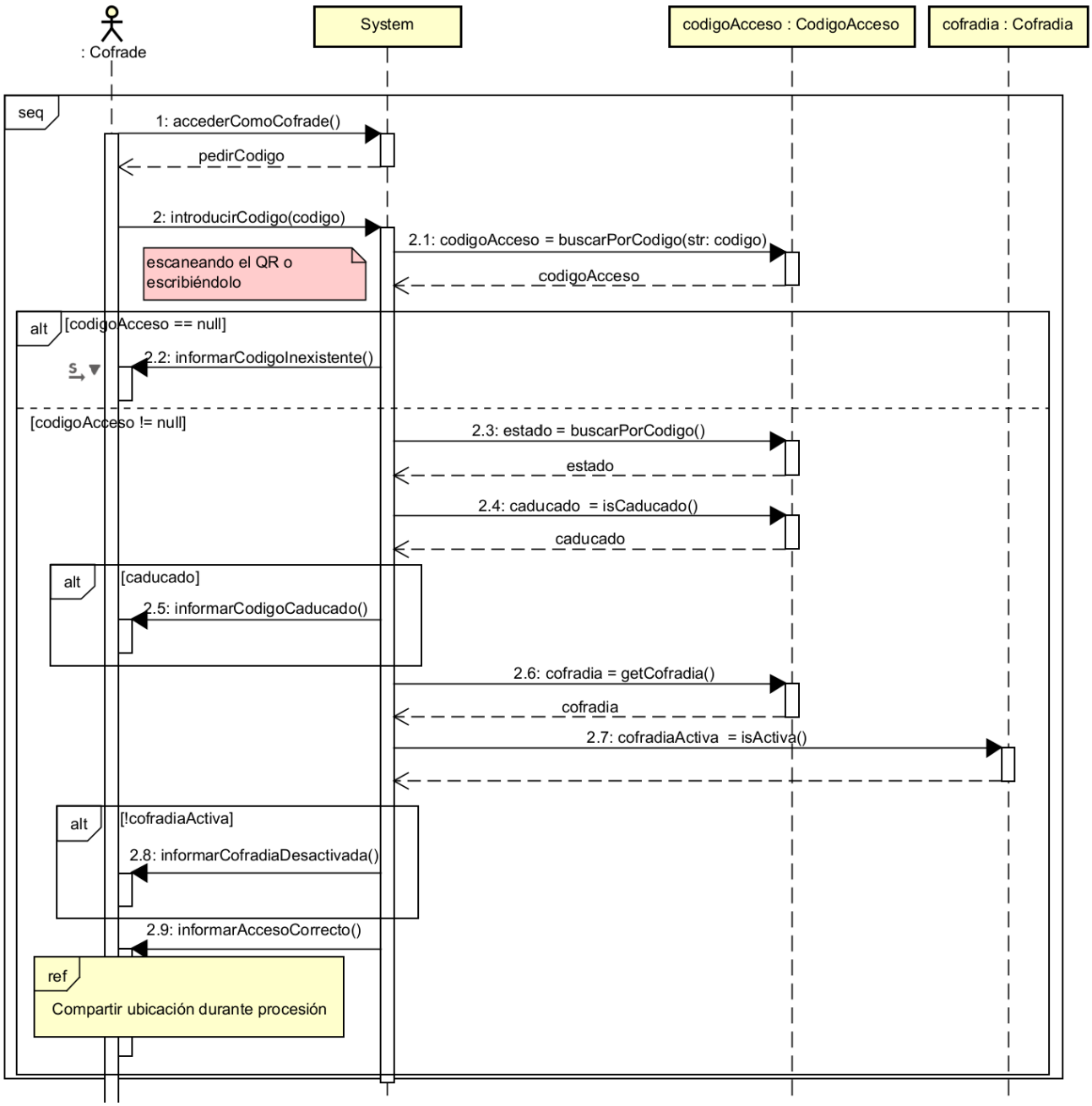


Figura 3.7: Diagrama de secuencia del CU “Iniciar sesión con código de acceso”

En el caso de uso *Compartir ubicación durante procesión* (Figura 3.8), el cofrade pide compartir su ubicación y el sistema busca la procesión en curso de su cofradía. Si no hay ninguna, se le informa y el caso de uso termina, y si hay varias a la vez, se le pide que elija en cuál está. Después, el sistema pide permiso para usar la ubicación del dispositivo; si el cofrade lo deniega, se le avisa de que sin él no se puede compartir. Si lo concede, mientras la procesión siga en curso se envía su posición al menos cada 30

segundos. Con cada posición recibida, el sistema comprueba si está a menos de 50 metros del `Recorrido`: si lo está, crea una nueva `PosicionActual` de la procesión, sin guardar quién la ha enviado, y actualiza lo que han avanzado la cabeza y la cola; si no, la descarta (RF-25). Se ha decidido crear una posición nueva con cada envío, en lugar de ir actualizando siempre la misma, porque cada lectura es un dato distinto y la posición de la procesión se calcula a partir de todas las más recientes.

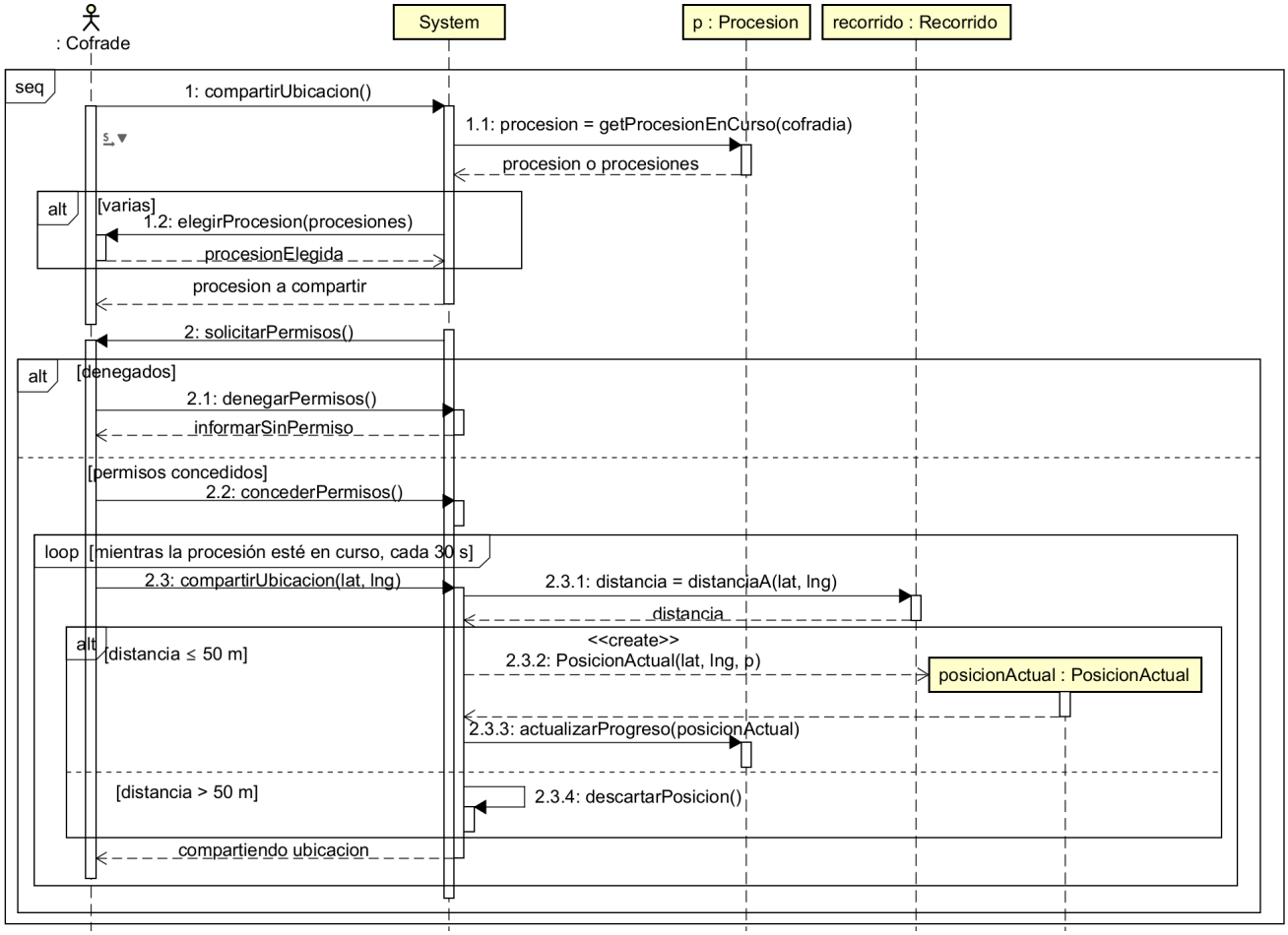


Figura 3.8: Diagrama de secuencia del CU “Compartir ubicación durante procesión”

En el caso de uso *Crear notificación* (Figura 3.9), el miembro de la Junta de Cofradías acaba de guardar un cambio en una procesión o un evento e indica que quiere avisar de él a los ciudadanos. El sistema le pide el tipo de notificación, la prioridad y, si quiere, un mensaje. Si falta el tipo o la prioridad, se le indica qué datos son obligatorios. Cuando están completos, el sistema crea la `Notificacion` en la `Ciudad` de la Junta, usando como título el nombre de la procesión o del evento, y si el tipo es cancelación, cambia además el estado del evento a cancelado. Por último, la notificación se envía a los `DispositivoPush` de los ciudadanos que consultan esa ciudad. Aunque falle este envío, la notificación queda guardada y se puede consultar dentro de la aplicación.

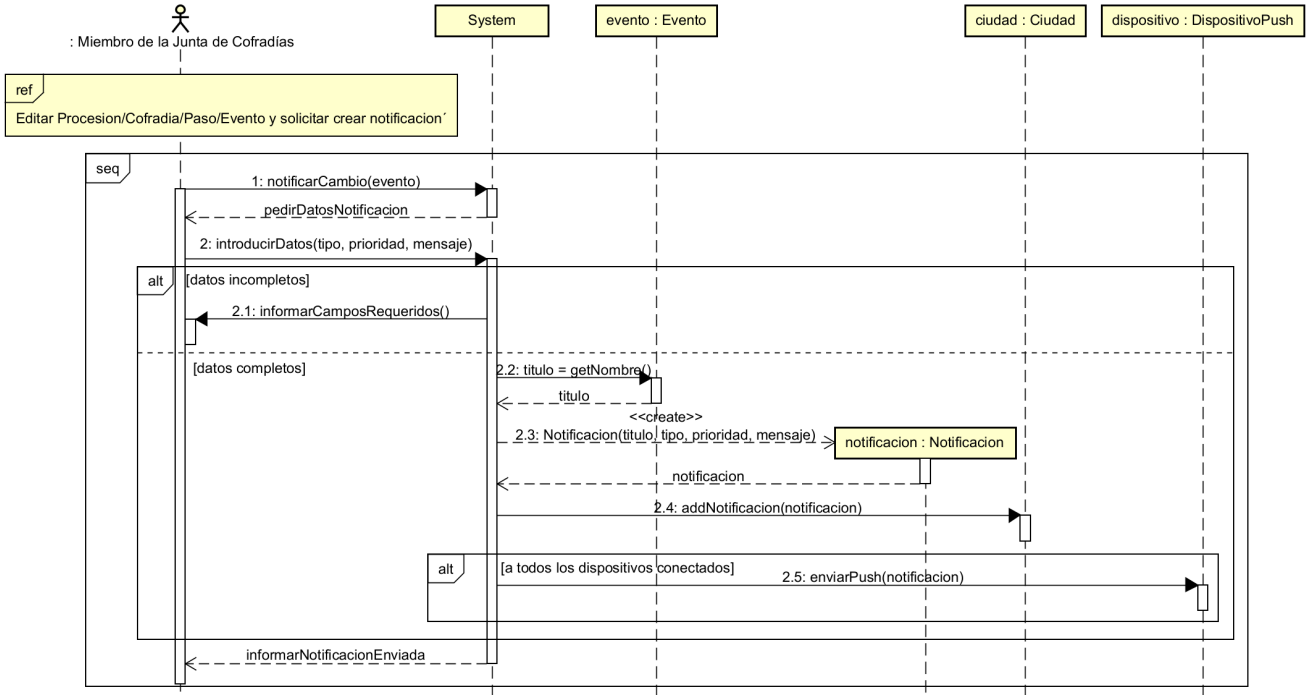


Figura 3.9: Diagrama de secuencia del CU “Crear notificación”

Por último, en el caso de uso *Gestionar una ciudad*, en su variante de alta (Figura 3.10), el administrador pide gestionar las ciudades y el sistema le muestra las que ya existen. El administrador indica que quiere dar de alta una nueva e introduce sus datos, y el sistema crea la `Ciudad` y comprueba que los datos son válidos. Si falta algún dato obligatorio, se le indica cuál y se le vuelven a pedir, hasta que son correctos y la ciudad queda dada de alta. Solo se ha dibujado el alta porque la edición y la baja siguen el mismo esquema.

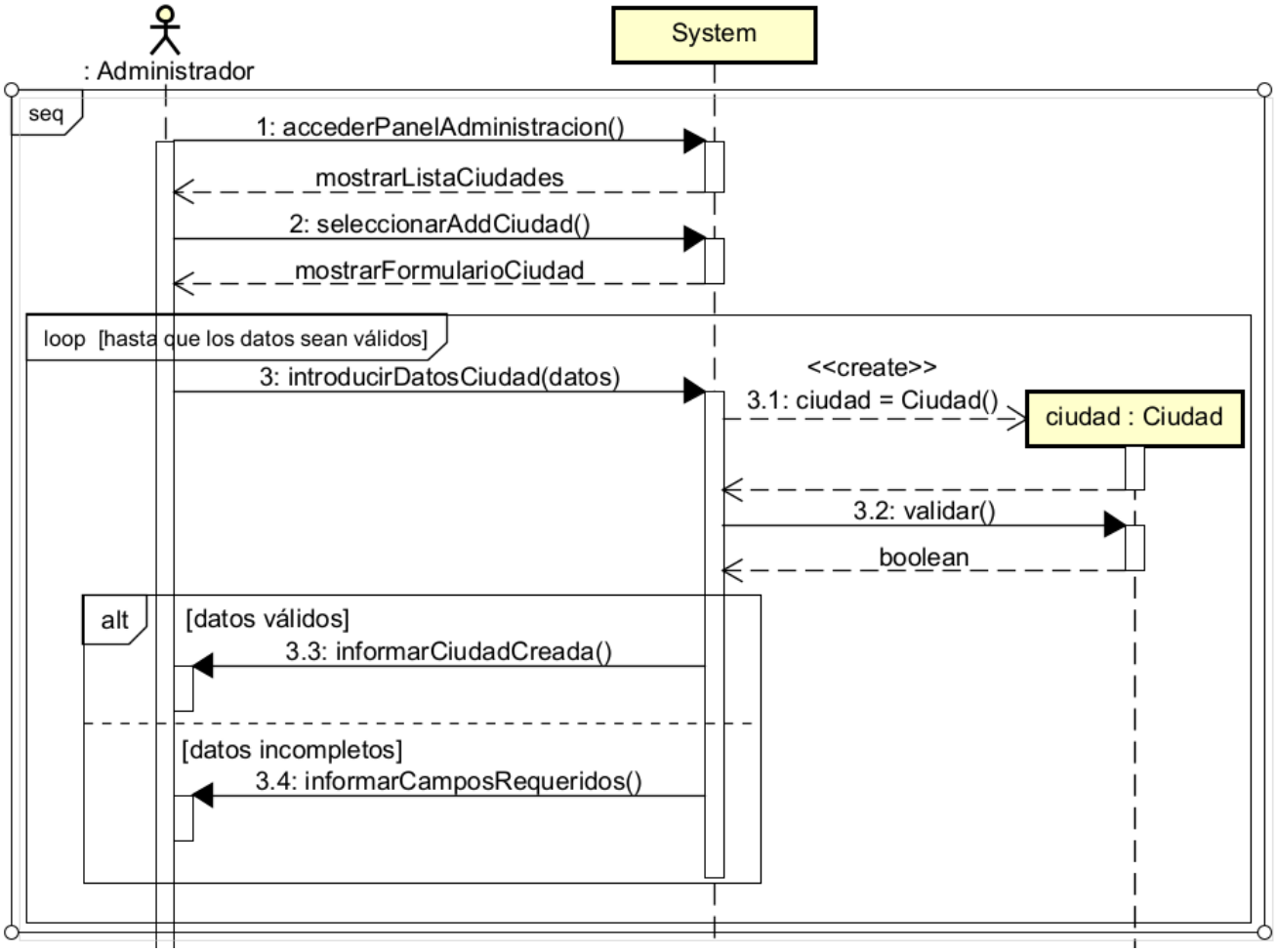


Figura 3.10: Diagrama de secuencia del CU “Gestionar una ciudad” (alta)

Capítulo 4

Descripción de las iteraciones

4.1. 1ª Iteración

4.1.1. Análisis

Según la planificación (Sección 2.2), esta primera iteración se centra en el módulo básico del ciudadano, es decir, en la selección de la ciudad (RF-01, RF-02 y RF-04), la consulta de la información general de la Semana Santa y los listados y fichas de cofradías, procesiones, pasos y eventos (RF-05 a RF-08 y RF-11). Sin embargo, al tratarse de la primera iteración, también es en la que se define la base sobre la que se construye el resto del proyecto.

Debido a esto, antes de empezar con el diseño se analizaron los principales problemas que había que resolver, valorando para cada uno de ellos las distintas alternativas posibles y eligiendo la que mejor se adaptaba al proyecto. En este apartado se recogen las decisiones necesarias para construir esta primera base del sistema.

Problemas planteados y alternativas valoradas

Aplicación para Android e iOS El requisito RNF-04 exige que la aplicación funcione tanto en Android como en iOS. Para ello se valoraron tres alternativas: desarrollar dos aplicaciones nativas (una en Kotlin para Android y otra en Swift para iOS), utilizar Flutter o utilizar React Native [**?**]. Las aplicaciones nativas se descartaron desde el principio, ya que al ser un proyecto desarrollado por una única persona supondría programar y probar todo dos veces sin obtener ninguna ventaja funcional. Entre Flutter y React Native, ambas permiten mantener un único código para las dos plataformas, pero finalmente se optó por React Native, ya que utiliza JavaScript, un lenguaje muy extendido y con una gran cantidad de librerías para las funcionalidades que necesita la aplicación, como los mapas, la geolocalización o las notificaciones. Además, se decidió utilizar Expo [**?**] en lugar de configurar React Native de forma manual, siguiendo la recomendación de la propia documentación oficial para quienes empiezan en el desarrollo móvil. Expo ofrece ya integrados los módulos nativos que necesita la aplicación, como la ubicación, las notificaciones

o los mapas, permite probar la aplicación directamente en el móvil durante el desarrollo y genera los instaladores en la nube (EAS Build), sin necesidad de instalar y configurar Android Studio y Xcode.

Servidor propio o servicio externo En un primer momento se planteó utilizar una plataforma *Backend-as-a-Service* (BaaS) como Firebase, que ofrece ya resueltos la base de datos, la autenticación y el control de acceso, lo que agiliza mucho el desarrollo. La alternativa era desarrollar una API REST propia. Finalmente se optó por la API propia, desarrollada con Spring Boot [**?**], ya que con Firebase gran parte de estas decisiones las tomaba la propia plataforma, y se perdía precisamente la parte de realizar el diseño de la arquitectura en capas, modelar la base de datos, implementar la autenticación y diseñar la API. Además, de esta forma el proyecto no depende de las condiciones de un servicio externo.

Tipo de base de datos Se compararon las bases de datos NoSQL, como Firestore o MongoDB, con una base de datos relacional como PostgreSQL [**?**]. Se eligió la base de datos relacional porque el dominio del proyecto (Sección 3.4) está formado por muchas entidades relacionadas entre sí, como una ciudad con sus cofradías, una cofradía con sus pasos o un evento en el que participan varias cofradías. En una base de datos relacional es la propia base de datos la que garantiza que estas relaciones sean correctas mediante las claves foráneas, mientras que en una NoSQL habría que controlarlo desde el código.

Consulta de información sin conexión Para cumplir el requisito RNF-06 se podía desarrollar una caché propia que guardara las respuestas en el móvil, o utilizar una librería que ya lo hiciera. Se eligió TanStack Query [**?**], que guarda en el dispositivo la última respuesta de cada consulta y la vuelve a pedir automáticamente cuando hay conexión, evitando tener que programarlo desde cero.

Imágenes y favoritos Para las imágenes de las cofradías y de los pasos se valoró subirlas a un servicio de almacenamiento como Firebase Storage [**?**] o guardar únicamente su dirección (URL). Se eligió guardar la URL por ser la opción más sencilla para esta versión, dejando la subida de imágenes como trabajo futuro (Sección **??**). En cuanto a los favoritos (RF-14), se valoró guardarlos en el servidor o en el propio móvil, y se optó por guardarlos en el móvil, ya que el ciudadano no tiene ninguna cuenta a la que asociarlos.

En la Tabla 4.1 se resumen todas estas decisiones.

Problema	Alternativas valoradas	Opción elegida
Aplicación Android e iOS	Dos aplicaciones nativas, Flutter, React Native	React Native con Expo
Servidor	Firebase (BaaS), API propia	API REST propia con Spring Boot
Base de datos	NoSQL (Firestore, MongoDB), relacional	PostgreSQL
Información sin conexión	Caché propia, TanStack Query	TanStack Query
Imágenes	Servicio de almacenamiento, URL	URL
Favoritos	En el servidor, en el móvil	En el móvil

Tabla 4.1: Problemas analizados en la primera iteración y opción elegida

4.1.2. Diseño

Una vez elegidas las alternativas en el análisis, en este apartado se explica cómo se ha diseñado cada una de ellas: cómo se organiza el sistema, cómo se guardan los datos, cómo son las pantallas y qué patrones de diseño se han aplicado.

Arquitectura del sistema

La arquitectura de un sistema describe, de forma general, en qué partes se divide y cómo se comunican entre sí. En este proyecto se ha definido una arquitectura **cliente-servidor con servidor propio**, en la que la aplicación React Native (el cliente[1]) se comunica mediante peticiones HTTPS[2] con la API REST propia (el *backend*[3]), desarrollada con Spring Boot. Esta API ofrece un conjunto de *endpoints*[4] organizados por recurso, como `/ciudades`, `/cofradias`, `/pasos`, `/eventos`, `/procesiones`, `/recorridos`, `/notificaciones` o `/auth`.

La API es la que contiene toda la lógica de negocio y la única que accede a la base de datos PostgreSQL, de forma que el cliente nunca se comunica directamente con ella.

Estructura del front-end El código del cliente se organiza siguiendo una **arquitectura en capas** [**?**], en la que el proyecto se divide en varios grupos de carpetas (o *paquetes*), cada uno con una responsabilidad concreta. Cada capa solo puede depender de la capa inmediatamente inferior y nunca al revés, de forma que un cambio en una capa afecta al menor número posible de partes del código y cada capa se puede probar por separado. Las cuatro capas definidas son:

[1]Cliente: en una arquitectura cliente-servidor, es la parte del sistema con la que interactúa directamente el usuario; en este caso, la propia aplicación instalada en el teléfono móvil.

[2]HTTPS: versión segura (cifrada) del protocolo HTTP, usado habitualmente para la comunicación entre aplicaciones y servicios en internet.

[3]Backend: la parte del sistema que no ve el usuario directamente, encargada de almacenar los datos, gestionar la lógica de negocio y responder a las peticiones que le hace el cliente.

[4]Endpoint: dirección concreta de una API a la que el cliente envía una petición para realizar una operación determinada (por ejemplo, obtener el listado de procesiones de una ciudad).

- **UI** (`screens`, `components`, `navigation`): contiene las pantallas que ve el usuario, agrupadas por rol (ciudadano, cofrade, Junta de Cofradías y administrador) siguiendo los diagramas de casos de uso del Capítulo 3 y del Apendice **??**, los componentes visuales que se reutilizan entre pantallas, como los mapas, y la navegación entre pantallas.

- **Application** (`context`): contiene el estado que comparten varias pantallas a la vez, como la sesión del usuario (`AuthContext`), la ciudad seleccionada (`CiudadContext`), el día de la Semana Santa que se está consultando (`DiaContext`) o los favoritos (`FavoritosContext`).

- **Data** (`services`, `models`, `utils`): contiene un servicio por cada tipo de información del dominio (`ciudadService`, `cofradiaService`, `procesionService`, `seguimientoService`, `pushService`, etc.), que son los únicos que se comunican con la API, los modelos que describen cada objeto del dominio dentro de la aplicación y algunas utilidades, como el cálculo de las fechas de la Semana Santa de cada año.

- **Infrastructure** (`api`): contiene la configuración del cliente HTTP (`apiClient`), que añade automáticamente el token JWT a cada petición y avisa a la aplicación cuando la sesión ha caducado, y la caché de TanStack Query (`queryClient`), que guarda las respuestas en el dispositivo para poder consultarlas sin conexión.

En la Figura 4.1 se muestra esta organización mediante un diagrama de paquetes, donde las flechas discontinuas indican de qué paquete depende cada uno.

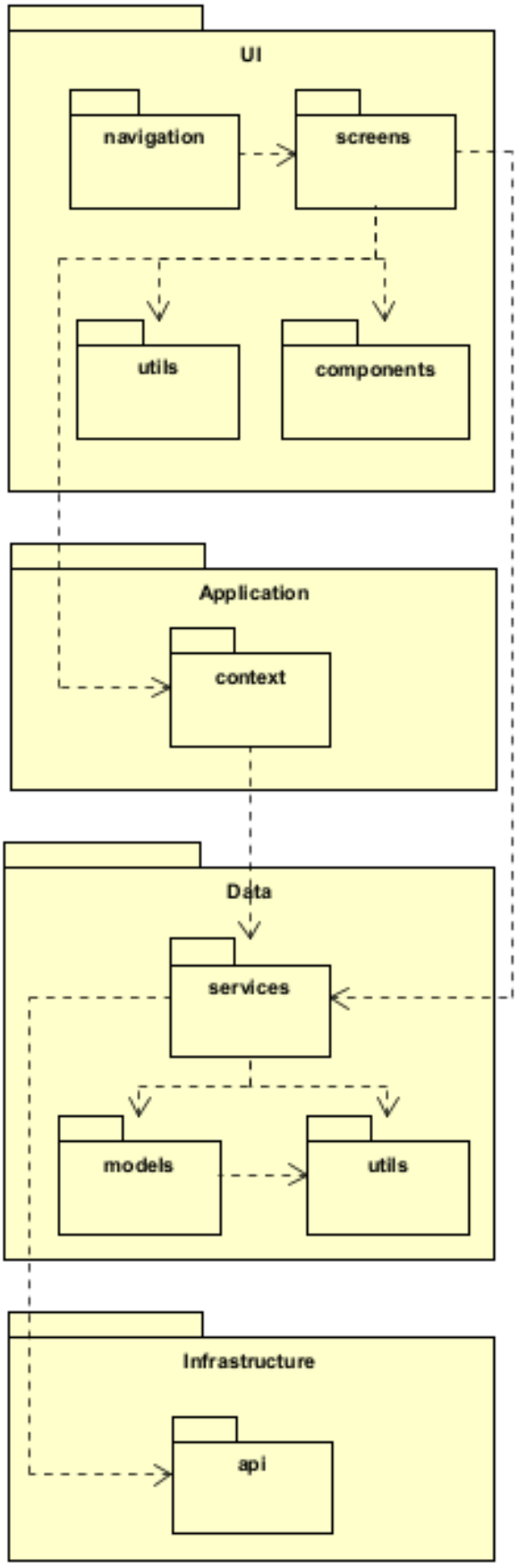


Figura 4.1: Diagrama de paquetes de la aplicación cliente

Estructura del back-end El back-end también se organiza en capas, siguiendo la estructura habitual de una API desarrollada con Spring Boot [**?**]. Esta estructura se define en esta primera iteración y es la que se sigue después con cada nueva entidad del sistema:

- **Controller**: ofrece los *endpoints* REST de cada recurso, recibe las peticiones HTTP, delega el trabajo en la capa de servicio y devuelve la respuesta HTTP correspondiente (código de estado y cuerpo en JSON).

- **Service**: contiene la lógica de negocio, como comprobar que los datos obligatorios de una cofradía están completos o que no se elimina una ciudad que todavía tiene cofradías.

- **Repository**: contiene las interfaces de Spring Data JPA [**?**], una por cada entidad, que se encargan de acceder a PostgreSQL y cuya implementación genera Spring automáticamente.

- **Security**: contiene la configuración de Spring Security [**?**], que decide qué peticiones necesitan que el usuario se identifique. En esta iteración todas las consultas son públicas, y esta capa se completa a partir de la Iteración 3, cuando aparece el acceso de los cofrades.

- **DTO** (*Data Transfer Object*): contiene las clases que definen exactamente qué datos entran y salen de cada *endpoint*, de forma que no se exponen directamente las entidades de la base de datos y un cambio interno no afecta a lo que recibe el cliente.

En esta iteración se aplica a las entidades del módulo de ciudadano (`Ciudad`, `JuntaCofradias`, `Cofradia`, `Paso` y `Evento`), y en las siguientes iteraciones el resto de entidades se van añadiendo siguiendo la misma estructura. Por ello, en la Figura 4.2 se muestra el diagrama de paquetes con todas las entidades de la versión final de la aplicación. Además de estas capas, existe un paquete `exception`, que transforma los errores en respuestas HTTP comprensibles para el cliente, y un paquete `util` con las clases que no pertenecen a ninguna entidad concreta, como la lectura de los archivos GPX. El esquema completo de la base de datos se detalla en el Apéndice **??**, y su diseño se explica en el apartado Diseño de la base de datos.

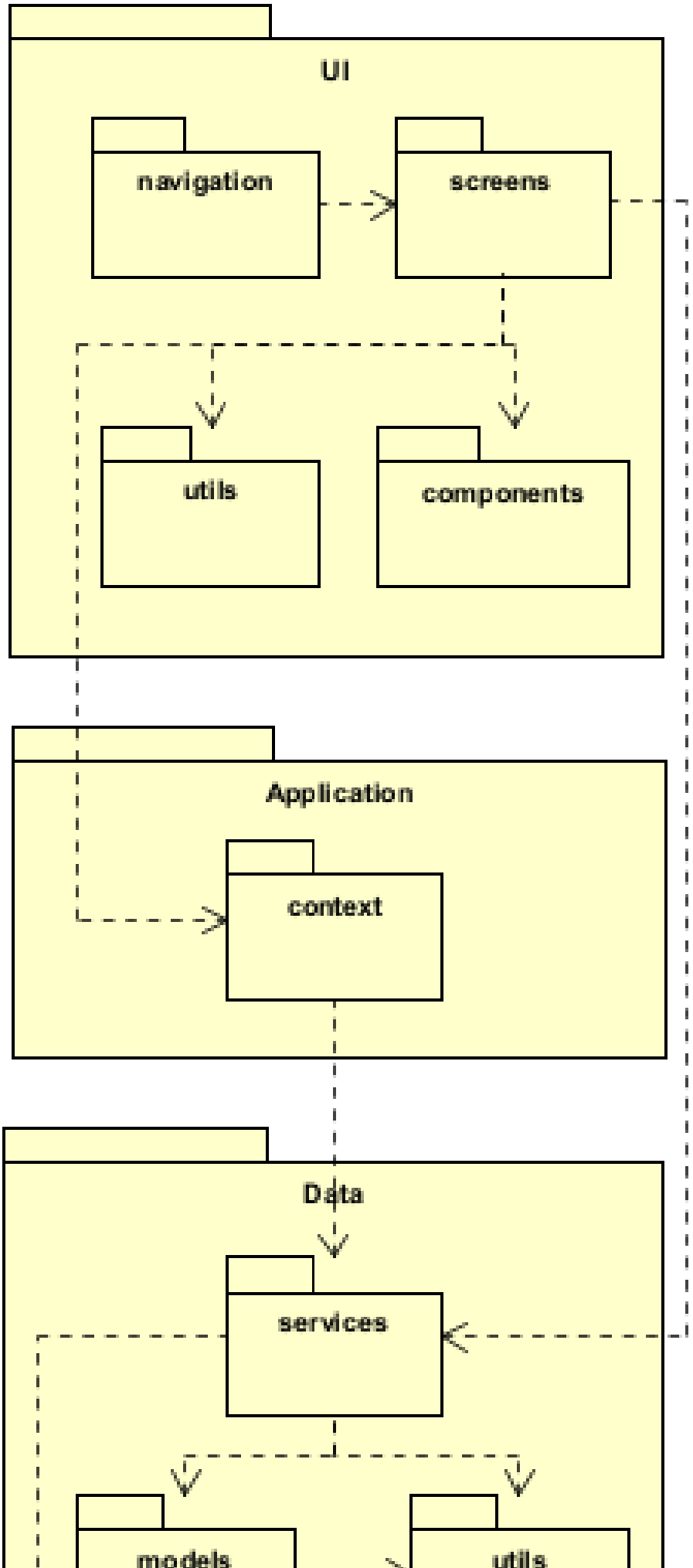

UI
navigation
screens
utils
components
Application
context
Data
services
models
utils

Diagrama de despliegue Un diagrama de despliegue representa dónde se ejecuta físicamente cada parte del sistema y cómo se comunican entre sí. En la Figura 4.3 se recoge el despliegue de este proyecto, en el que el móvil ejecuta la aplicación React Native, que se comunica únicamente con la API propia desplegada en el servidor virtual que proporciona la Escuela de Ingeniería Informática de la Universidad de Valladolid. La API es el único componente que accede a la base de datos PostgreSQL, alojada en ese mismo servidor, y es también la que envía las notificaciones *push* mediante una petición HTTP al servicio de Expo Push (`exp.host`). Por último, los mapas se muestran directamente en el móvil mediante react-native-maps, utilizando Google Maps en Android y el mapa de Apple en iOS.

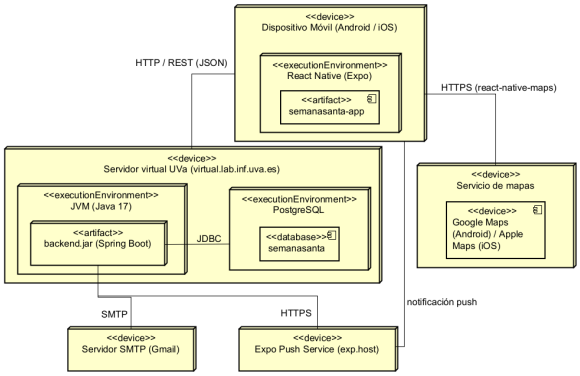


Figura 4.3: Diagrama de despliegue de la aplicación

Diseño de la base de datos

PostgreSQL [**?**] es una base de datos **relacional**, en la que la información se organiza en tablas con filas y columnas, y las relaciones entre entidades se representan mediante **claves foráneas**[5], que la propia base de datos se encarga de validar. Para acceder a ella desde Java se utiliza Spring Data JPA [**?**] junto con Hibernate [**?**], un *ORM*[6] que evita tener que escribir a mano la mayoría de las consultas SQL.

En esta iteración se definen los criterios con los que se pasa el modelo de dominio de la Figura 3.5 a tablas, y que se siguen durante todo el proyecto. Aunque en esta iteración solo se crean las tablas del módulo de ciudadano, algunos ejemplos corresponden a tablas que se añaden más tarde, ya que sirven para explicar mejor cada criterio. Las decisiones propias de cada funcionalidad posterior se explican en el diseño de la iteración correspondiente. Los criterios generales son los siguientes:

- Las relaciones de uno a muchos se representan con una clave foránea en la tabla del lado "muchos". Por ejemplo, la tabla `pasos` tiene la columna `cofradia_id` y la tabla `cofradias` tiene la columna `ciudad_id`, de forma que para obtener los pasos de una cofradía basta con buscar por esa columna.
- Las relaciones de muchos a muchos, como las cofradías que participan en varios eventos o los pasos que participan en varios eventos y procesiones, se representan mediante **tablas intermedias**

[5] Clave foránea (*foreign key*): columna de una tabla que referencia la clave primaria de otra tabla, y que la propia base de datos utiliza para garantizar que esa relación siempre apunta a un registro que realmente existe (integridad referencial).

[6] ORM (*Object-Relational Mapping*): técnica que traduce automáticamente entre los objetos de un lenguaje de programación orientado a objetos (Java, en este caso) y las tablas de una base de datos relacional, evitando escribir manualmente la mayoría de las sentencias SQL.

(`eventos_cofradias` y `pasos_eventos`).

- La tabla `ubicaciones` (latitud, longitud y dirección) es independiente y a ella hacen referencia tanto `eventos` como `puntos_ruta`. De esta forma no se repiten estas columnas en cada tabla y se puede reutilizar una misma ubicación, como una iglesia en la que se celebra un evento y por la que pasa una procesión.

- Las herencias del modelo de dominio se representan con una **tabla por cada clase** (estrategia `JOINED` de JPA): `procesiones` extiende a `eventos`, `puntos_de_interes` a `puntos_ruta`, y `administradores` y `miembros_junta_cofradia` a `usuarios`. El identificador de cada subtipo es a la vez su clave primaria y una clave foránea a la tabla padre, evitando así tener una única tabla con muchas columnas vacías según el tipo.

- Los favoritos no tienen tabla, ya que se guardan en el propio móvil, almacenando únicamente el tipo y el identificador del elemento marcado.

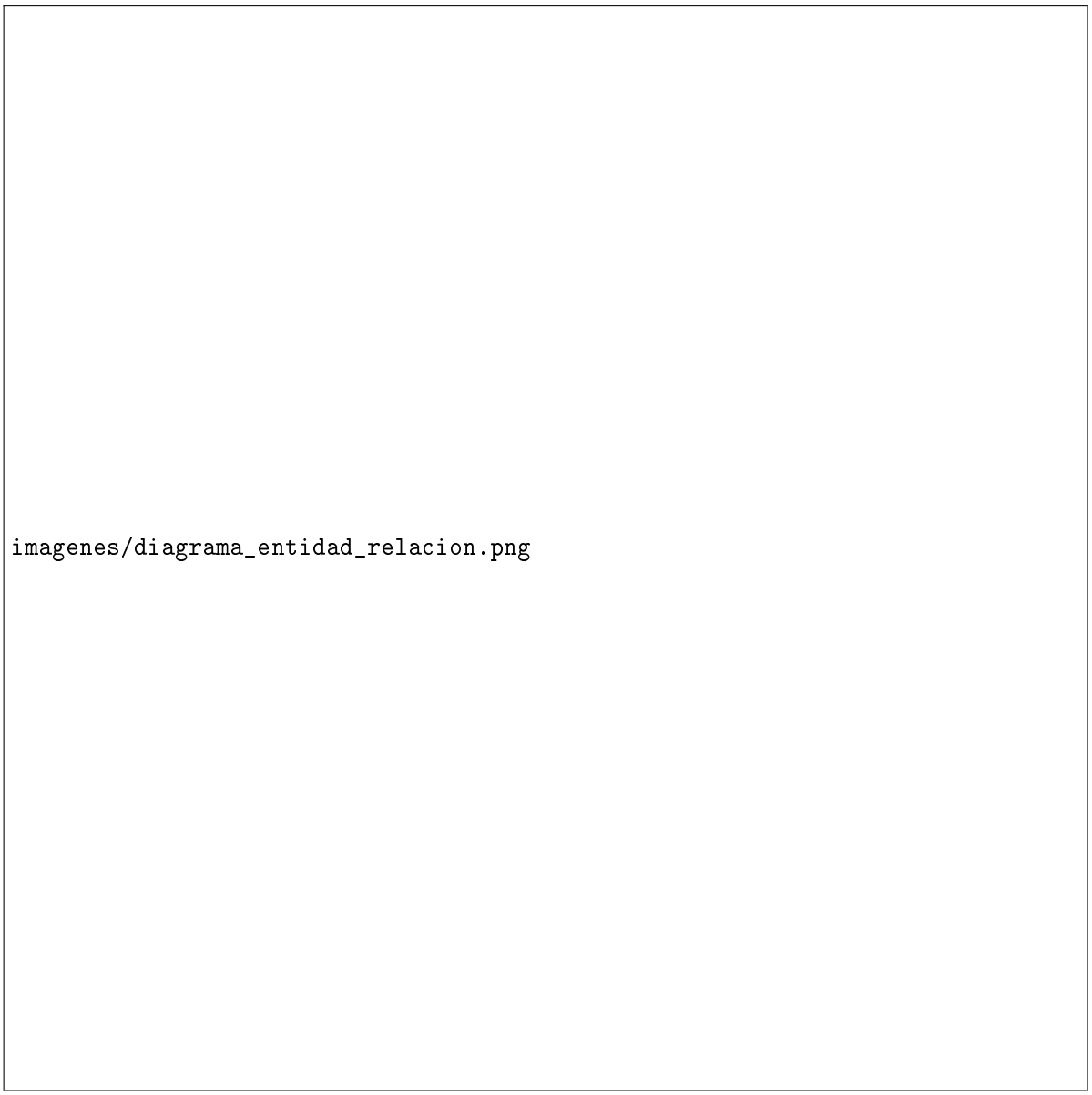


Figura 4.4: Diagrama entidad-relación de la base de datos

*Nota: este diagrama está desactualizado respecto al esquema real (Apéndice **??**): entre otras cosas, no tiene tabla `dispositivos_push`, sigue llamándose `pasos_procesiones` en vez de `pasos_eventos`, y le faltan columnas (`activa` en `ciudades` y `cofradias`, `latitud`/`longitud` en `ciudades`, y varias en `miembros_junta_cofradia`). Estoy pidiendo permisos para la licencia de Astah para poder actualizarlo.*

Al igual que en el back-end, el diagrama entidad-relación de la Figura 4.4 muestra la base de datos completa de la versión final, y en el Apéndice **??** se recoge el esquema de cada tabla, con sus tipos de datos y sus claves primarias y foráneas.

Interfaces de usuario

Paralelamente a las decisiones de arquitectura y de base de datos, se ha diseñado en Figma [**?**] mediante *mockups*[7] el conjunto de pantallas correspondiente al alcance de esta primera iteración según la planificación (Sección 2.2): las funcionalidades fundamentales del perfil de ciudadano, es decir, la visualización de la información general de la Semana Santa, el listado de organizaciones y la consulta de eventos. Siguiendo ese mismo criterio, el resto de pantallas de la aplicación, como el mapa en vivo, el modo cofrade o los paneles de la Junta de Cofradías y del administrador, se diseñan en la iteración a la que corresponden según la planificación, y se documentan en el apartado de Diseño de esa misma iteración.

Antes de entrar en el detalle de cada pantalla, esta primera iteración fija también los criterios visuales comunes a toda la aplicación, para que el resto de iteraciones los reutilicen sin tener que redefinirlos: una estética oscura con acentos dorados, que evoca la iluminación de cirios y el ambiente nocturno característico de las procesiones, y una barra de navegación inferior fija con cinco secciones (*Inicio*, *Calendario*, *Mapa*, *Buscar* y *Perfil*), que se corresponde directamente con el paquete `navigation/` descrito en el apartado de arquitectura. Los listados de cofradías, procesiones, pasos y eventos no tienen una sección propia en la barra de navegación, sino que se accede a ellos desde un menú en la pantalla de *Inicio*, tal y como se explica más adelante.

La paleta de color se apoya en tonos marrón muy oscuro, casi negro, para el fondo (`#1E1008`, `#120A06`, `#34150E`, `#472F17`), dorados como color de acento para títulos, iconos activos y elementos interactivos (`#C9A84C`, `#9A7D60`), tonos crema para tarjetas y textos secundarios (`#F0E0C8`, `#E3D595`) y blanco (`#FFFFFF`) para el texto principal sobre fondo oscuro; un verde puntual (`#37E37E`) se reserva para indicadores de estado, como una procesión en curso. En cuanto a la tipografía, se combinan tres familias: **Cormorant Garamond**, una serifa elegante, para los títulos, coherente con la estética evocada; **Lora**, también serifa pero más neutra, para el texto de lectura extensa (historia de una cofradía, descripción de un paso); y **Sora**, de palo seco, para los elementos de interfaz (botones, etiquetas, barra de navegación), donde interesa más la legibilidad a tamaños pequeños que el carácter estético. Estas decisiones se apoyan, además del criterio propio, en las pautas de *Material Design 3* [**?**], la guía de diseño de interfaz de Google, en particular en lo relativo al contraste de color, el tamaño mínimo táctil de los elementos interactivos y la estructura de la barra de navegación inferior.

Selección de ciudad Es la primera pantalla que ve cualquier usuario, ya que toda la información de la aplicación (cofradías, procesiones, eventos) está filtrada por la ciudad seleccionada, tal y como se refleja en la tabla `ciudades` descrita en el apartado anterior. El usuario puede buscar su ciudad o elegirla de una lista con el número de procesiones disponibles en cada una.

[7] Mockup: representación visual estática de la interfaz de una aplicación, sin funcionalidad real, utilizada para planificar el diseño antes de su implementación.



Figura 4.5: Mockup de la pantalla de selección de ciudad

Inicio Muestra la información general del día de Semana Santa seleccionado: una notificación destaca si existe alguna incidencia relevante (por ejemplo, un retraso en el horario de salida), la procesión que está en curso en ese momento si la hay, unas estadísticas generales de la ciudad (número de cofradías, procesiones y nazarenos) y el listado de procesiones y eventos programados para el día, marcando cuáles son favoritos del ciudadano. Desde el menú contextual de esta pantalla (icono *overflow* superior derecho) se accede, además, a los listados independientes de cofradías, procesiones, pasos y eventos.

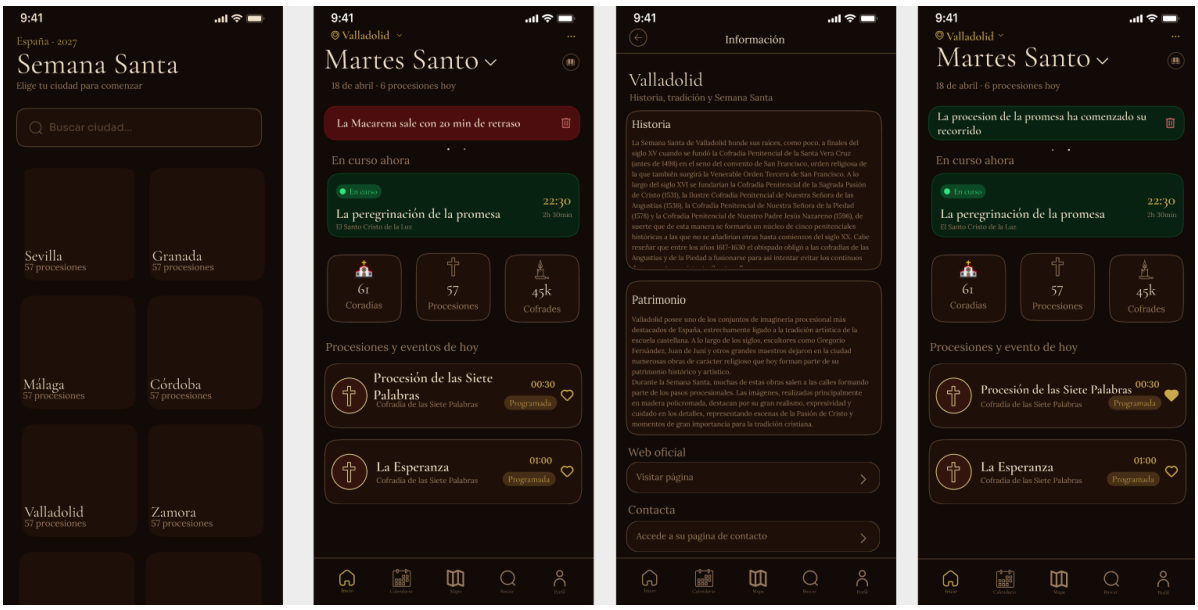


Figura 4.6: Mockup de la pantalla de inicio: notificación de retraso, procesión en curso, estadísticas y agenda del día

Calendario Complementa a la pantalla de inicio permitiendo consultar cualquier otro día de la Semana Santa, no solo el actual. Ofrece dos niveles de detalle: una vista mensual completa, con un indicador en los días que tienen procesiones programadas, y una vista semanal con la agenda del día seleccionado, listando sus procesiones con horario y duración.



Figura 4.7: Mockup de la pantalla de calendario: vista mensual y agenda del día seleccionado

Listados de cofradías, procesiones, pasos y eventos Cada uno de estos cuatro elementos del dominio cuenta con su propio listado, accesible desde el menú de la pantalla de Inicio, con buscador propio y sin más información que el nombre de cada elemento, ya que su detalle se consulta accediendo a la ficha correspondiente.

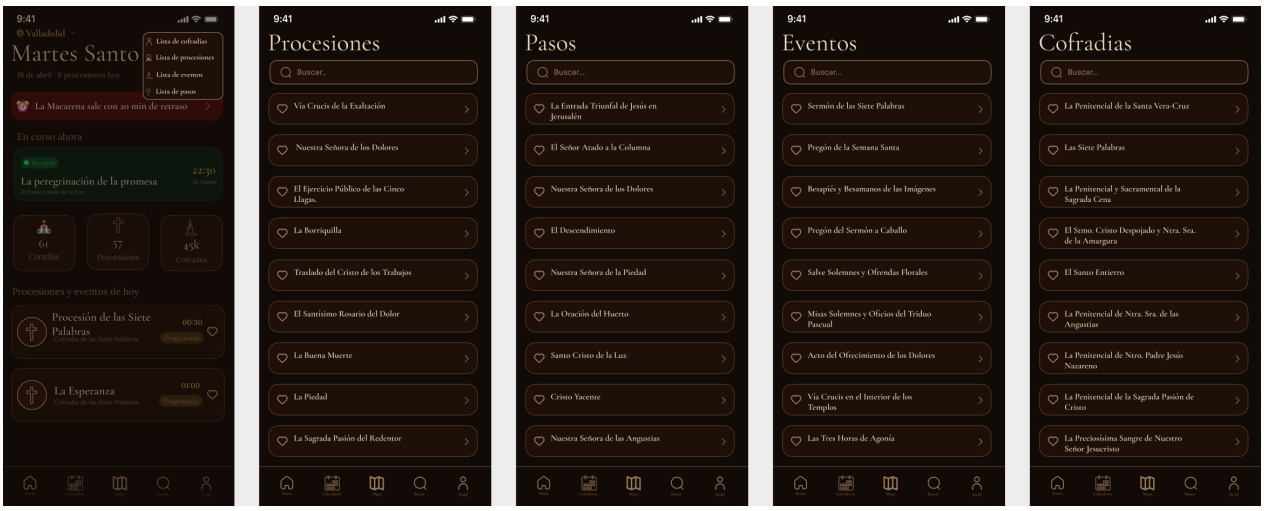


Figura 4.8: Mockup del menú de la pantalla de inicio y de los listados de procesiones, pasos, eventos y cofradías

Detalle de cofradía y de paso La ficha de una cofradía muestra su historia, su web oficial, las procesiones y el evento que organiza y los pasos que la componen. De forma análoga, la ficha de un paso muestra una imagen, su historia y origen, un análisis artístico y su web oficial, siguiendo la misma estructura visual que la ficha de cofradía para mantener la coherencia del conjunto.

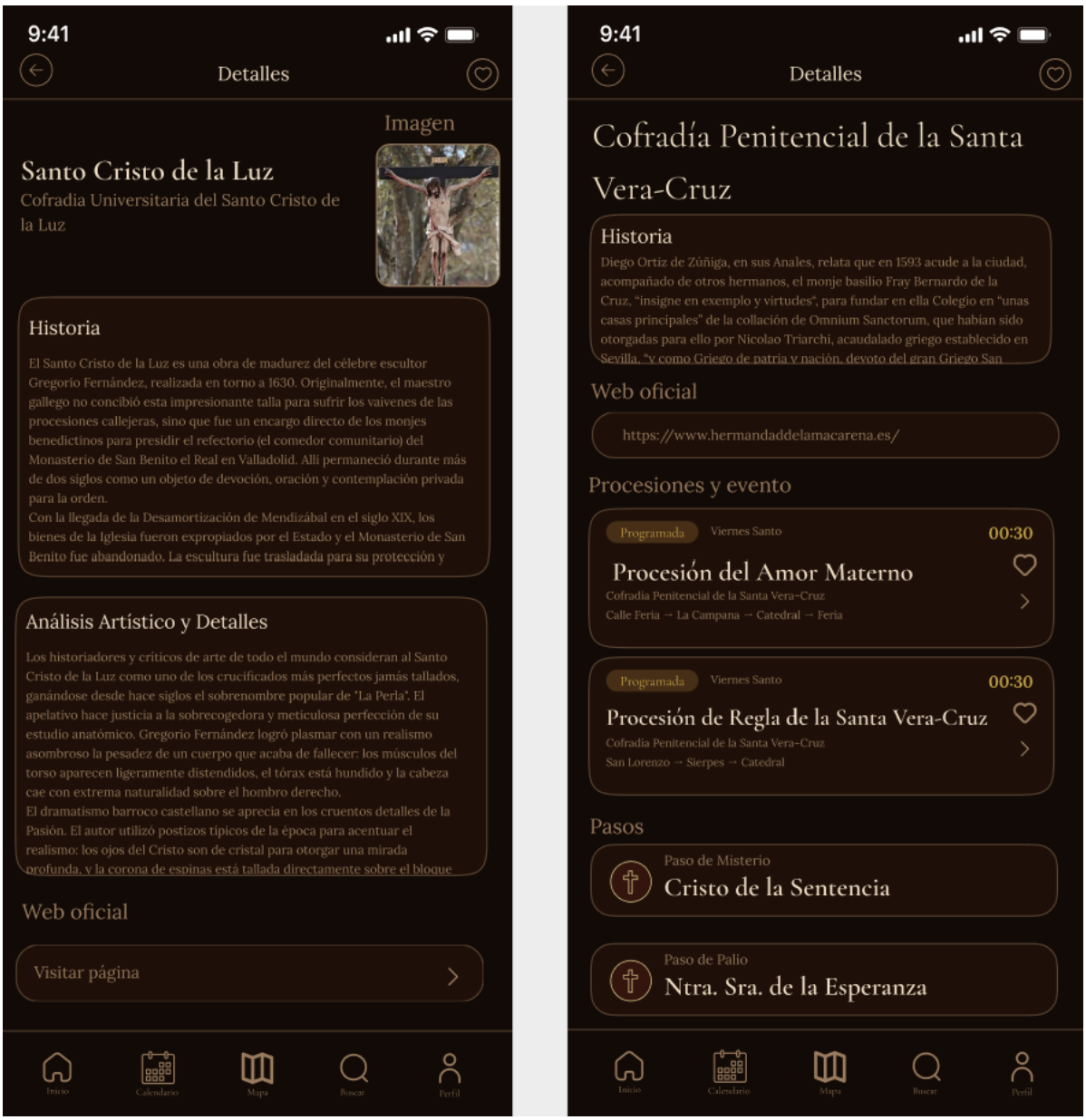


Figura 4.9: Mockup del detalle de un paso y de una cofradía

Detalle de evento La ficha de un evento muestra su nombre, día y hora, duración, ubicación y los pasos que participan en él, siguiendo la misma estructura visual que la ficha de cofradía y de paso.



Figura 4.10: Mockup del detalle de un evento

Detalle de procesión Muestra el estado de la procesión (programada, en curso o finalizada), su día y hora de salida, duración, número de nazarenos, los pasos que participan en ella y su recorrido sobre un mapa, con un botón de acceso directo al mapa en vivo si la procesión está en curso. Además, de forma similar a la ficha de una cofradía, cada procesión tiene una pantalla de información ampliada con la historia y el origen del desfile.

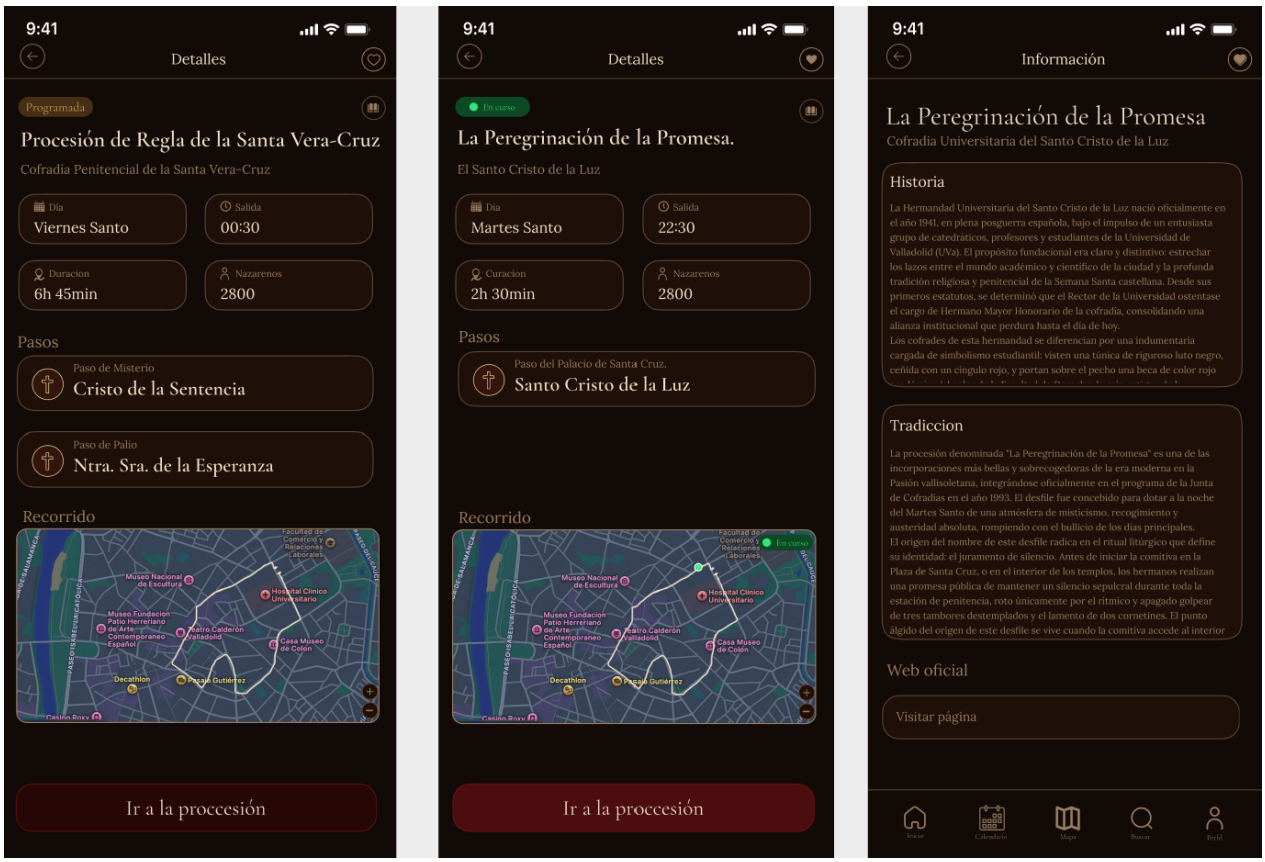


Figura 4.11: Mockup del detalle de una procesión (programada y en curso) y de su información ampliada

Buscar Permite localizar procesiones, pasos y eventos combinando texto libre con filtros por día de Semana Santa y por estado (programada, en curso o finalizada), mostrando un mensaje cuando ningún resultado cumple los filtros seleccionados. Su funcionalidad se implementa en una iteración posterior.

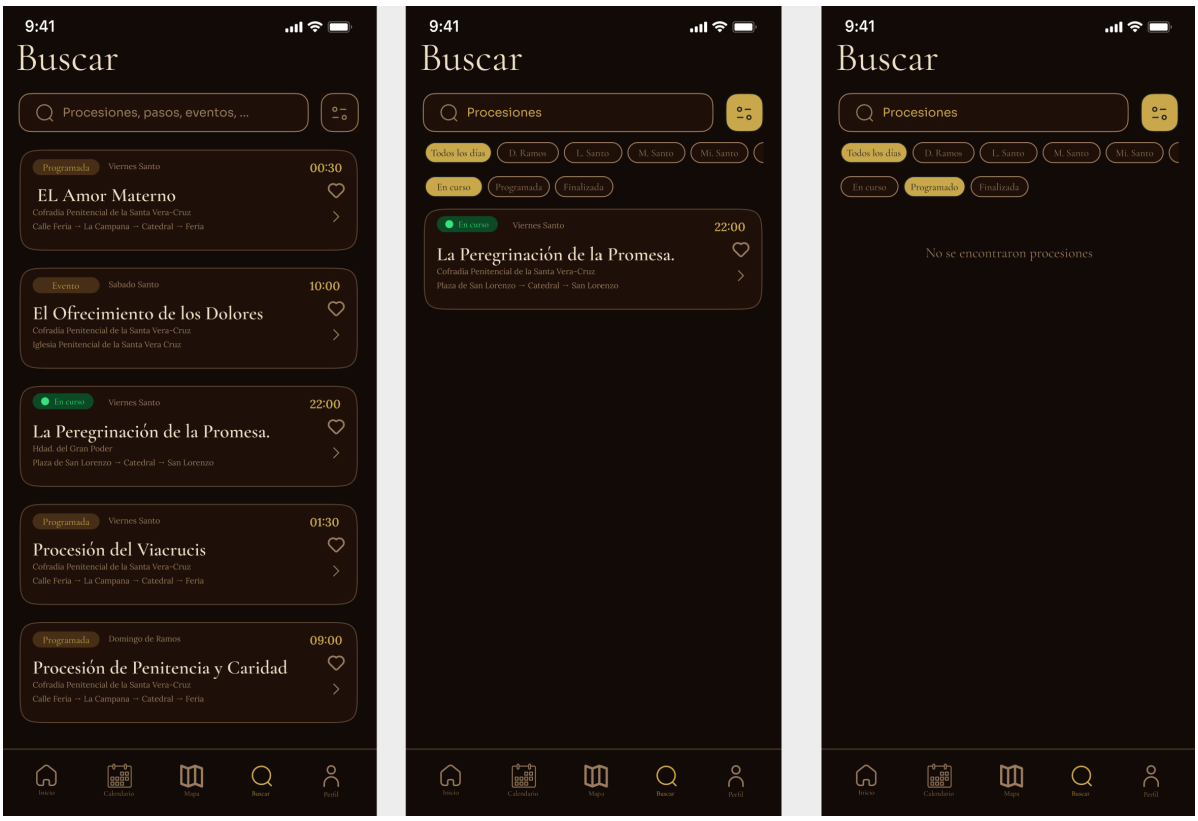


Figura 4.12: Mockup de la pantalla de búsqueda con filtros por día y estado

Patrones de diseño

Un patrón de diseño [?] es una solución ya conocida y reutilizable a un problema que se repite con frecuencia en el desarrollo de software, y su uso facilita la legibilidad, el mantenimiento y la escalabilidad del código. Para elegir los patrones de este proyecto se revisaron dos TFG anteriores de la escuela con una arquitectura parecida (una aplicación móvil apoyada en Firebase). De ellos se descartaron los patrones *Builder* y *Abstract Factory*, ya que en Java se utilizan porque los constructores no admiten bien parámetros opcionales, pero en JavaScript basta con un objeto con esos campos opcionales, y además en ningún punto del sistema es necesario intercambiar familias completas de implementaciones. Los seis patrones que se han aplicado finalmente son los siguientes:

- **Singleton**: patrón que garantiza que una clase tenga una única instancia en toda la aplicación y un punto de acceso global a ella, evitando crear copias innecesarias. En este proyecto, el cliente HTTP de la API (`apiClient`), que añade el token JWT a cada petición, y la caché de TanStack Query (`queryClient`) se crean una única vez y se reutilizan en toda la aplicación.
- **Repository**: patrón que separa el acceso a los datos del resto de la aplicación, ocultando cómo y dónde se guardan. En el cliente, la capa `services` es la única que conoce el cliente HTTP de la API, de forma que el resto de la aplicación solo llama a funciones como `procesionService.getProcesionesEnCur` sin saber que por detrás se está haciendo una petición a la API. En el backend, este patrón se aplica directamente en la capa `repository`, ya que las interfaces de Spring Data JPA son una implemen-

tación del propio patrón Repository. Gracias a esto, se podría cambiar la forma de guardar los datos sin tener que modificar las pantallas ni los controladores.

- **Observer**: patrón en el que uno o varios elementos (observadores) se suscriben a los cambios de otro y reciben un aviso automáticamente cuando este cambia, sin tener que preguntar continuamente. En este proyecto se aplica a la recepción de notificaciones *push*, ya que la aplicación se registra como observadora de las notificaciones que llegan (`Notifications.setNotificationHandler` de Expo Notifications) y recibe cada una de ellas automáticamente, sin tener que preguntar al servidor si hay alguna nueva. En cambio, este patrón no se aplica a la posición de las procesiones, ya que en ese caso es la propia aplicación la que pregunta al servidor cada 30 segundos.

- **Context/Provider**: mecanismo propio de React (no es un patrón GoF[8] clásico) que permite compartir un dato o un estado con cualquier pantalla de la aplicación sin tener que pasarlo de componente en componente. De esta forma, `AuthContext`, `CiudadContext` y `CofradeContext` comparten con las pantallas que lo necesiten la sesión del usuario, la ciudad seleccionada y si el cofrade está compartiendo su ubicación.

- **Facade**: patrón que ofrece una única función sencilla para realizar una operación que, por detrás, necesita varios pasos. En el caso de uso "Compartir ubicación durante procesión" (RF-24), la función `seguimientoService.iniciarSeguimiento` se encarga de pedir el permiso de ubicación, de enviar periódicamente la posición a la API, incluso con el móvil bloqueado, y de gestionar una posible pérdida de conexión.

- **Adapter**: patrón que convierte el formato de datos de un sistema externo al formato que espera la aplicación, de forma que ambos puedan funcionar juntos sin modificar ninguno de ellos. Al importar el recorrido de una procesión, el backend recibe un archivo GPX, un formato estándar que generan herramientas como gpx.studio, Wikiloc o Strava. La clase `GpxParser` lee este archivo y lo convierte en la lista ordenada de puntos (latitud y longitud) que necesita `RecorridoService` para crear el `Recorrido`.

4.1.3. Implementación

Se ha implementado el backend propio descrito en el apartado de diseño anterior: un proyecto Spring Boot [**?**] generado con Spring Initializr, organizado en los paquetes `model`, `repository`, `service`, `controller`, `dto`, `security` y `exception` que siguen la estructura de capas ya justificada, y con el esquema de PostgreSQL gestionado mediante migraciones versionadas de Flyway [**?**] (un script SQL numerado por cada cambio de esquema), en lugar de dejar que Hibernate genere o modifique las tablas automáticamente; así el esquema queda documentado paso a paso y se aplica igual en local, en el servidor de despliegue y en cualquier entorno nuevo, tal y como recoge el manual de despliegue (Apéndice **??**).

Sobre esa base se han implementado las cuatro capas para todas las entidades necesarias para el módulo de ciudadano de esta primera iteración –`Ciudad`, `JuntaCofradias`, `Cofradia`, `Paso` y `Evento`–, cada una

[8] GoF (*Gang of Four*): apodo con el que se conoce a los cuatro autores del libro *Design Patterns* [**?**] (1994), que popularizó el catálogo clásico de patrones de diseño orientado a objetos (Singleton, Observer, Factory Method, entre otros).

con su correspondiente conjunto de *endpoints* REST de consulta y gestión (`GET`, `POST`, `PUT`, `DELETE`), documentados automáticamente en `/swagger-ui.html` mediante springdoc-openapi [**?**]. `JuntaCofradias` se implementa en esta misma iteración por ser, igual que `Ciudad`, un dato base del que depende `Cofradia` (Apéndice **??**), aunque su gestión efectiva –dar de alta, editar o dar de baja una Junta– corresponda al panel de administrador de la Iteración 5.

Durante esta implementación se han detectado varias necesidades no recogidas en el diseño inicial de la base de datos, que se han ido resolviendo sobre la marcha y quedan documentadas con su justificación en el Apéndice **??**: entre ellas, que la información de una cofradía y de un paso necesitaba más campos de los previstos inicialmente (`web` en lugar de un archivo adjunto, `analisis_artistico` para el paso), o que `Cofradia` debía depender de `Ciudad` y no de `JuntaCofradias`, para que la información de las cofradías de una ciudad no se viera afectada por una renovación de su Junta.

Queda pendiente para el cierre de esta iteración la implementación de las pantallas de cliente correspondientes sobre esta API, que hasta ahora consumían datos de prueba (*mock*).

4.1.4. Pruebas

En esta fase del proyecto la verificación del backend ha sido manual: compilación exitosa tras cada cambio (`mvn compile`) y comprobación funcional de los *endpoints* mediante la documentación interactiva de springdoc-openapi (`/swagger-ui.html`) contra una instancia de PostgreSQL local levantada con Docker. Queda pendiente una batería de pruebas automatizadas (pruebas unitarias de la capa `service` y pruebas de integración de los controladores con `Spring Boot Test`), que se abordará más adelante y se documentará en el Capítulo **??**.

4.2. 2ª Iteración

4.2.1. Análisis

Según la planificación (Sección 2.2), esta segunda iteración se centra en el sistema de mapas, que permite ver sobre un mapa el recorrido de las procesiones y la ubicación de los eventos, e interactuar con él (RF-15 y RF-17). Para ello, antes de empezar con el diseño se analizaron los problemas que había que resolver, valorando para cada uno de ellos las distintas alternativas posibles y eligiendo la que mejor se adaptaba al proyecto.

Problemas planteados y alternativas valoradas

Mapas Se valoraron Google Maps Platform [**?**], OpenStreetMap [**?**] y Mapbox. Se optó por la librería react-native-maps, que utiliza Google Maps en Android y el mapa propio de Apple en iOS, ya que está bien integrada con Expo y aprovecha el mapa que incluye cada sistema. OpenStreetMap y Mapbox se mantienen como plan de contingencia ante el riesgo de tener problemas con Google Maps (Sección 2.3).

Cómo guardar el recorrido Un recorrido es una línea formada por muchos puntos, por lo que se valoró guardarlo entero como una lista de coordenadas dentro del propio recorrido, o guardar cada punto por separado. La primera opción es más sencilla, pero no permite añadir información a un punto concreto ni reutilizarlo. Por ello se eligió guardar cada punto por separado (`PuntoRuta`), con su posición en una `Ubicacion` independiente, ya que algunos puntos necesitan información propia y un mismo punto puede aparecer en varios recorridos, como una calle por la que pasan varias procesiones.

Puntos de interés Para los puntos que merece la pena destacar en el mapa, como una iglesia o el lugar de un encuentro (RF-17), se valoró añadir sus datos (nombre, descripción, imagen y tipo) a todos los puntos del recorrido, o crear un tipo de punto especial. Se optó por crear `PuntoDeInteres` como un tipo de `PuntoRuta`, ya que la gran mayoría de los puntos de un recorrido no son de interés y tendrían todos esos datos vacíos.

En la Tabla 4.2 se resumen todas estas decisiones.

Problema	**Alternativas valoradas**	**Opción elegida**
Mapas	Google Maps, OpenStreetMap, Mapbox	react-native-maps
Guardar el recorrido	Lista de coordenadas en el recorrido, cada punto por separado	Cada punto por separado (`PuntoRuta`)
Puntos de interés	Datos en todos los puntos, tipo de punto especial	`PuntoDeInteres` como tipo de `PuntoRuta`

Tabla 4.2: Problemas analizados en la segunda iteración y opción elegida

4.2.2. Diseño

Una vez elegidas las alternativas en el análisis, en este apartado se explica cómo se ha diseñado el sistema de mapas sobre la arquitectura definida en la Iteración 1 (Sección 4.1): cómo se muestran los mapas, cómo se guardan los recorridos y cómo es la nueva pantalla.

Mapas

Los mapas se muestran mediante componentes propios de la capa UI que utilizan react-native-maps, como el mapa de un recorrido o el de la ubicación de un evento, de forma que el resto de pantallas no depende directamente de la librería. Sobre el mapa se dibuja el recorrido de cada procesión como una línea que une, en orden, los puntos de su `Recorrido`, y se marcan con un icono los puntos de interés y la ubicación de los eventos. En Android se utiliza Google Maps y en iOS el mapa propio de Apple, sin que esto cambie nada en el resto de la aplicación.

Diseño de la base de datos

- Cada punto del recorrido se guarda en la tabla `puntos_ruta`, que hace referencia a su ubicación en la tabla `ubicaciones`. Los puntos de interés se guardan en `puntos_de_interes`, que extiende a `puntos_ruta` siguiendo el criterio de herencia definido en la Iteración 1.

- Como un mismo punto puede formar parte de varios recorridos, la relación entre `recorridos` y `puntos_ruta` se guarda en una tabla intermedia, `recorrido_puntos_ruta`. Esta tabla guarda además el orden y la hora prevista de paso del punto en ese recorrido concreto, ya que un mismo punto puede ocupar una posición distinta en cada recorrido (Apéndice **??**).

Interfaces de usuario

Según la planificación (Sección 2.2), esta segunda iteración se centra en el sistema de mapas, por lo que la única pantalla nueva que se diseña en Figma [**?**] en esta iteración es el mapa en vivo, apoyada en los mismos criterios visuales (color, tipografía y navegación) ya fijados en la Iteración 1 (Sección 4.1).

Mapa en Vivo Muestra sobre un mapa de Google Maps la posición en tiempo real de las procesiones en curso y la ubicación del propio ciudadano, junto con un listado de las procesiones actualmente en movimiento desde el que centrar el mapa sobre una de ellas.

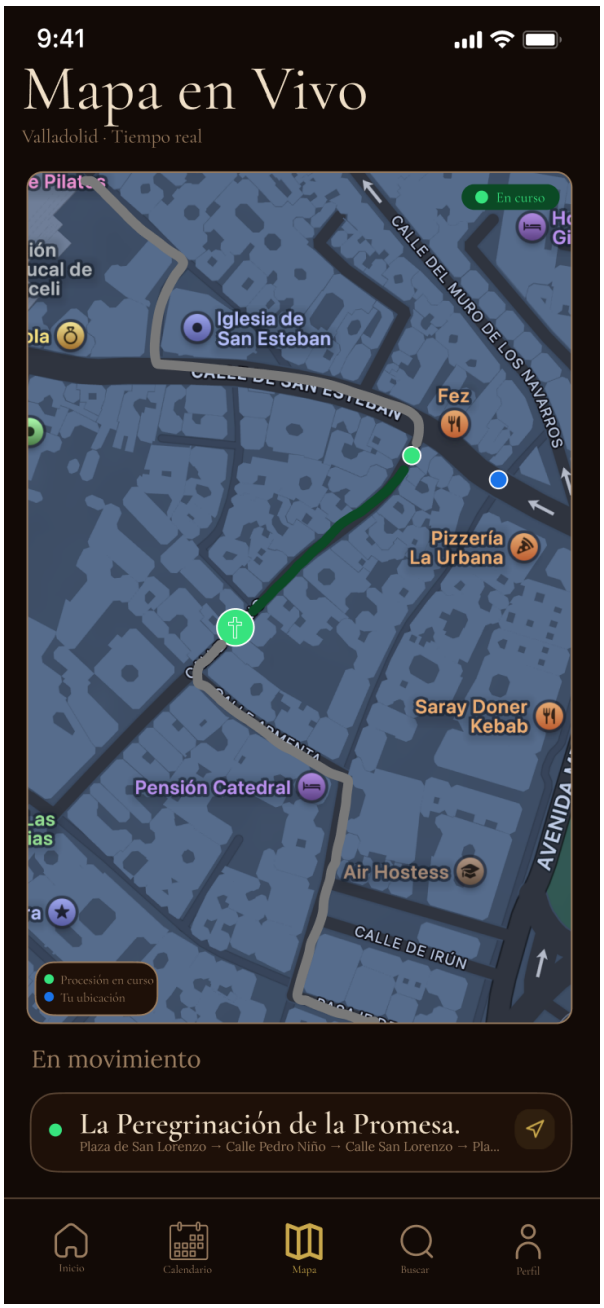


Figura 4.13: Mockup del mapa en vivo

4.2.3. Implementación

En esta segunda iteración se implementan en el back-end las entidades necesarias para el sistema de mapas: `Recorrido`, `Ubicacion`, `PuntoRuta` y `PuntoDeInteres`, siguiendo la misma estructura de capas definida en la Iteración 1. Cada una de ellas tiene sus propios *endpoints* (`/recorridos`, `/ubicaciones`, `/puntos-ruta` y `/puntos-de-interes`), y desde `/recorridos/{id}/puntos-ruta` se obtienen, en orden, los puntos que forman un recorrido.

En la aplicación se añaden los componentes de mapa, que se utilizan en la ficha de una procesión para mostrar su recorrido y en la ficha de un evento para mostrar dónde se celebra, además de la pantalla del

mapa en vivo. En esta iteración el mapa en vivo muestra el recorrido de las procesiones, mientras que su posición en tiempo real se añade en la Iteración 3, junto con la geolocalización.

4.2.4. Pruebas

Al igual que en la Iteración 1, la verificación de estas entidades se ha realizado de forma manual mediante la documentación interactiva de springdoc-openapi (`/swagger-ui.html`), y probando los mapas en la aplicación desde el móvil. Las pruebas automatizadas se recogen en el Capítulo **??**.

4.3. 3ª Iteración

4.3.1. Análisis

Según la planificación (Sección 2.2), esta tercera iteración se centra en la geolocalización en tiempo real (RF-16 y RF-18) y en el perfil de cofrade, que permite acceder con el código de la cofradía y compartir la ubicación durante una procesión (RF-23 a RF-25). Al igual que en las iteraciones anteriores, antes de empezar con el diseño se analizaron los problemas que había que resolver, valorando para cada uno de ellos las distintas alternativas posibles.

Problemas planteados y alternativas valoradas

Identificación del cofrade Para controlar lo que puede hacer cada rol (RNF-03) se valoró guardar la sesión de cada usuario en el servidor o utilizar tokens JWT [**?**]. Se optó por los tokens JWT, ya que el servidor no necesita recordar ninguna sesión, sino que cada petición incluye un token firmado con el rol del usuario y el servidor solo tiene que comprobarlo. En el caso del cofrade también se plantearon dos opciones: que se registrara con una cuenta propia o que accediera con un código de su cofradía. Se eligió el código de acceso sin registro, ya que los cofrades solo utilizan la aplicación durante la Semana Santa y no tenía sentido obligarles a crearse una cuenta ni guardar sus datos personales.

Introducción del código de acceso Para que el cofrade introduzca el código se valoró que lo escribiera a mano o que lo escaneara como un código QR. Escribirlo es lo más sencillo, pero el día de la procesión hay muchos cofrades accediendo a la vez y es fácil equivocarse. Por ello se optó por ofrecer las dos opciones: la Junta puede mostrar el código también como QR y el cofrade puede escanearlo con la cámara o escribirlo si lo prefiere. Además, para evitar que un código compartido siga sirviendo indefinidamente, se decidió que cada código caduque 24 horas después de emitirse.

Envío y consulta de la posición Para que el ciudadano pueda ver cómo avanzan las procesiones (RF-18) había que elegir entre un mecanismo en el que el servidor avisa al momento de cada cambio, como

WebSockets o *Server-Sent Events*, o que la aplicación pregunte al servidor cada cierto tiempo (*polling*). Como el requisito RNF-05 pide una actualización cada 30 segundos como mínimo, se optó por la consulta periódica, que es mucho más sencilla de desarrollar y suficiente para cumplir el requisito. También había que decidir cómo guardar las posiciones: una única posición por procesión que se va sobrescribiendo, o un histórico con todas las lecturas. Se eligió el histórico, ya que hay varios cofrades compartiendo su ubicación a la vez en distintos puntos de la procesión, y con todas sus lecturas se puede calcular mejor dónde se encuentra. Además, cada lectura se guarda de forma anónima, cumpliendo así el requisito RNF-01.

Recogida de las posiciones y estela de la procesión Una procesión no ocupa un único punto, sino un tramo de calle que puede medir varios cientos de metros, por lo que no bastaba con mostrar un punto en el mapa. Por ello se decidió pintar sobre el recorrido una **estela** que indique qué parte ya ha recorrido la procesión, qué parte ocupa en ese momento y qué parte le queda por recorrer. Para conseguirlo se valoraron tres alternativas:

- **Recoger las posiciones de todos los cofrades y pintarlas todas en el mapa.** Es la opción más sencilla, ya que no requiere ningún cálculo, pero con muchos cofrades compartiendo a la vez el mapa se llenaría de puntos difíciles de interpretar, algunos de ellos fuera del recorrido por los errores del GPS. Además, no indica qué parte del recorrido se ha completado, y mostrar la posición exacta de cada persona no encaja con la idea de que la ubicación sea anónima (RNF-01).
- **Que solo enviaran su posición el primer y el último cofrade de la procesión.** Con estas dos posiciones se puede pintar el tramo que ocupa la procesión entre ambas. Sin embargo, la Junta tendría que decidir antes de cada procesión qué personas van a ir en cabeza y en cola, y si a una de ellas se le acaba la batería, se queda sin cobertura o se le olvida activarlo, la estela deja de funcionar. Además, como los cofrades acceden de forma anónima con el código de su cofradía, el sistema no puede saber quién va realmente delante o detrás.
- **Que todos los cofrades envíen su posición y que el sistema calcule cuál es la cabeza y cuál la cola.** Cada posición recibida se sitúa sobre el recorrido para saber cuánto ha avanzado, y la más adelantada se toma como la cabeza de la procesión y la más atrasada como la cola. De esta forma no hace falta asignar ningún papel a ningún cofrade, y si uno de ellos deja de enviar su posición, los demás siguen alimentando la estela.

Finalmente se optó por la tercera alternativa, mediante un algoritmo que se explica en el apartado de diseño. Además, para situar la procesión con un único marcador en el mapa se utiliza el promedio de las posiciones recibidas en el último minuto, que es mucho más estable que mostrar la última posición recibida, la cual saltaría de un lado a otro de la procesión según qué cofrade la hubiera enviado. Por último, para descartar los errores del GPS o las lecturas de personas que no van en la procesión, se decidió ignorar las posiciones que estén a más de 50 metros del recorrido (RF-25).

Envío con la pantalla bloqueada Durante la procesión el cofrade no va a tener la aplicación abierta todo el tiempo, por lo que se valoró enviar la ubicación solo mientras la aplicación está abierta o seguir

enviándola también en segundo plano. Se optó por seguir enviándola en segundo plano cuando el cofrade concede el permiso de ubicación permanente, y solo con la aplicación abierta en caso contrario, tal y como se indica en el requisito RF-24.

En la Tabla 4.3 se resumen todas estas decisiones.

Problema	Alternativas valoradas	Opción elegida
Identificación del cofrade	Sesión en el servidor, JWT; cofrade con cuenta o con código	JWT; cofrade con código sin registro
Introducción del código	Escribirlo, escanear un QR	Las dos opciones; caduca a las 24 horas
Envío y consulta de la posición	WebSockets/SSE, consulta periódica; una posición o histórico	Consulta cada 30 segundos; histórico anónimo
Estela de la procesión	Pintar todas las posiciones, que solo envíen el primero y el último, calcular la cabeza y la cola a partir de todas	Calcular la cabeza y la cola a partir de todas las posiciones
Envío con la pantalla bloqueada	Solo con la aplicación abierta, también en segundo plano	En segundo plano si hay permiso permanente

Tabla 4.3: Problemas analizados en la tercera iteración y opción elegida

4.3.2. Diseño

Una vez elegidas las alternativas en el análisis, en este apartado se explica cómo se ha diseñado el acceso del cofrade y la geolocalización en tiempo real sobre la arquitectura definida en la Iteración 1 (Sección 4.1).

Autenticación del cofrade

La autenticación y la autorización de la API se realizan con Spring Security [**?**] y tokens JWT (*JSON Web Token*) [**?**]. El cofrade accede introduciendo o escaneando el código de acceso de su cofradía (`POST /auth/codigo-acceso`), y si el código existe, no está revocado ni ha caducado y su cofradía está activa, el servidor le devuelve un token firmado que indica su rol y la cofradía a la que pertenece, sin identificar en ningún momento a la persona. La aplicación envía este token en cada petición a una operación protegida, y el servidor lo comprueba antes de permitirla. Si el token caduca o el código se revoca, el servidor indica que la sesión ya no es válida y la aplicación deja de compartir la ubicación y vuelve a la pantalla de inicio de sesión. Como el servidor no guarda ninguna sesión, el sistema resulta más sencillo de desplegar.

Posición en tiempo real

Siguiendo la consulta periódica elegida en el análisis, mientras el cofrade comparte su ubicación, la aplicación envía su posición cada 30 segundos mediante `POST /procesiones/{id}/posiciones`. El servidor sitúa esa posición sobre el tramo más cercano del recorrido y la descarta si se encuentra a más de 50 metros (RF-25). Si es válida, la guarda como una nueva lectura en el histórico.

Por otro lado, la aplicación del ciudadano consulta cada 30 segundos la posición de cada procesión en curso (`GET /procesiones/{id}/ubicacion`), calculada como el promedio de las lecturas recibidas en el último minuto, y el tramo que ocupa sobre el recorrido (`GET /procesiones/{id}/estela`), cumpliendo así el requisito RNF-05. Si en el futuro se quisiera una actualización instantánea o dar servicio a muchos más usuarios a la vez, se podría cambiar a WebSockets, tal y como se recoge en el trabajo futuro (Sección **??**).

Algoritmo de la estela

Para calcular la estela, el recorrido se trata como una lista ordenada de puntos, de forma que cada par de puntos consecutivos forma un *tramo* recto, y el avance de cualquier punto del recorrido se expresa como la fracción de la longitud total que hay desde la salida hasta él, siendo 0 la salida y 1 la llegada. Cada vez que el ciudadano consulta la estela de una procesión, el servidor sigue estos pasos:

1. **Obtener las lecturas recientes.** Se toman las posiciones que han enviado los cofrades de esa procesión en el último minuto, de forma que solo cuentan los cofrades que están compartiendo su ubicación en ese momento.

2. **Situar cada lectura sobre el recorrido.** Para cada lectura se recorren todos los tramos del recorrido y, en cada uno, se calcula el punto del tramo más cercano a la lectura, proyectándola de forma perpendicular sobre él. Como los tramos son cortos, para este cálculo se puede tratar la zona como si fuera un plano. Después se calcula la distancia real entre la lectura y ese punto con la fórmula de Haversine[9], y los metros que hay desde la salida hasta él.

3. **Elegir el tramo correcto.** Normalmente se elige el tramo más cercano a la lectura, que es también el que se utiliza para descartar las lecturas a más de 50 metros. Sin embargo, la mayoría de las procesiones salen y vuelven al mismo templo, y algunas pasan dos veces por la misma calle, de forma que una lectura junto al templo está igual de cerca del primer tramo que del último. Si se eligiera el último, la procesión aparecería como terminada nada más salir. Para evitarlo, entre los tramos que están casi igual de cerca (con una diferencia de menos de 30 metros, que es del orden del error del GPS de un móvil) se elige el que está más próximo al avance que ya había alcanzado la procesión.

4. **Calcular el avance de cada lectura.** El avance es la distancia desde la salida hasta el punto elegido dividida entre la longitud total del recorrido, obteniendo así un valor entre 0 y 1.

5. **Obtener la cabeza y la cola.** De todas las lecturas recientes, la que tiene un avance mayor corresponde a la cabeza de la procesión y la que tiene un avance menor corresponde a la cola.

6. **Guardar el avance sin que retroceda.** El avance de la cabeza y el de la cola se guardan en la propia procesión, pero solo se actualizan si el nuevo valor es mayor que el que ya había. De esta forma, si el cofrade que iba en cabeza pierde la cobertura unos segundos o bloquea el móvil, la

[9] Fórmula de Haversine: fórmula que calcula la distancia entre dos puntos de la superficie terrestre a partir de su latitud y su longitud, teniendo en cuenta que la Tierra es esférica.

estela no retrocede ni da saltos, sino que se mantiene el último valor conocido hasta que llegue una lectura que la haga avanzar.

Con estos dos valores, la aplicación del ciudadano pinta el recorrido en tres partes de distinto color: desde la salida hasta la cola, el tramo que ya ha recorrido toda la procesión; desde la cola hasta la cabeza, el tramo que ocupa en ese momento; y desde la cabeza hasta la llegada, el tramo que todavía le queda por recorrer.

Envío en segundo plano

Para seguir enviando la ubicación con la pantalla bloqueada se utiliza una tarea en segundo plano (`expo-task-manager`), que el sistema operativo despierta cada vez que hay una nueva posición. En Android, mientras se comparte la ubicación se muestra una notificación fija que avisa al cofrade de ello, como exige el propio sistema. La aplicación se encarga de detener esta tarea cuando el cofrade deja de compartir, cuando la procesión finaliza o cuando su código deja de ser válido, informándole del motivo (RF-24).

Diseño de la base de datos

- La tabla `posiciones_actuales` es un **histórico de lecturas** independiente de `procesiones`, en la que cada posición enviada por un cofrade es una fila nueva que solo hace referencia a la procesión, y no a quién la envió. Al estar en una tabla propia, estas escrituras tan frecuentes no afectan a los datos de la procesión, que apenas cambian.
- La tabla `procesiones` extiende a `eventos` y guarda, además de su recorrido, el avance máximo alcanzado por la cabeza y por la cola de la procesión, que se utiliza para pintar el tramo que ocupa.
- La tabla `codigos_acceso` tiene una clave foránea a la cofradía a la que da acceso y un estado (`EMITIDO`, `VALIDADO` o `REVOCADO`). Además, guarda su fecha de emisión y su fecha de caducidad, ya que cada código solo sirve durante las 24 horas siguientes a su emisión. Al validar un código no se crea ningún usuario, sino que el servidor devuelve directamente un token asociado a esa cofradía.

Interfaces de usuario

Según la planificación (Sección 2.2), esta tercera iteración cubre la geolocalización en tiempo real y el perfil de cofrade, por lo que la pantalla que se diseña en Figma [**?**] en esta iteración es el perfil, ya con su variante de modo cofrade, sobre los mismos criterios visuales fijados en la Iteración 1 (Sección 4.1).

Perfil Reúne la ciudad seleccionada y las cofradías, eventos, procesiones o pasos favoritos del ciudadano, el cambio entre el modo ciudadano y el modo cofrade, y un apartado de preferencias de notificaciones por tipo (procesiones en curso, incidencias). En modo cofrade se añade además el botón para

activar la ubicación compartida durante una procesión. Este apartado de preferencias por tipo se descartó más adelante, ya que un interruptor propio dentro de la aplicación sería redundante con el permiso de notificaciones del propio móvil. En la versión final, el perfil muestra si este permiso está activado y ofrece un acceso directo a los ajustes del móvil para cambiarlo (RF-22).

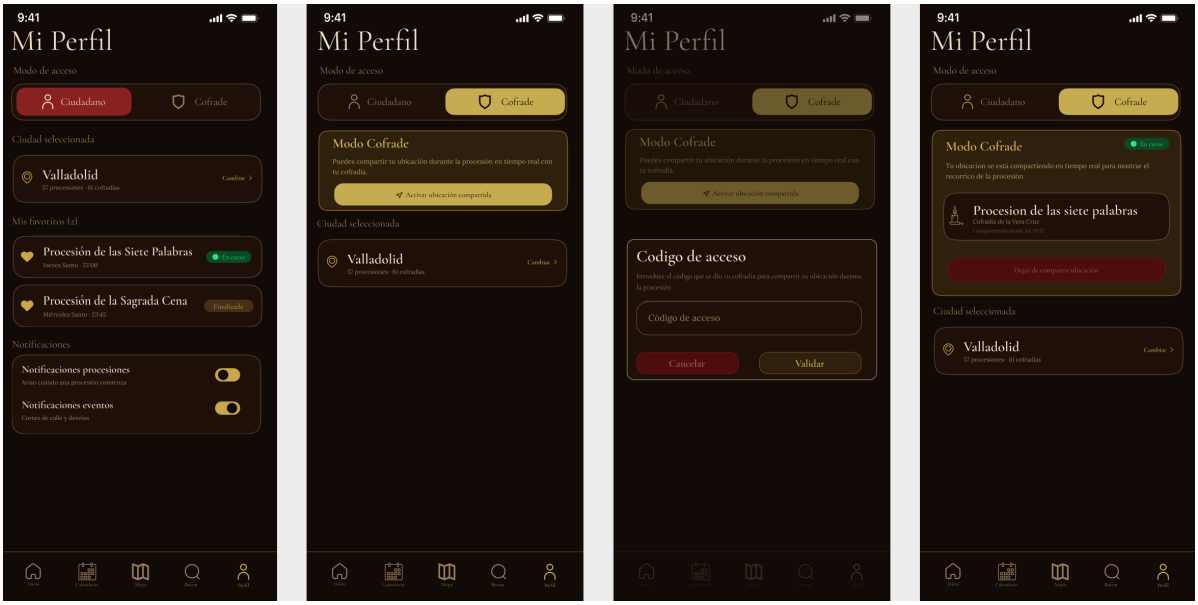


Figura 4.14: Mockup de la pantalla de perfil, en modo ciudadano y en modo cofrade

4.3.3. Implementación

Esta tercera iteración implementa la geolocalización en tiempo real y el perfil de cofrade, para lo que hacen falta dos piezas que hasta ahora no eran necesarias: la entidad `Procesion` y, por primera vez en el proyecto, un mecanismo de autenticación, ya que hasta este momento todas las consultas eran públicas.

`Procesion` se implementa como una extensión de `Evento` (herencia con estrategia `JOINED` de JPA, Apéndice **??**), de forma que hereda su nombre, historia, fechas, estado, ubicación y cofradías participantes, y añade su `Recorrido`, que es opcional porque una procesión puede crearse antes de tener el recorrido definido, y el avance de la cabeza y de la cola.

Para el acceso al modo cofrade (caso de uso “Iniciar sesión con código de acceso”, Sección 3.4.1) se implementa `CodigoAcceso` junto con la autenticación: los tokens JWT se generan y se validan con la librería `jjwt`, un filtro de Spring Security comprueba el token de cada petición y el *endpoint* `POST /auth/codigo-acceso` valida el código y devuelve un token asociado a la cofradía, sin crear ningún usuario (Sección **??**). De esta forma, el `Cofrade` del modelo de dominio no se guarda en la base de datos, sino que se obtiene del token en cada petición. La caducidad y la revocación del código se comprueban tanto al iniciar sesión como en cada envío de ubicación. Además, la aplicación muestra a la Junta cada código también como QR, y el cofrade puede escanearlo con la cámara (mediante la librería `expo-camera`) o escribirlo a mano, enviando en ambos casos el mismo código al mismo *endpoint*.

Por último, se implementa la funcionalidad de compartir la ubicación (RF-24) mediante el *endpoint*

`POST /procesiones/{id}/posiciones`, que guarda cada posición del cofrade como una lectura anónima en `PosicionActual` (RI-06). Al recibirla solo se comprueba que quien la envía es un cofrade de alguna de las cofradías que participan en esa procesión, y se descarta si está a más de 50 metros del recorrido. A partir de este histórico, `GET /procesiones/{id}/ubicacion` calcula la posición de la procesión como el promedio de las lecturas del último minuto, y `GET /procesiones/{id}/estela` devuelve el avance de la cabeza y de la cola, con el que la aplicación pinta sobre el recorrido el tramo ya recorrido, el que ocupa la procesión y el que le queda. En la aplicación, el envío se realiza con una tarea en segundo plano, de forma que la ubicación se sigue enviando aunque el móvil esté bloqueado si el cofrade concede el permiso de ubicación permanente.

4.3.4. Pruebas

Al igual que en las iteraciones anteriores, la verificación del back-end se ha realizado de forma manual mediante springdoc-openapi (`/swagger-ui.html`), obteniendo un token válido con `POST /auth/codigo-acceso` y utilizándolo en las peticiones protegidas. Además, para probar el seguimiento en tiempo real sin tener que salir a la calle, se ha utilizado un simulador que envía posiciones a lo largo del recorrido de una procesión como si fuera un cofrade. Las pruebas automatizadas se recogen en el Capítulo **??**.

4.4. 4ª Iteración

4.4.1. Análisis

Según la planificación (Sección 2.2), esta cuarta iteración se centra en el panel de gestión de la Junta de Cofradías, desde el que sus miembros pueden crear, editar y eliminar las cofradías, procesiones, eventos y pasos de su ciudad, importar los recorridos y emitir los códigos de acceso (RF-26 a RF-33). Al ser el primer rol que puede modificar la información de la aplicación, en esta iteración se analizaron sobre todo los problemas relacionados con el acceso y los permisos, además de cómo introducir los recorridos.

Problemas planteados y alternativas valoradas

Acceso de la Junta de Cofradías Se valoró que la Junta accediera con un código compartido, igual que el cofrade, o que cada miembro tuviera su propia cuenta con correo electrónico y contraseña. Se optó por las cuentas individuales, ya que la Junta puede crear, modificar y eliminar información, por lo que es necesario saber quién accede y poder desactivar a un miembro concreto sin afectar al resto. Para identificarlos se reutilizan los mismos tokens JWT ya utilizados con el cofrade en la Iteración 3, añadiendo un nuevo rol.

Control de lo que puede modificar cada Junta Cada Junta solo debe poder modificar la información de su propia ciudad (RNF-03). Se valoró hacer esta comprobación por separado en cada operación o

hacerla en un único sitio que reutilicen todas. Se optó por la segunda opción, ya que la comprobación es la misma para cofradías, pasos, eventos, procesiones y códigos de acceso, y repetirla en cada uno aumentaba el riesgo de olvidarla en alguno. En el caso de los recorridos, los puntos y las ubicaciones, que no pertenecen directamente a ninguna ciudad, se decidió deducirla a partir de la procesión o el evento que los utiliza.

Introducción del recorrido Para crear el recorrido de una procesión (RF-32) se valoró que la Junta lo dibujara punto a punto sobre el mapa dentro de la propia aplicación, o que lo importara desde un archivo GPX. Dibujarlo en el móvil es lento y poco preciso para un recorrido de varios kilómetros, mientras que el formato GPX es un estándar que generan herramientas gratuitas como gpx.studio, Wikiloc o Strava. Por ello se optó por importar el recorrido desde un archivo GPX, y sobre los puntos importados la Junta puede marcar después los puntos de interés (RF-33).

Cofradías y pasos de una procesión Una procesión o un evento puede tener varias cofradías, como ocurre en la Procesión General del Viernes Santo, y a la hora de asignar sus pasos se valoró ofrecer todos los pasos de la ciudad o solo los de las cofradías que participan. Se optó por la segunda opción, ya que un paso pertenece siempre a una única cofradía (RF-30), de forma que la lista es más corta y no se puede asignar por error un paso de una cofradía que no participa. Además, si se quita una cofradía de una procesión, sus pasos también se quitan.

En la Tabla **??** se resumen todas estas decisiones.

Problema	**Alternativas valoradas**	**Opción elegida**
Acceso de la Junta	Código compartido, cuentas individuales	Cuentas con correo y contraseña
Control de cada Junta	Comprobación en cada operación, comprobación común	Comprobación común reutilizada
Introducción del recorrido	Dibujarlo en la aplicación, importar un GPX	Importar un archivo GPX
Pasos de una procesión	Todos los de la ciudad, solo los de sus cofradías	Solo los de sus cofradías

Tabla 4.4: Problemas analizados en la cuarta iteración y opción elegida

4.4.2. Diseño

Una vez elegidas las alternativas en el análisis, en este apartado se explica cómo se ha diseñado el acceso y el panel de la Junta de Cofradías sobre la arquitectura definida en la Iteración 1 (Sección 4.1).

Autenticación de la Junta de Cofradías

Los miembros de la Junta de Cofradías inician sesión con su correo electrónico y su contraseña (`POST /auth/login`). La contraseña se guarda como *hash* mediante BCrypt, de forma que nunca se almacena

en texto plano (RNF-01). Si los datos son correctos, el servidor devuelve un token JWT con el rol de la Junta, igual que en el caso del cofrade, y la aplicación lo guarda para que el miembro no tenga que volver a iniciar sesión cada vez que la abre.

Control de acceso

En cada operación de creación, edición o eliminación, la capa `Service` comprueba que quien la solicita es un miembro activo de la Junta que gestiona la ciudad de esa información, utilizando para ello una única comprobación común. Si no es así, la API responde con un código `403` (prohibido). En el caso de los recorridos, los puntos de ruta y las ubicaciones, la ciudad se obtiene a partir de la procesión o el evento que los utiliza, y si todavía no están asignados a ninguno, como un recorrido que se acaba de importar, basta con que quien lo modifica sea una Junta activa. De esta forma se cubre el requisito RNF-03.

Importación del recorrido

La Junta selecciona el archivo GPX desde el móvil y la aplicación lo envía al servidor (`POST /recorridos/import`). En el servidor, la clase `GpxParser` lee el archivo y lo convierte en la lista ordenada de puntos que necesita `RecorridoService` para crear el `Recorrido`, aplicando el patrón Adapter descrito en la Iteración 1. Una vez creado, el recorrido se asigna a la procesión y la Junta puede marcar cualquiera de sus puntos como punto de interés.

Diseño de la base de datos

La tabla `usuarios` solo contiene a los usuarios con credenciales propias, es decir, a los miembros de la Junta de Cofradías y, más adelante, a los administradores (Iteración 5), con su correo y su contraseña guardada como *hash*[10] mediante BCrypt (RNF-01). Los datos propios de cada tipo de usuario se guardan en su propia tabla (`miembros_junta_cofradia` y `administradores`), siguiendo el criterio de herencia definido en la Iteración 1. Cada miembro hace referencia a la Junta a la que pertenece, y el ciudadano y el cofrade no generan ninguna fila en estas tablas (Sección **??**).

Interfaces de usuario

Según la planificación (Sección 2.2), esta cuarta iteración cubre el panel de la Junta de Cofradías, por lo que las pantallas que se diseñan en Figma [**?**] en esta iteración son el inicio de sesión y todo el panel de gestión (perfil, cofradías, procesiones, eventos y pasos), sobre los mismos criterios visuales fijados en la Iteración 1 (Sección 4.1).

[10]Hash: transformación irreversible de una contraseña en una cadena de caracteres, de forma que el sistema pueda comprobar que una contraseña es correcta sin necesidad de guardarla en texto plano.

Inicio de sesión A diferencia del ciudadano, que no necesita registrarse, tanto la Junta de Cofradías como el Administrador (Iteración 5) acceden con credenciales propias (*email* y contraseña). La pantalla de bienvenida distingue entre continuar como ciudadano o iniciar sesión, y esta última ofrece un selector entre los dos roles con cuenta, ya que ambos comparten la misma pantalla y el mismo *endpoint* de autenticación.



Figura 4.15: Mockup de la bienvenida y del inicio de sesión, para Junta de Cofradías y Administrador

Perfil de la Junta de Cofradías Muestra el rol y la ciudad que gestiona la Junta, y da acceso a la gestión de cofradías, procesiones, eventos y pasos, además de a la edición del propio perfil.

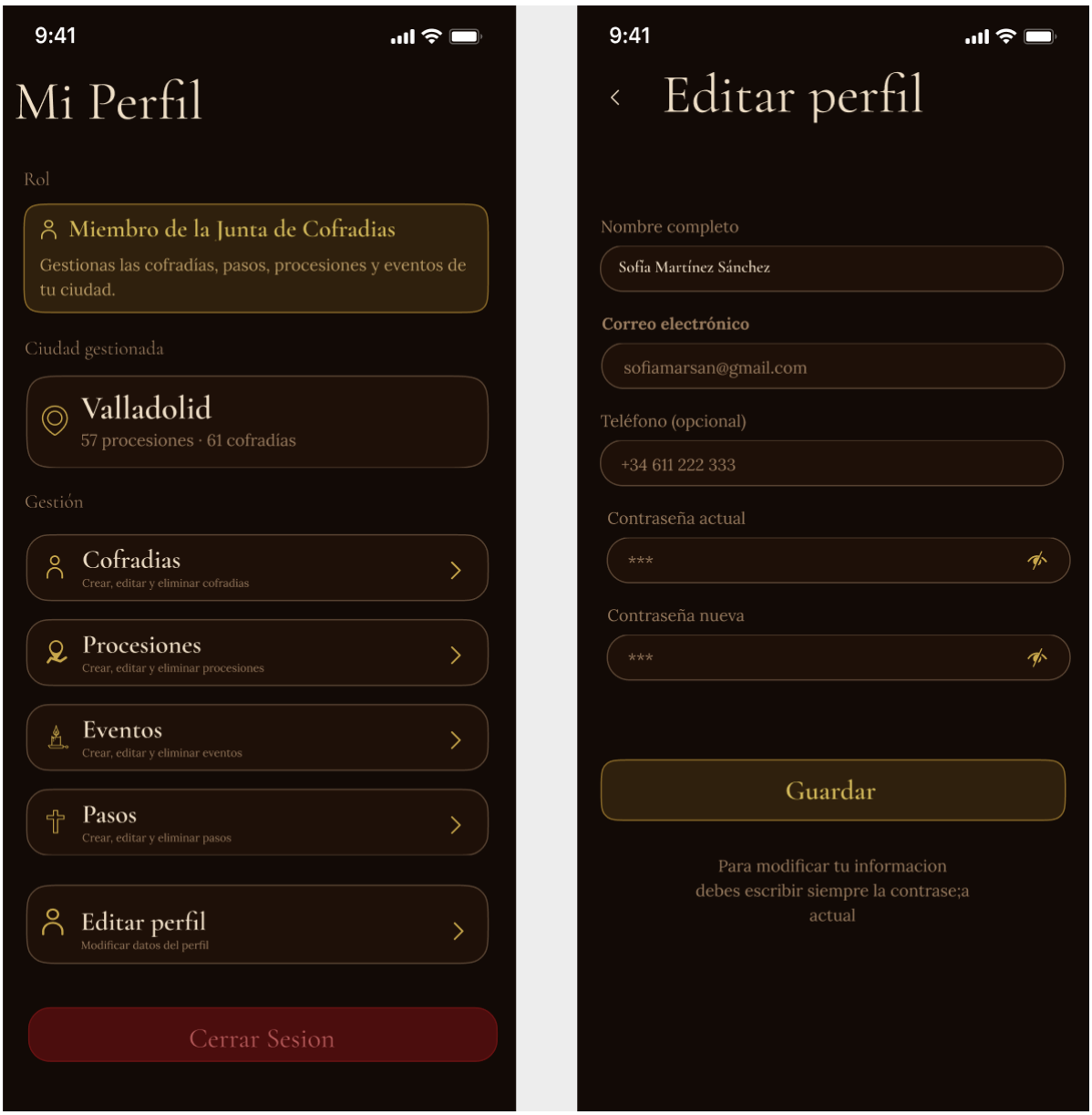


Figura 4.16: Mockup del perfil de la Junta de Cofradías y de la edición de sus datos

Cofradías Permite dar de alta una cofradía y, tras crearla, añadirle directamente procesiones, eventos o pasos, o dejarlo para más tarde. Al editar una cofradía existente se puede además consultar y gestionar esos mismos elementos ya asociados a ella.

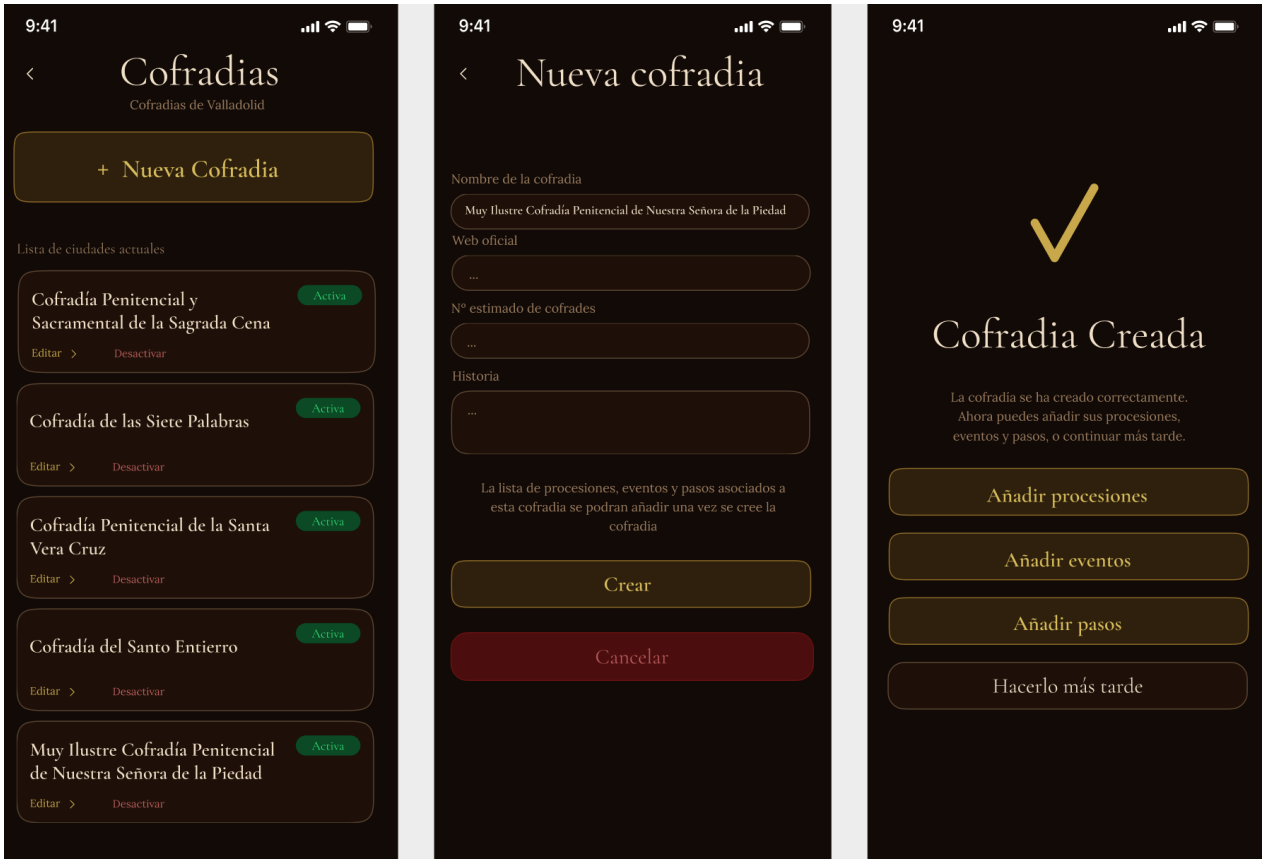


Figura 4.17: Mockup del listado de cofradías y de su creación

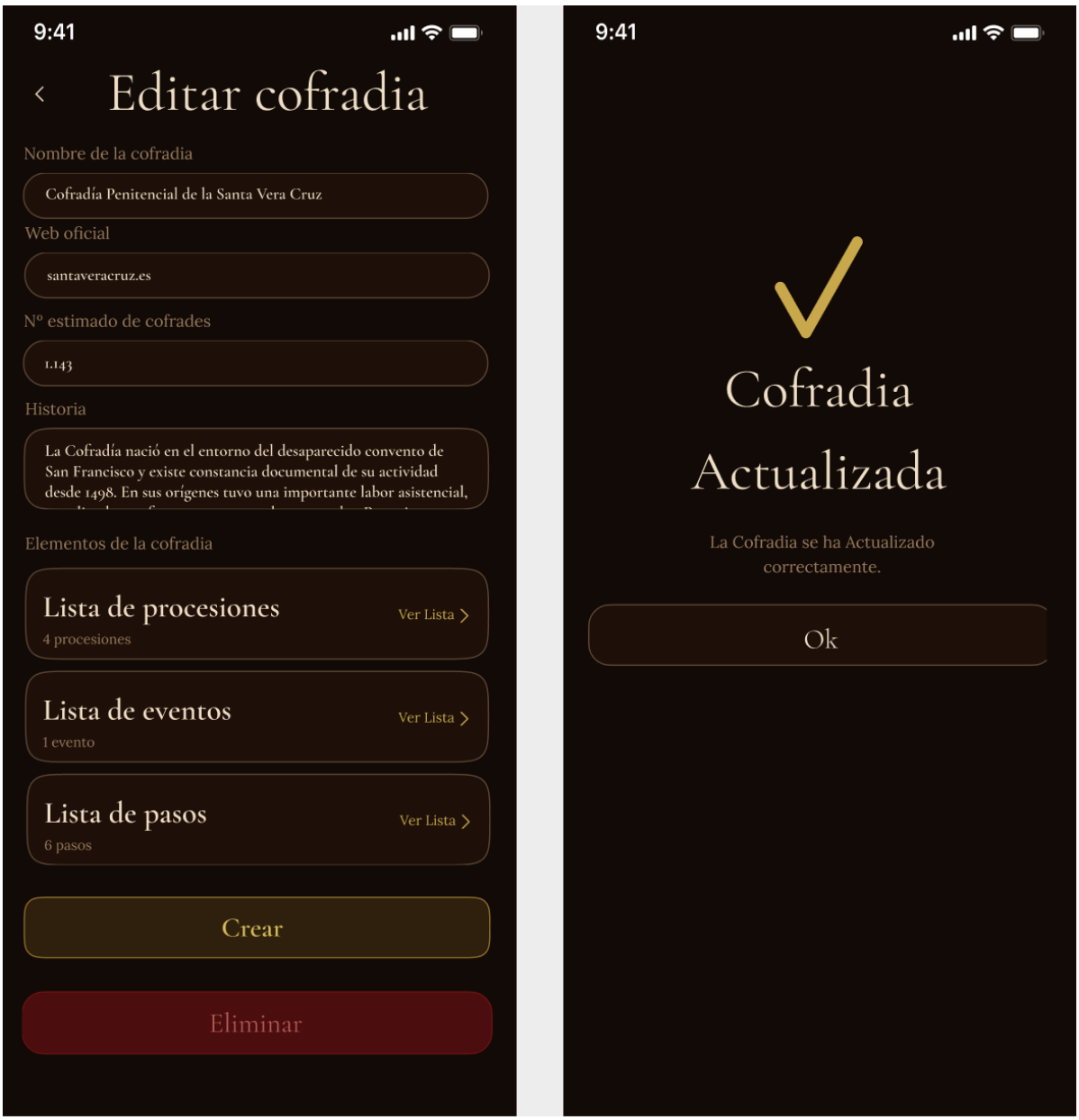


Figura 4.18: Mockup de la edición de una cofradía

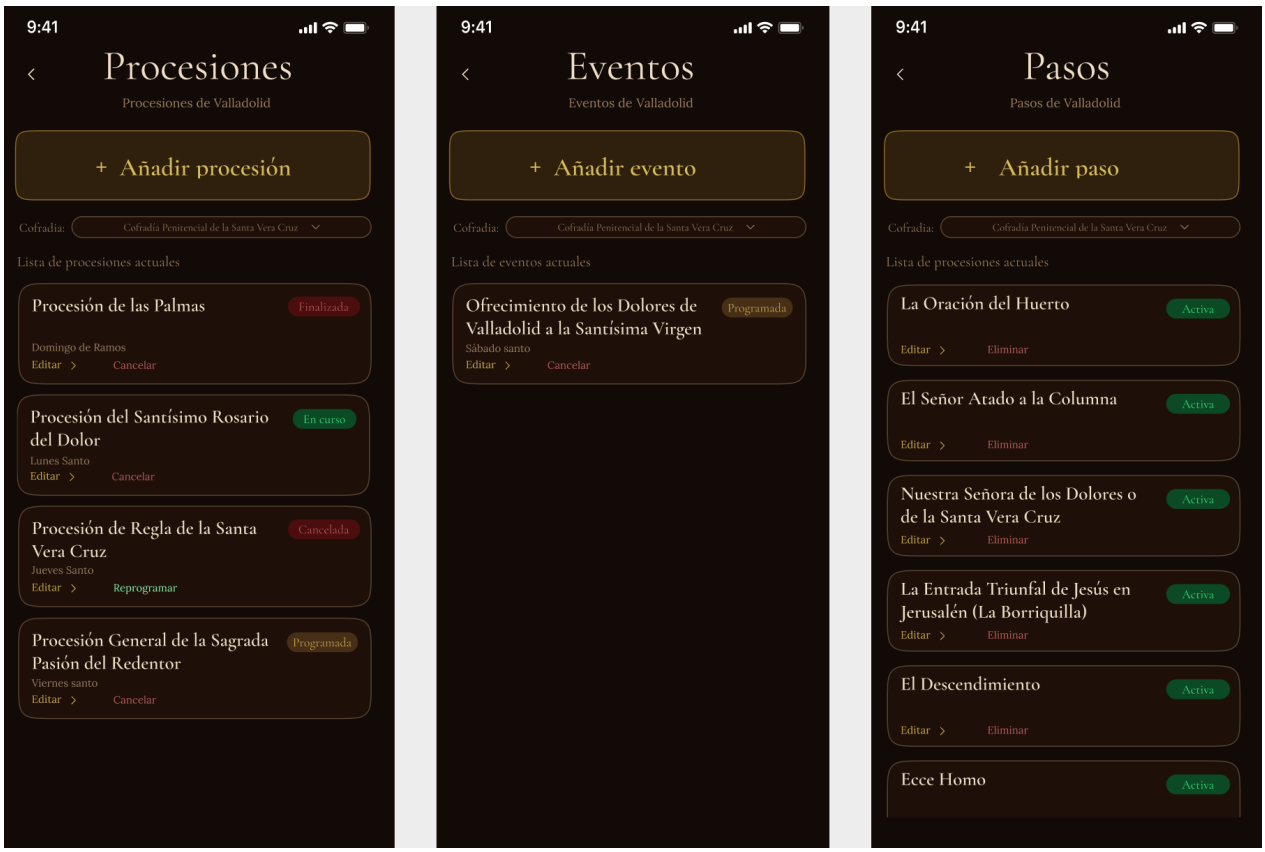


Figura 4.19: Mockup de las procesiones, eventos y pasos de una cofradía, accesibles desde su edición

Procesiones El listado de procesiones permite, además de editarlas, notificar sobre ellas (cancelación, cambio de horario o incidencia) o cancelarlas directamente. Al crear una procesión se importa su recorrido desde un archivo GPX, y tanto al crearla como al editarla se pueden asignar los pasos que participan en ella.

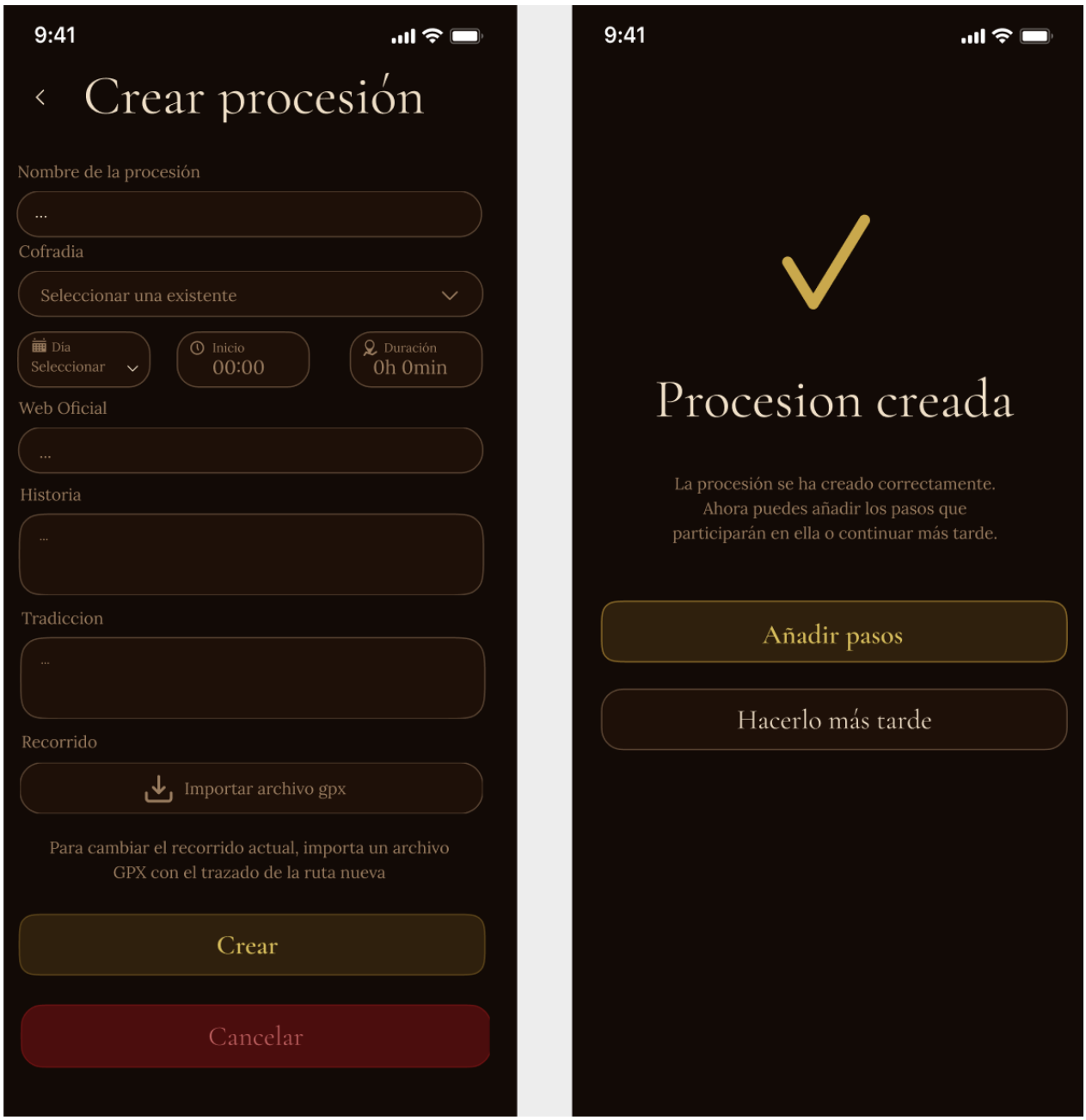


Figura 4.20: Mockup de la creación de una procesión

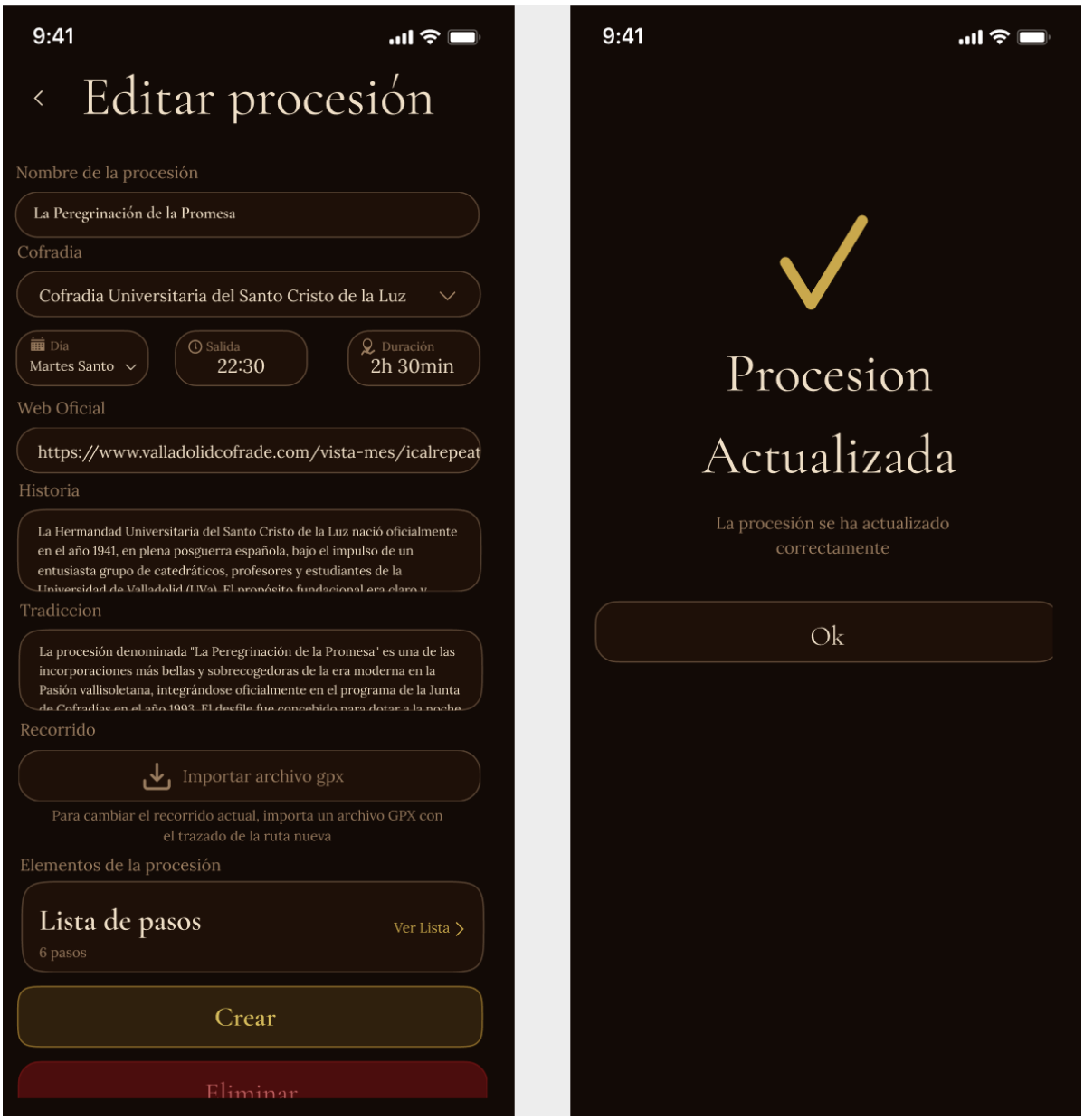


Figura 4.21: Mockup de la edición de una procesión

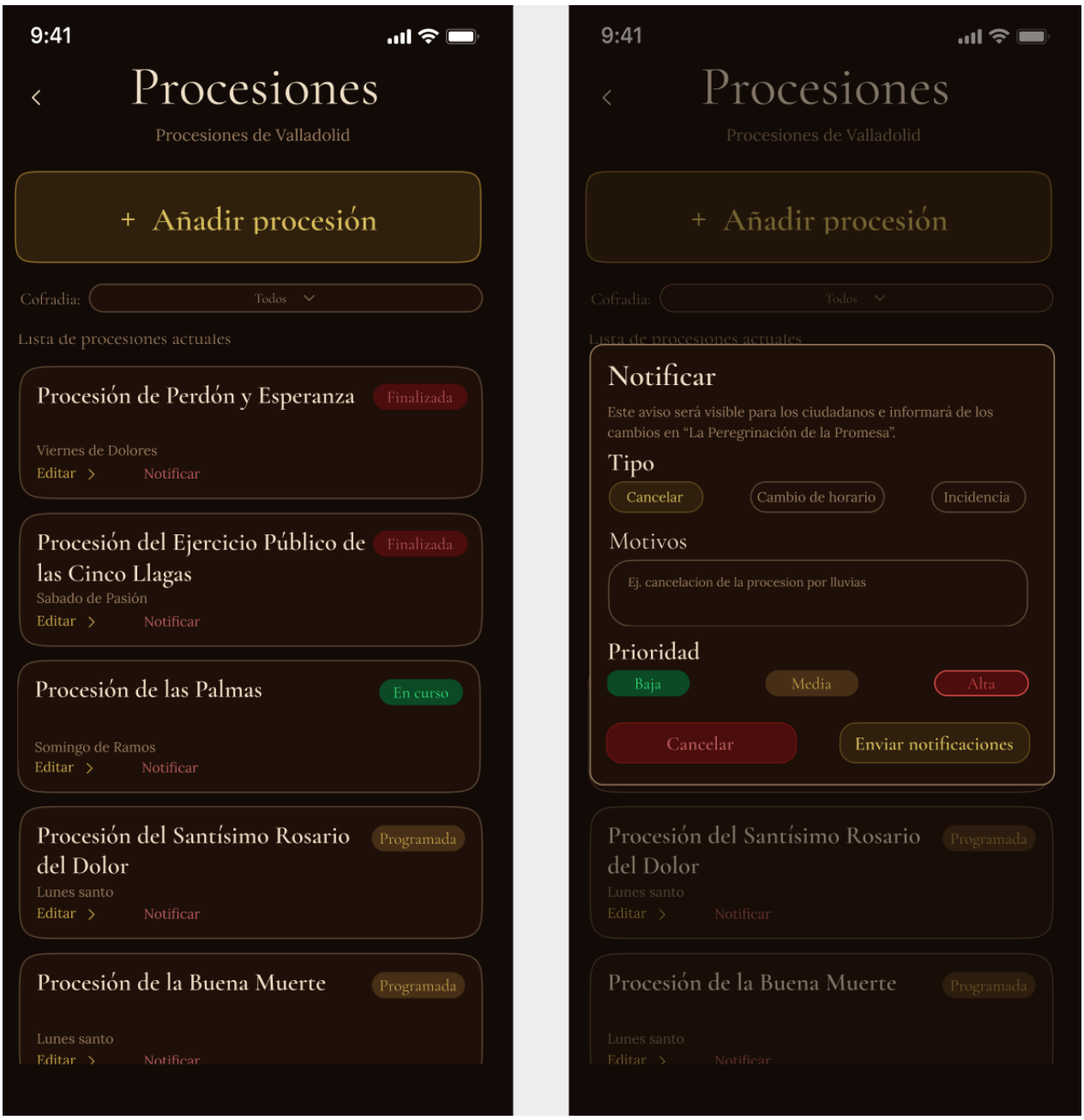


Figura 4.22: Mockup del listado de procesiones y del envío de una notificación

Eventos Análogo al de procesiones, pero sin recorrido: al crear un evento se pueden asignar directamente los pasos que participan en él, o hacerlo más tarde desde su edición.

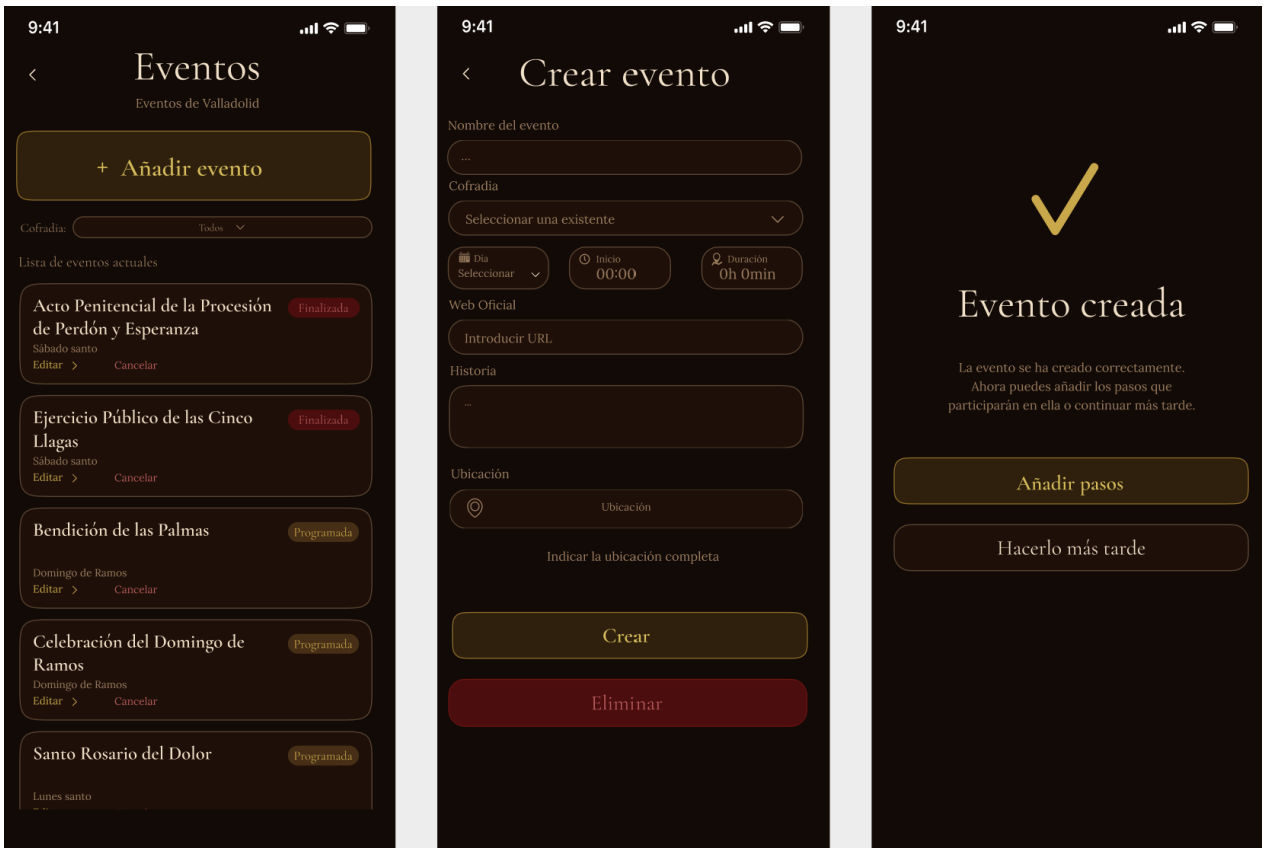


Figura 4.23: Mockup del listado de eventos y de su creación



Figura 4.24: Mockup de la edición de un evento

Pasos Cada paso pertenece a una única cofradía, y su ficha incluye su historia, un análisis artístico y una imagen, cuya URL se pega a mano en el formulario (Sección **??**).

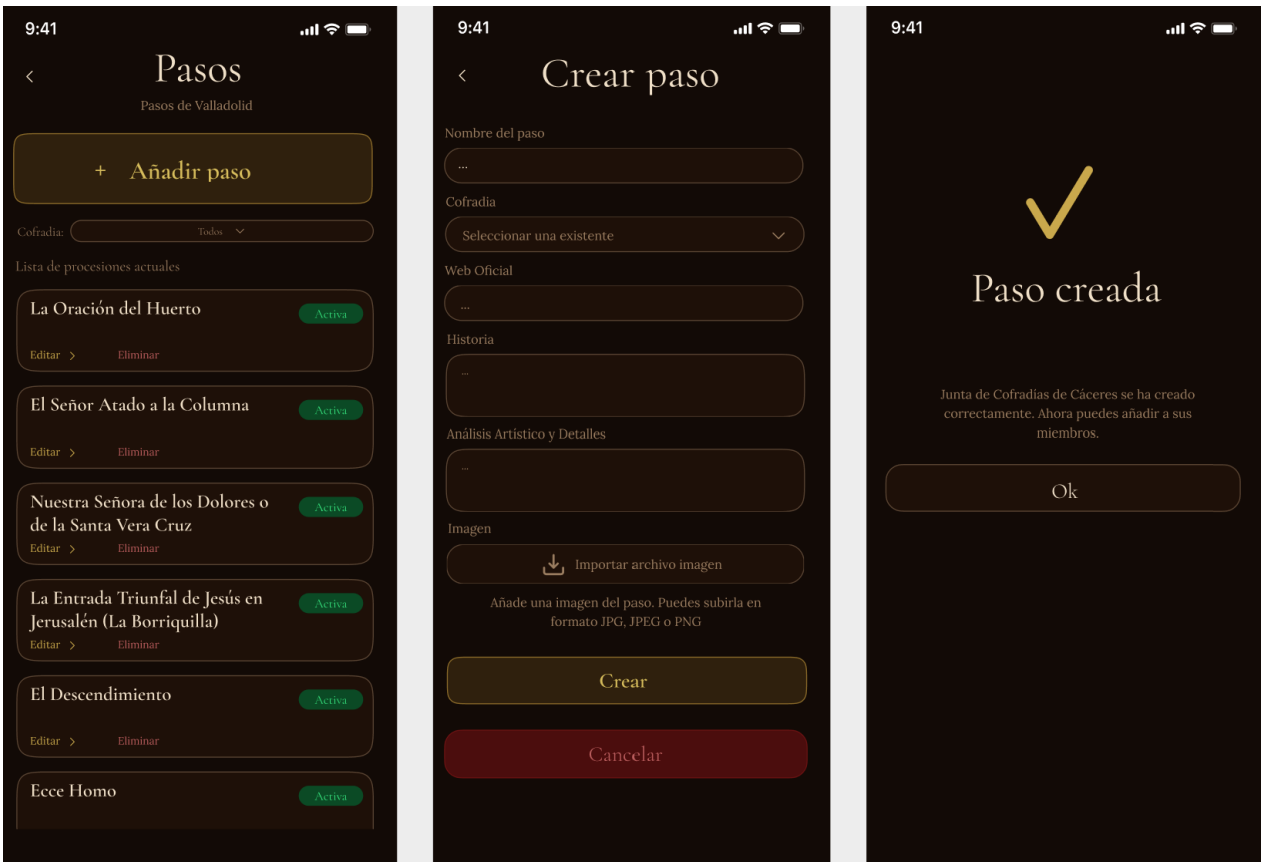


Figura 4.25: Mockup del listado de pasos y de su creación



Figura 4.26: Mockup de la edición de un paso

4.4.3. Implementación

Esta cuarta iteración implementa el panel de la Junta de Cofradías, es decir, el acceso con credenciales propias y el control de lo que puede gestionar cada Junta.

Se implementa `MiembroJuntaCofradia`, que representa a una persona con credenciales que pertenece a la Junta de una ciudad, pudiendo haber varios miembros en cada Junta, y el *endpoint* `POST /auth/login`, que comprueba el correo y la contraseña recibidos y devuelve un token JWT con el identificador del miembro y su rol, siguiendo el mismo mecanismo de autenticación de la Iteración 3.

Sobre esta autenticación se implementa la autorización por rol que exige el requisito RNF-03. En la capa `Service` de `Cofradia`, `Paso`, `Evento`, `Procesion` y `CodigoAcceso`, las operaciones de creación, edición y eliminación comprueban que quien las solicita es un miembro de la Junta que gestiona la ciudad de esa cofradía, respondiendo con un código `403` en caso contrario. Para no repetir la misma lógica en cada entidad, esta comprobación se centraliza en un único método. En el caso de `Recorrido`, `Ubicacion`, `PuntoRuta` y `PuntoDeInteres`, que no pertenecen directamente a ninguna cofradía, se implementa `AutorizacionRecorridoService`, que obtiene la ciudad a partir de la procesión o el evento que los utiliza.

En la aplicación se implementan todas las pantallas del panel: el perfil de la Junta, la edición de la información de la ciudad y la gestión de cofradías, procesiones, eventos, pasos y códigos de acceso. Al crear una procesión, el recorrido se importa desde un archivo GPX seleccionado con la librería `expo-document-picker`, y tanto en las procesiones como en los eventos se pueden elegir varias cofradías y asignar los pasos de todas ellas. Además, desde el listado de procesiones y eventos la Junta puede cambiar su estado a mano, por ejemplo para cancelar uno de ellos.

4.4.4. Pruebas

Al igual que en las iteraciones anteriores, la verificación se ha realizado de forma manual, obteniendo un token con `POST /auth/login` y comprobando que las operaciones de gestión se aceptan o se rechazan (`403`) según la ciudad que gestiona cada Junta de prueba. Las pruebas automatizadas se recogen en el Capítulo **??**.

4.5. 5ª Iteración

4.5.1. Análisis

Según la planificación (Sección 2.2), esta quinta y última iteración se centra en el panel de administrador, desde el que se gestionan las ciudades, las Juntas de Cofradías y sus miembros (RF-37 a RF-41), y en el sistema de notificaciones (RF-21, RF-22, RF-35 y RF-36). En el caso del administrador se reutiliza el mismo acceso con correo y contraseña de la Junta de Cofradías (Iteración 4), por lo que los problemas

nuevos que había que resolver estaban relacionados con las notificaciones y con el alta de los miembros de la Junta.

Problemas planteados y alternativas valoradas

Notificaciones *push* Se valoró utilizar Firebase Cloud Messaging [**?**], que era la opción prevista inicialmente, o Expo Push Notifications. Finalmente se eligió Expo Push, ya que la aplicación está desarrollada con Expo y de esta forma no es necesario añadir otro proveedor con su propio proyecto y sus credenciales.

A quién se envía cada notificación Como el ciudadano no tiene cuenta, no se puede enviar una notificación a una persona concreta. Se valoró enviar todas las notificaciones a todos los dispositivos o asociar cada dispositivo a una ciudad. Se optó por la segunda opción, de forma que cada dispositivo se registra con la ciudad que tiene seleccionada y solo recibe las notificaciones de esa ciudad (RF-21). Para dejar de recibirlas, en lugar de añadir un interruptor propio dentro de la aplicación, se decidió utilizar el propio permiso de notificaciones del móvil (RF-22), ya que hacer las dos cosas sería redundante.

Cambio de estado de los eventos Las notificaciones de inicio y de fin dependen de que los eventos y las procesiones cambien de estado a su hora. Se valoró que fuera la Junta quien cambiara siempre el estado a mano o que el sistema lo hiciera de forma automática. Se optó por el cambio automático (RF-34), ya que el día de la procesión la Junta tiene muchas otras cosas de las que ocuparse y es fácil que se le olvide. Aun así, se mantuvo la posibilidad de corregir el estado a mano, por ejemplo para cancelar una procesión por la lluvia.

Alta de los miembros de la Junta Para dar de alta a un miembro se valoró que el administrador escribiera su contraseña y se la comunicara, que el propio miembro se registrara, o que el sistema generara una contraseña provisional y se la enviara por correo. Se optó por la última opción, ya que así el administrador nunca conoce la contraseña del miembro y no cualquiera puede registrarse como Junta. Además, el miembro está obligado a cambiar esa contraseña la primera vez que inicia sesión (RF-26 y RF-39).

En la Tabla **??** se resumen todas estas decisiones.

Problema	**Alternativas valoradas**	**Opción elegida**
Notificaciones *push*	Firebase Cloud Messaging, Expo Push	Expo Push
Destinatarios	Todos los dispositivos, dispositivos por ciudad; interruptor propio o permiso del móvil	Dispositivos por ciudad; permiso del móvil
Cambio de estado	A mano, automático	Automático, con corrección a mano
Alta de miembros	Contraseña del administrador, registro propio, contraseña provisional por correo	Contraseña provisional por correo

Tabla 4.5: Problemas analizados en la quinta iteración y opción elegida

4.5.2. Diseño

Una vez elegidas las alternativas en el análisis, en este apartado se explica cómo se ha diseñado el sistema de notificaciones y el panel de administrador sobre la arquitectura definida en la Iteración 1 (Sección 4.1).

Notificaciones

Cuando el ciudadano selecciona una ciudad, la aplicación obtiene un token de Expo Push mediante `expo-notifications` y lo registra en la API junto con esa ciudad (`POST /dispositivos-push`). Después, cada vez que se crea una notificación, ya sea de forma automática al empezar o terminar un evento o porque la escribe la Junta de Cofradías, la API la guarda y la envía al servicio de Expo Push (`exp.host`) para todos los dispositivos registrados en esa ciudad. De esta forma, el cliente nunca se comunica directamente con el servicio de notificaciones, y aunque el envío falle, la notificación se puede seguir consultando dentro de la aplicación.

Cambio automático de estado

En el servidor se ejecuta una tarea programada cada 30 segundos que revisa los eventos y procesiones. Los que están programados pasan a en curso al llegar su hora de inicio, y los que están en curso pasan a finalizado al llegar su hora de fin, generando en cada caso la notificación correspondiente (RF-35). Los eventos cancelados o finalizados no se modifican nunca. Cuando la Junta cambia el estado a mano, el sistema no le permite hacer un cambio que contradiga el horario, como volver a programado una vez pasada la hora de inicio, y le indica que modifique la hora en su lugar (RF-34).

Alta de miembros e invitación

Al dar de alta a un miembro, el servidor genera una contraseña provisional aleatoria, la guarda como *hash* y se la envía por correo electrónico junto con un mensaje de bienvenida, utilizando un servidor SMTP. Mientras el miembro no la cambie, el servidor rechaza cualquier modificación que intente hacer y la aplicación le lleva directamente a la pantalla de cambio de contraseña. Si la invitación se pierde, el administrador puede reenviarla, lo que genera una contraseña provisional nueva e invalida la anterior (RF-39).

Diseño de la base de datos

- Las tablas `notificaciones` y `dispositivos_push` hacen referencia a su ciudad, de forma que cada notificación se envía a los dispositivos registrados en la ciudad a la que pertenece. No se guarda qué ciudadano ha leído cada notificación, ya que el ciudadano no tiene cuenta.

- La tabla `administradores` extiende a `usuarios` y no hace referencia a ninguna ciudad, ya que cualquier administrador puede gestionar cualquier ciudad.

- En `miembros_junta_cofradia` se guarda si el miembro sigue con la contraseña provisional, si está activo y si ha solicitado su reactivación (RF-41).

Interfaces de usuario

Según la planificación (Sección 2.2), esta quinta iteración cubre el panel de administrador, por lo que las pantallas que se diseñan en Figma [**?**] en esta iteración son el perfil de administrador y la gestión de ciudades, Juntas de Cofradías y sus miembros, reutilizando la misma pantalla de inicio de sesión ya diseñada para la Junta de Cofradías en la Iteración 4 (Sección 4.4).

Perfil de administrador Muestra un resumen del número de ciudades y de Juntas de Cofradías del sistema, y da acceso a la gestión de ambas, a la de los miembros de las distintas Juntas, y a la edición del propio perfil.

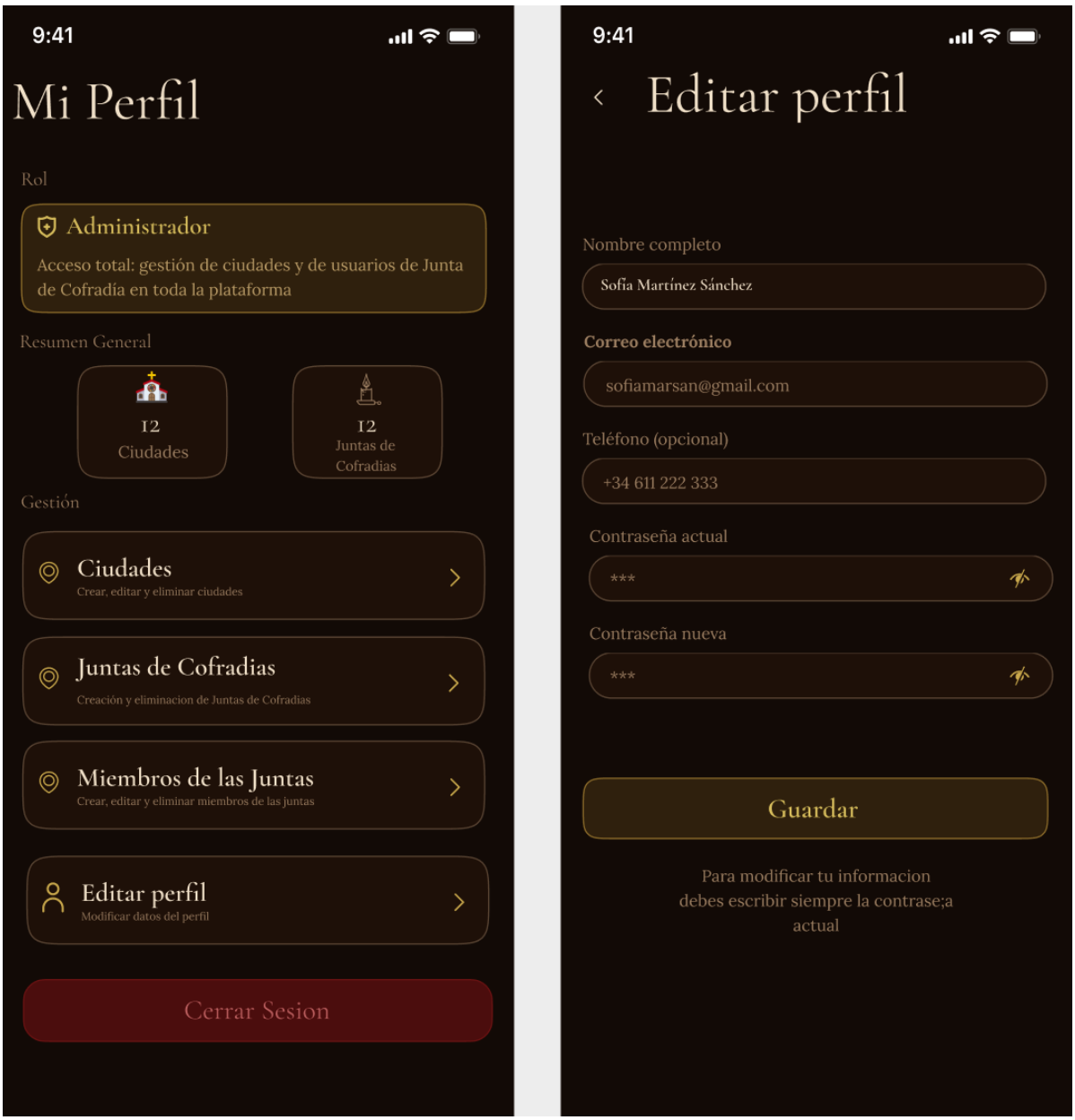


Figura 4.27: Mockup del perfil de administrador y de la edición de sus datos

Ciudades Permite dar de alta una ciudad y, sobre las ya existentes, activarlas o desactivarlas y asignarles una Junta de Cofradías.

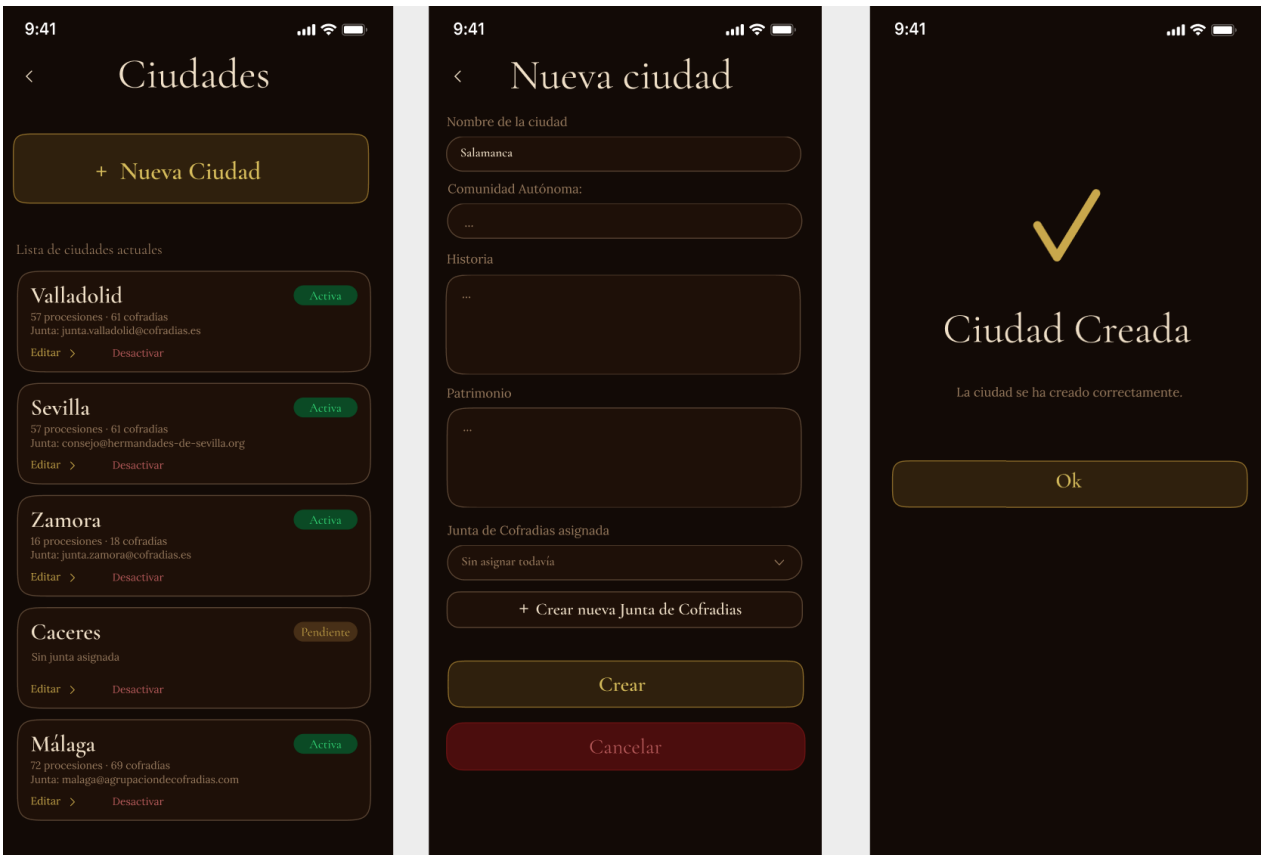


Figura 4.28: Mockup del listado de ciudades y de su creación

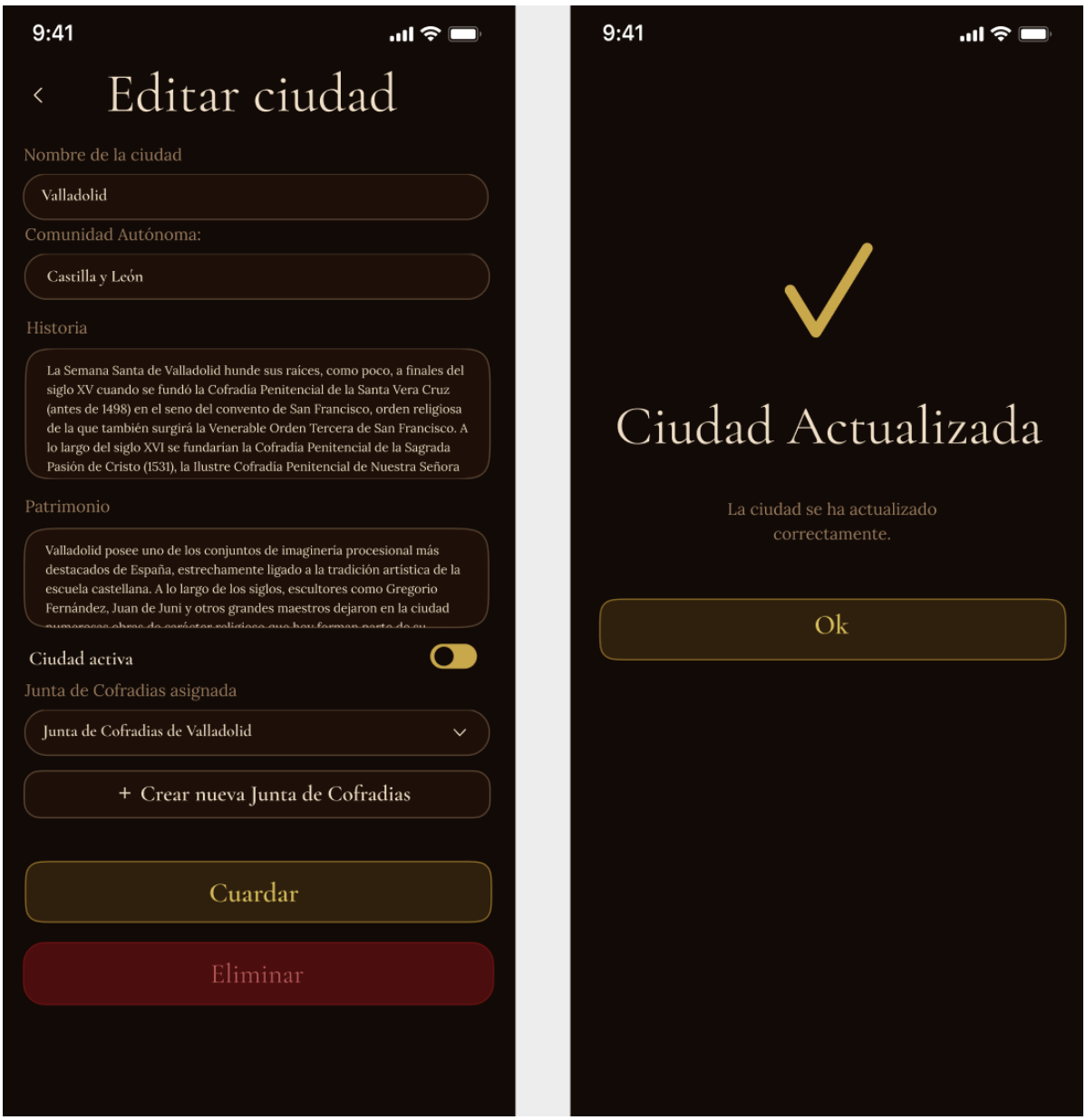


Figura 4.29: Mockup de la edición de una ciudad

Juntas de Cofradías Análogo a la gestión de ciudades: al crear una Junta se le asigna la ciudad que gestionará, ofreciendo solo las ciudades que todavía no tienen una, y tras crearla se puede pasar directamente a añadirle miembros.

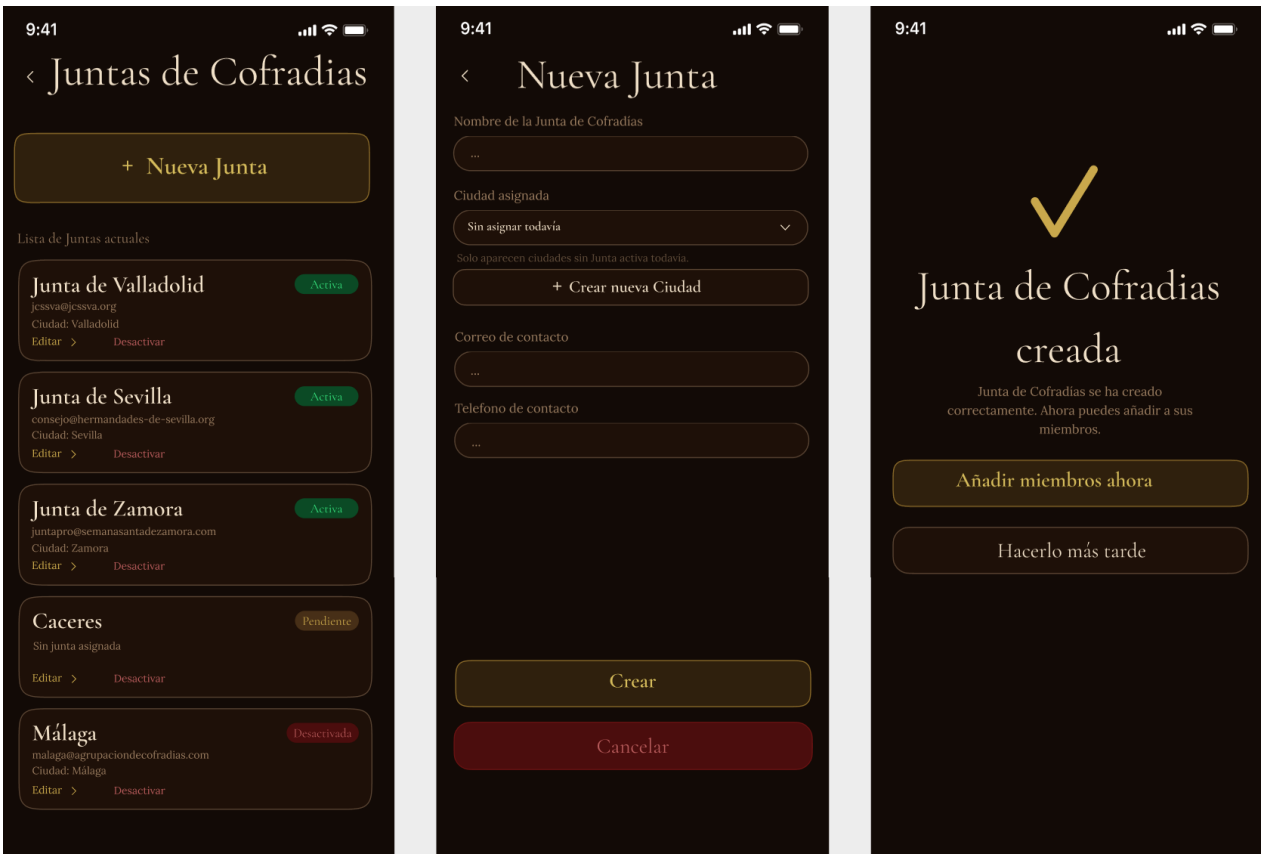


Figura 4.30: Mockup del listado de Juntas de Cofradías y de su creación



Figura 4.31: Mockup de la edición de una Junta de Cofradías

Miembros de la Junta de Cofradías Al dar de alta un miembro se le envía por correo una contraseña provisional generada automáticamente, que debe cambiar en su primer inicio de sesión; mientras no lo haga, el listado lo marca como *pendiente* y permite reenviarle la invitación. Un miembro también puede desactivarse o reactivarse sin perder sus datos.

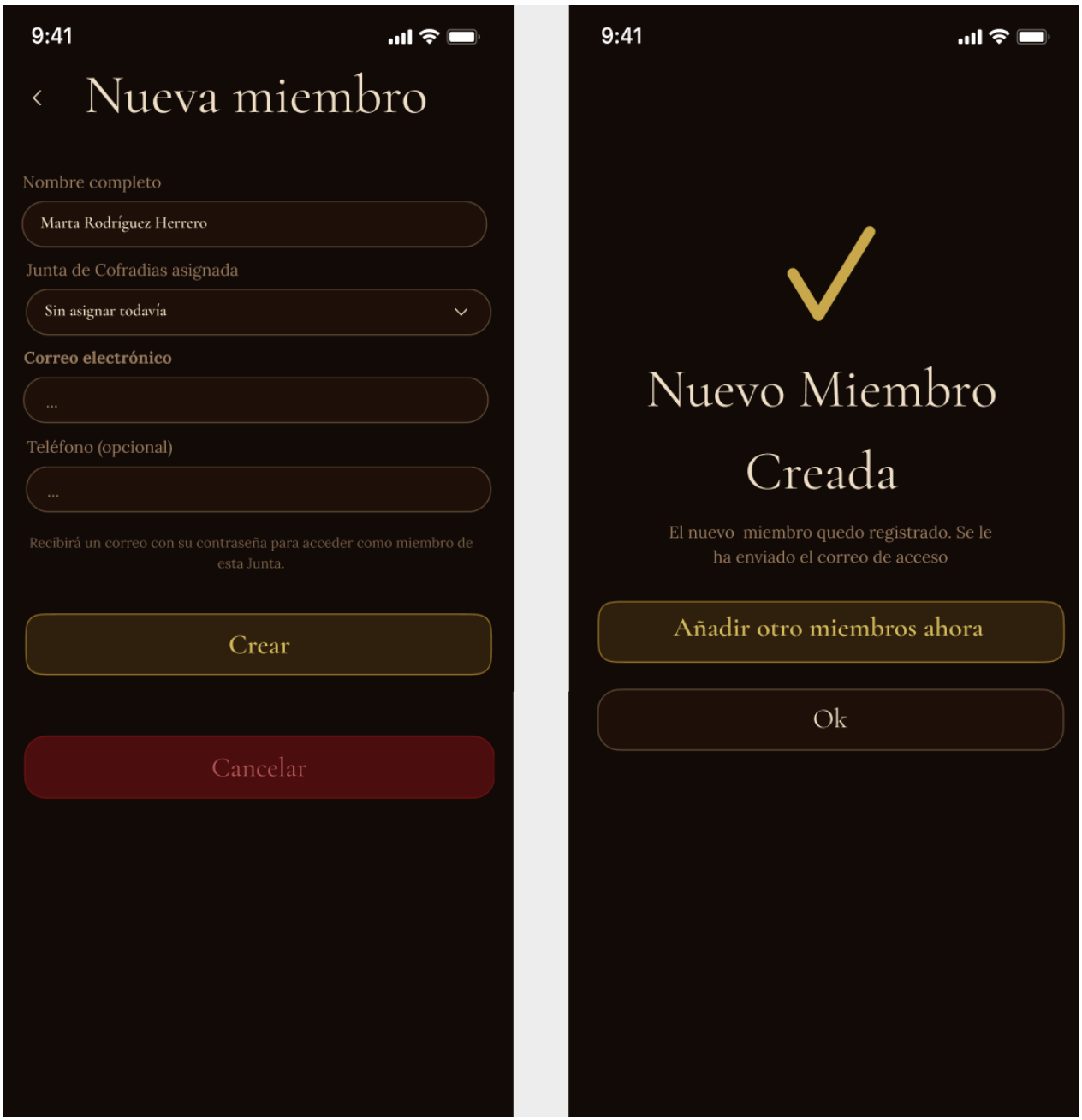


Figura 4.32: Mockup de la creación de un miembro de la Junta de Cofradías

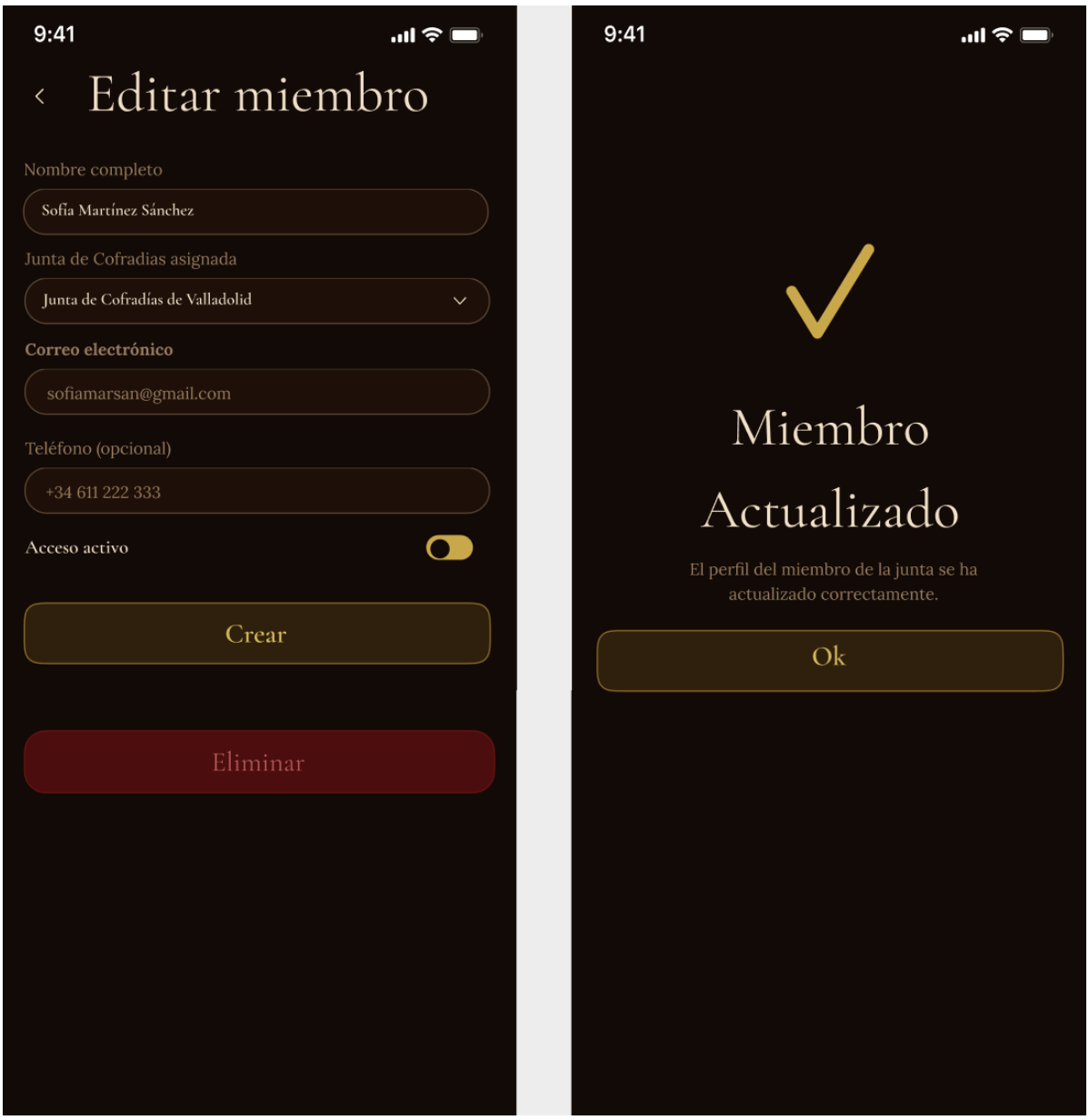


Figura 4.33: Mockup de la edición de un miembro de la Junta de Cofradías

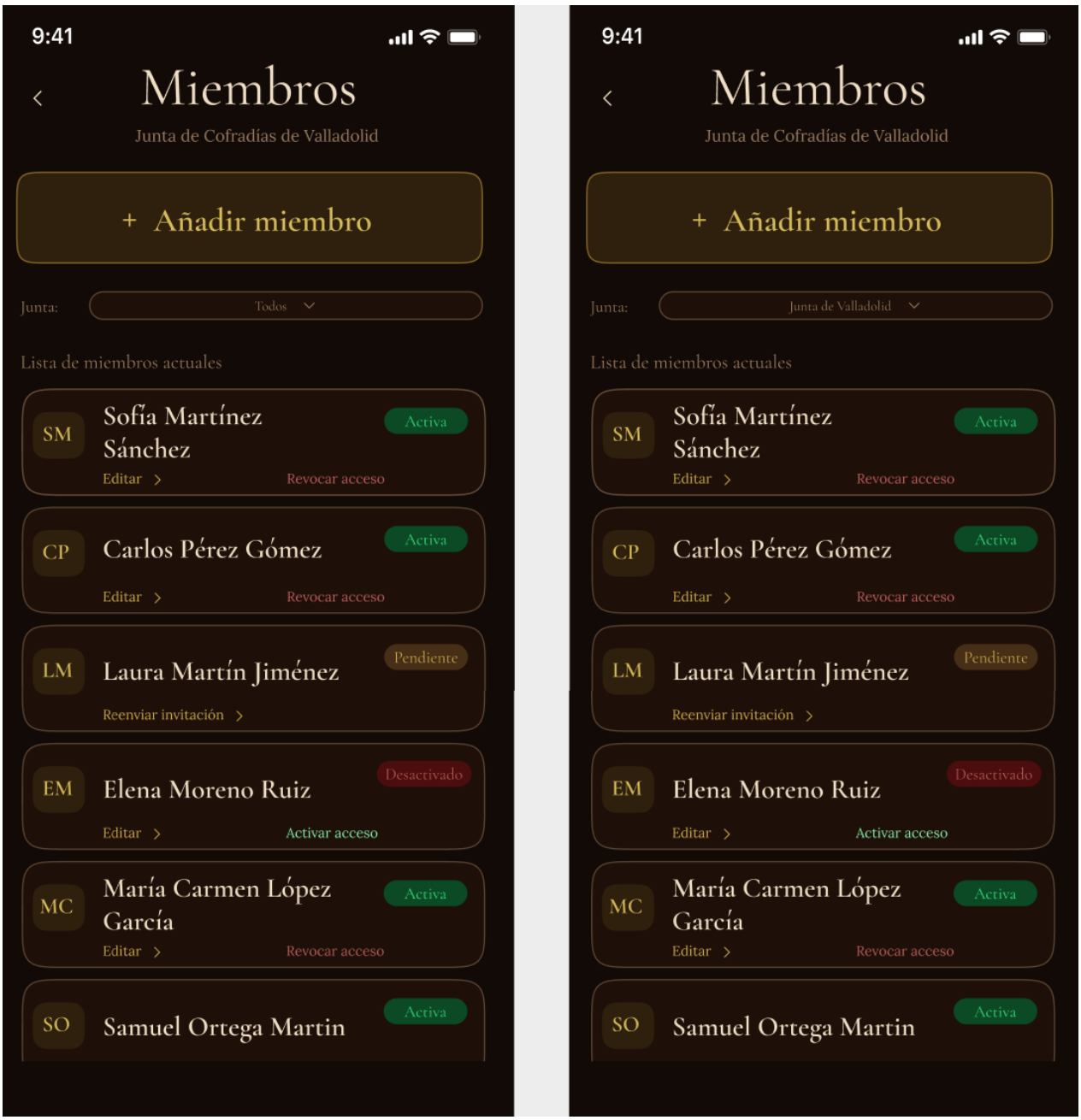


Figura 4.34: Mockup del listado de miembros, sin filtrar y filtrado por Junta de Cofradías

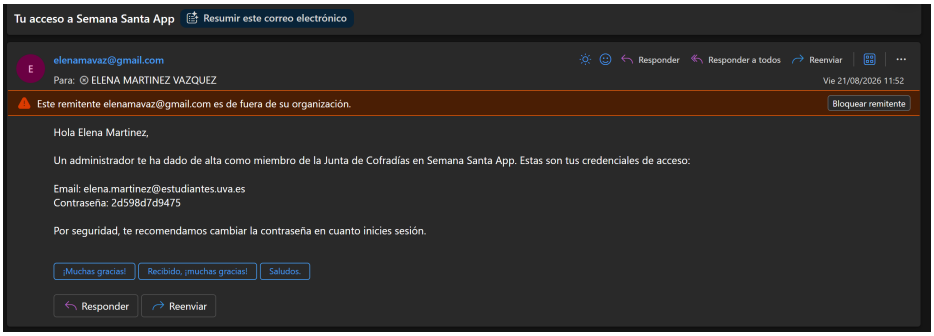


Figura 4.35: Correo de invitación recibido por un nuevo miembro de la Junta de Cofradías

4.5.3. Implementación

Esta quinta y última iteración implementa el panel de administrador y el sistema de notificaciones.

Se implementa `Administrador`, que es un rol global del sistema sin ninguna referencia a `Ciudad`, ya que cualquier administrador puede gestionar cualquier ciudad, y su acceso mediante el mismo *endpoint* `POST /auth/login` de la Junta de Cofradías, que distingue el rol según el tipo de cuenta. Sobre esta autenticación se completa el control de acceso por rol: la creación, edición y eliminación de `Ciudad`, `JuntaCofradias` y `MiembroJuntaCofradia` quedan restringidas al administrador. En este caso no hace falta comparar ninguna ciudad, a diferencia de la comprobación de la Junta de Cofradías de la Iteración 4. Para el alta de los miembros se implementa `CorreoService`, que envía el correo de bienvenida con la contraseña provisional, junto con el reenvío de la invitación y la solicitud de reactivación de los miembros desactivados.

En cuanto a las notificaciones, se implementan `Notificacion` y `DispositivoPush`, el registro de los dispositivos (`POST /dispositivos-push`) y `PushNotificacionService`, que envía cada notificación al servicio de Expo Push. Además, se implementa `CambioEstadoAutomaticoService`, la tarea programada que cambia el estado de los eventos y las procesiones a su hora y genera las notificaciones de inicio y de fin. En la aplicación, el ciudadano puede consultar las notificaciones vigentes de su ciudad, y en su perfil ve si tiene activado el permiso de notificaciones, con un acceso directo a los ajustes del móvil para cambiarlo. Por su parte, la Junta puede crear una notificación después de modificar una procesión o un evento, y al cancelarlo se abre directamente la creación de la notificación de cancelación para que indique el motivo (RF-35 y RF-36).

4.5.4. Pruebas

Al igual que en las iteraciones anteriores, la verificación se ha realizado de forma manual, obteniendo un token de administrador con `POST /auth/login` y comprobando que las operaciones sobre `Ciudad`, `JuntaCofradias` y `MiembroJuntaCofradia` solo se aceptan con ese rol. Las notificaciones se han probado enviándolas a un móvil real. Las pruebas automatizadas se recogen en el Capítulo **??**.

Capítulo 5

Pruebas

Este capítulo describe cómo se ha comprobado que el sistema funciona y que cumple los requisitos del Capítulo 3. Las pruebas se han organizado en cuatro niveles, de menor a mayor alcance:

1. **Pruebas automáticas del backend**, que comprueban la lógica de negocio y la seguridad de la API clase a clase.
2. **Pruebas de simulación del seguimiento en tiempo real**, con un simulador propio que imita a varios cofrades compartiendo su ubicación.
3. **Pruebas en directo** con el sistema desplegado en el servidor, el simulador enviando las ubicaciones de los cofrades y la aplicación en un dispositivo real.
4. **Pruebas manuales de aceptación** de cada caso de uso sobre la aplicación.

Por último, se resumen los errores detectados durante las pruebas y su corrección.

5.1. Pruebas automáticas del backend

El backend cuenta con un conjunto de pruebas automáticas escritas con JUnit 5 [**?**] y Mockito [**?**], que se ejecutan con el comando `./mvnw test`. Se distinguen dos tipos:

- **Pruebas unitarias de la capa de servicio y de las utilidades**. Cada prueba comprueba una sola clase, sustituyendo sus dependencias (repositorios, otros servicios, el envío de correo o de notificaciones *push*) por objetos simulados con Mockito. Así se verifica la lógica de negocio sin necesidad de una base de datos: las reglas de autorización por rol y por ciudad, los cambios de estado, la caducidad de los códigos de acceso, el cálculo de la estela, etc.
- **Pruebas de la API** (`SeguridadApiTest`), con Spring Boot Test y MockMvc. Envían peticiones HTTP a los controladores, sin arrancar un servidor real, para comprobar qué rutas son públicas y cuáles exigen sesión, cómo se traducen los errores a códigos HTTP y que se validan los datos de entrada.

La Tabla **??** recoge las clases de prueba, el número de pruebas de cada una y lo que comprueban. En total son **217 pruebas**, y todas se superan.

Clase de prueba	Pruebas	Qué comprueba
`SeguridadApiTest`	28	Rutas públicas y protegidas, tokens falsos o ausentes, datos personales no públicos, traducción de errores a códigos HTTP (404, 403, 401, 409, 410) y validación de los datos de entrada.
`PosicionActualServiceTest`	20	Envío de ubicaciones por los cofrades: descarte de las que están a más de 50 m del recorrido, rechazo de cofrades de otra cofradía y de códigos revocados o caducados, posición actual y estela de la procesión, que nunca retrocede.
`MiembroJuntaCofradiaServiceTest`	24	Autorización de la Junta (miembro activo, Junta activa, contraseña provisional ya cambiada, misma ciudad), alta con contraseña provisional y correo de bienvenida, reenvío de la invitación y solicitudes de reactivación.
`AdministradorServiceTest`	18	Alta del primer administrador con el secreto de despliegue, gestión de administradores y de su perfil, correos duplicados.
`AuthServiceTest`	17	Inicio de sesión con correo y contraseña, inicio de sesión del cofrade con código (emitido, validado, revocado, caducado, de cofradía desactivada) y cambio de contraseña.
`ProcesionServiceTest`	14	Alta y edición de procesiones, recorrido único por procesión, autorización por ciudad, cambios de estado con su notificación y cancelación.
`GeometriaRutaTest`	13	Proyección de una ubicación sobre el recorrido para calcular cuánto ha avanzado la procesión, incluidos los recorridos que salen y vuelven al mismo templo.
`CofradiaServiceTest`, `CiudadServiceTest`, `JuntaCofradiasServiceTest`, `PasoServiceTest`, `EventoServiceTest`, `InformacionCiudadServiceTest`	32	Gestión del contenido por la Junta y el administrador: autorización por rol y por ciudad, elementos activos e inactivos, una Junta por ciudad, borrado con datos asociados y que el listado de eventos no incluya las procesiones.
`NotificacionServiceTest`	10	Notificaciones manuales (prioridad obligatoria, solo la Junta de la ciudad) y automáticas de inicio y fin.
`HorarioEventosTest`, `CambioEstadoAutomaticoServiceTest`	15	Cambio automático de estado al llegar la hora de inicio y de fin, y cambios manuales que contradicen el horario.
`GpxParserTest`, `RecorridoServiceTest`, `RecorridoPuntoRutaServiceTest`, `AutorizacionRecorridoServiceTest`	32	Lectura de archivos GPX (orden de prioridad, máximo de 300 puntos, archivos no válidos y protección frente a ataques XXE), importación y borrado de recorridos y marcado de puntos de interés.
`JwtServiceTest`, `CodigoAccesoServiceTest`, `DispositivoPushServiceTest`, `ValidacionImagenTest`	22	Tokens manipulados, firmados con otra clave o caducados, generación de códigos de acceso únicos, registro de dispositivos para *push* y validación de las direcciones de imagen.
Total	**217**	

Tabla 5.1: Pruebas automáticas del backend

5.1.1. Cobertura

La cobertura de las pruebas se ha medido con JaCoCo [**?**], que genera un informe en `target/site/jacoco` al ejecutar las pruebas. La Tabla **??** muestra el resultado por paquete.

Paquete	Instrucciones	Ramas
`util` (GPX y geometría del recorrido)	99 %	95 %
`security` (JWT y filtro de autenticación)	98 %	75 %
`model`	81 %	50 %
`exception`	79 %	83 %
`service` (lógica de negocio)	77 %	78 %
`controller`	14 %	14 %
Total	**73 %**	**77 %**

Tabla 5.2: Cobertura de las pruebas automáticas del backend

El esfuerzo se ha concentrado en la lógica de negocio y en la seguridad, que es donde un error tiene más consecuencias: la geometría del recorrido y el análisis de los GPX están cubiertos casi por completo, y la capa de servicio en más de tres cuartas partes. La baja cobertura de los controladores se debe a que estos se limitan a recibir la petición, llamar al servicio y devolver la respuesta, sin lógica propia; sus rutas de seguridad y de errores se comprueban en `SeguridadApiTest`, pero no se ha escrito una prueba HTTP para cada *endpoint*.

5.2. Pruebas de simulación del seguimiento en tiempo real

El seguimiento de una procesión en tiempo real (RF-18, RF-24 y RF-25) no se puede probar a fondo con pruebas automáticas, porque depende de que varias personas compartan su ubicación a la vez mientras recorren las calles. Para probarlo sin salir a la calle con varios móviles se ha desarrollado un **simulador de procesiones**, un programa en Node.js incluido en el repositorio (`herramientas/simulador-procesion`).

El simulador hace lo mismo que la aplicación del cofrade: inicia sesión con un código de acceso y envía ubicaciones a la API, pero por varios cofrades virtuales repartidos a lo largo del cortejo, de la cabeza a la cola, que avanzan por el recorrido real de la procesión (el GPX importado). Permite configurar el número de cofrades, la longitud del cortejo, la velocidad de la procesión, un factor para acelerar el tiempo, el error típico del GPS y una proporción de ubicaciones erróneas, alejadas del recorrido. En cada ronda muestra cuántas ubicaciones ha aceptado y descartado el backend, y la estela que este calcula, que es lo que ve el ciudadano en el mapa.

La Tabla **??** recoge los escenarios probados con el simulador.

Escenario	Qué se comprueba	Requisitos
Seguimiento normal	La estela avanza en el mapa sin saltos y la cola va por detrás de la cabeza.	RF-16, RF-18, RF-24, RNF-05
Ubicaciones erróneas	Con un 30 % de ubicaciones alejadas del recorrido, ninguna se guarda y el mapa no da saltos.	RF-25
Recorrido circular	En una procesión que sale y vuelve al mismo templo, la estela no aparece completa al empezar.	RF-18
Fin de la procesión	Al finalizar la procesión, el backend deja de aceptar ubicaciones y el simulador se detiene.	RF-24, RF-34
Código revocado o caducado	Al revocar el código de acceso durante la simulación, el backend rechaza las ubicaciones y se informa del motivo.	RF-24, RF-31

Tabla 5.3: Escenarios probados con el simulador de procesiones

En todos los escenarios el sistema se comportó como se esperaba. La Figura **??** muestra la salida del simulador durante una de las pruebas: en cada ronda indica cuántas ubicaciones ha guardado y descartado el backend, el tramo del recorrido que ocupa la procesión (de la cola a la cabeza) y el número de ubicaciones activas. Las ubicaciones erróneas que genera el simulador aparecen como descartadas, y la estela avanza de forma continua sin retroceder.

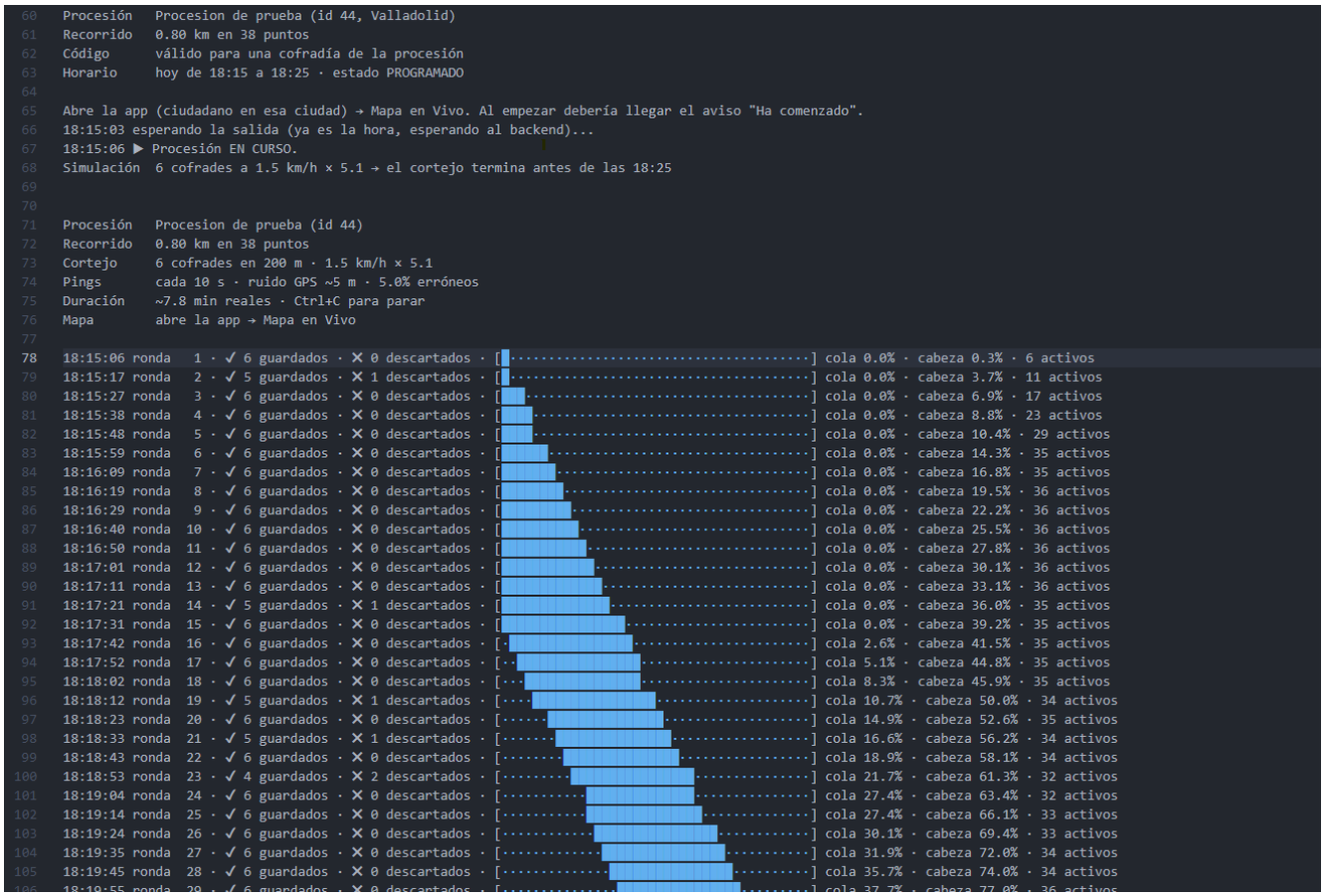


Figura 5.1: Salida del simulador de procesiones durante una prueba

5.3. Pruebas en directo

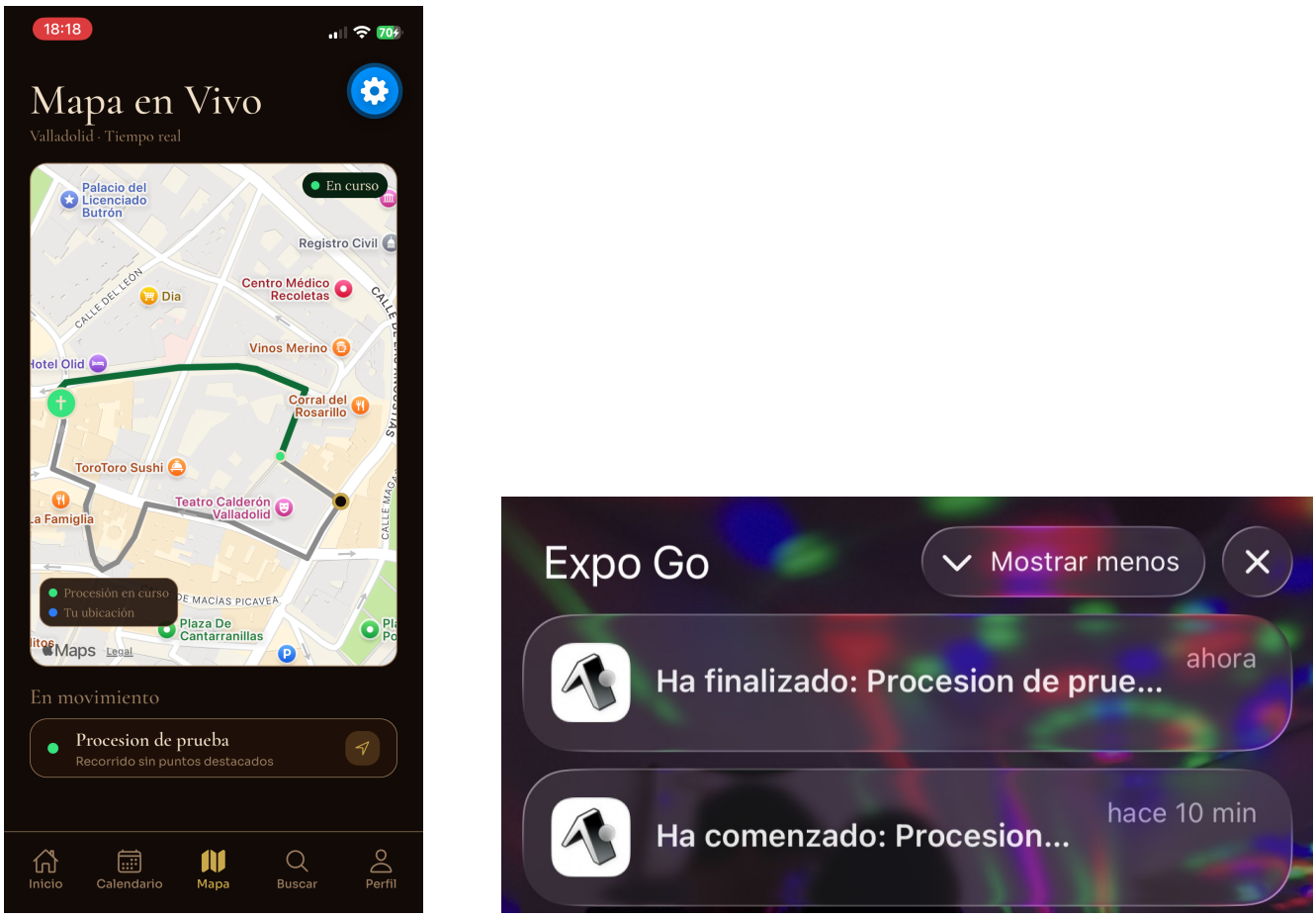
Además de las simulaciones, se han realizado pruebas con el sistema completo en condiciones reales: el backend desplegado en el servidor de la Universidad (Apéndice **??**), las notificaciones enviadas por Expo y la aplicación ejecutándose en un dispositivo físico. En estas pruebas las ubicaciones de los cofrades no se enviaron desde varios móviles, sino con el simulador de la Sección **??**, lanzado desde un programa auxiliar; el dispositivo físico se utilizó como ciudadano, para seguir las procesiones en el Mapa en Vivo y recibir las notificaciones.

5.3.1. Procesión de prueba

La primera prueba en directo se realizó el 26 de octubre de 2026 con una procesión de prueba de 10 minutos de duración, programada a las 18:15. Como el formulario de la aplicación solo permite elegir los días de la Semana Santa, se preparó con un programa auxiliar (`prueba-hoy.mjs`, junto al simulador) que traslada la procesión al día de la prueba, comprueba el código de acceso antes de la hora y lanza el simulador cuando el backend la pone en curso. Se comprobó:

- que el backend cambiaba la procesión a *en curso* por sí solo al llegar la hora de inicio, y que llegaba al dispositivo la notificación *push* de inicio (RF-34, RF-35, RNF-08);
- que la procesión aparecía en el Mapa en Vivo sin que el ciudadano tuviera que seleccionarla, y que su estela avanzaba a medida que se recibían las ubicaciones (RF-16, RF-18);
- que se descartaban las ubicaciones erróneas (RF-25);
- que al llegar la hora de fin el backend la finalizaba, enviaba la notificación de fin y dejaba de aceptar ubicaciones (RF-24, RF-34).

La prueba se completó sin incidencias. La Figura **??** muestra la procesión en el Mapa en Vivo, con la parte del recorrido ya ocupada por el cortejo marcada en verde, y las notificaciones de inicio y de fin recibidas en el dispositivo con 10 minutos de diferencia.



(a) Mapa en Vivo

(b) Notificaciones de inicio y de fin

Figura 5.2: Prueba en directo con una procesión

5.3.2. Varias procesiones simultáneas

Durante la Semana Santa es habitual que haya varias procesiones en la calle a la vez, a menudo con recorridos que comparten calles. Para comprobar que el sistema las distingue correctamente, el 28 de octubre de 2026 se realizó una segunda prueba en directo con tres procesiones de 10 minutos que salían con 3 minutos de diferencia, de forma que entre el minuto 6 y el 10 de la prueba las tres estaban en curso a la vez. Se utilizaron tres recorridos reales del centro de Valladolid, cortos y con calles en común, importados desde sus archivos GPX (Tabla **??**).

	Recorrido	**Cofradía**	**Horario**
P1	Ofrecimiento de los Dolores (0,9 km)	Santa Vera Cruz	H a H+10
P2	Soledad (0,8 km)	Nuestra Señora de las Angustias	H+3 a H+13
P3	Cristo de la Luz (1 km)	Santísimo Cristo de la Luz	H+6 a H+16

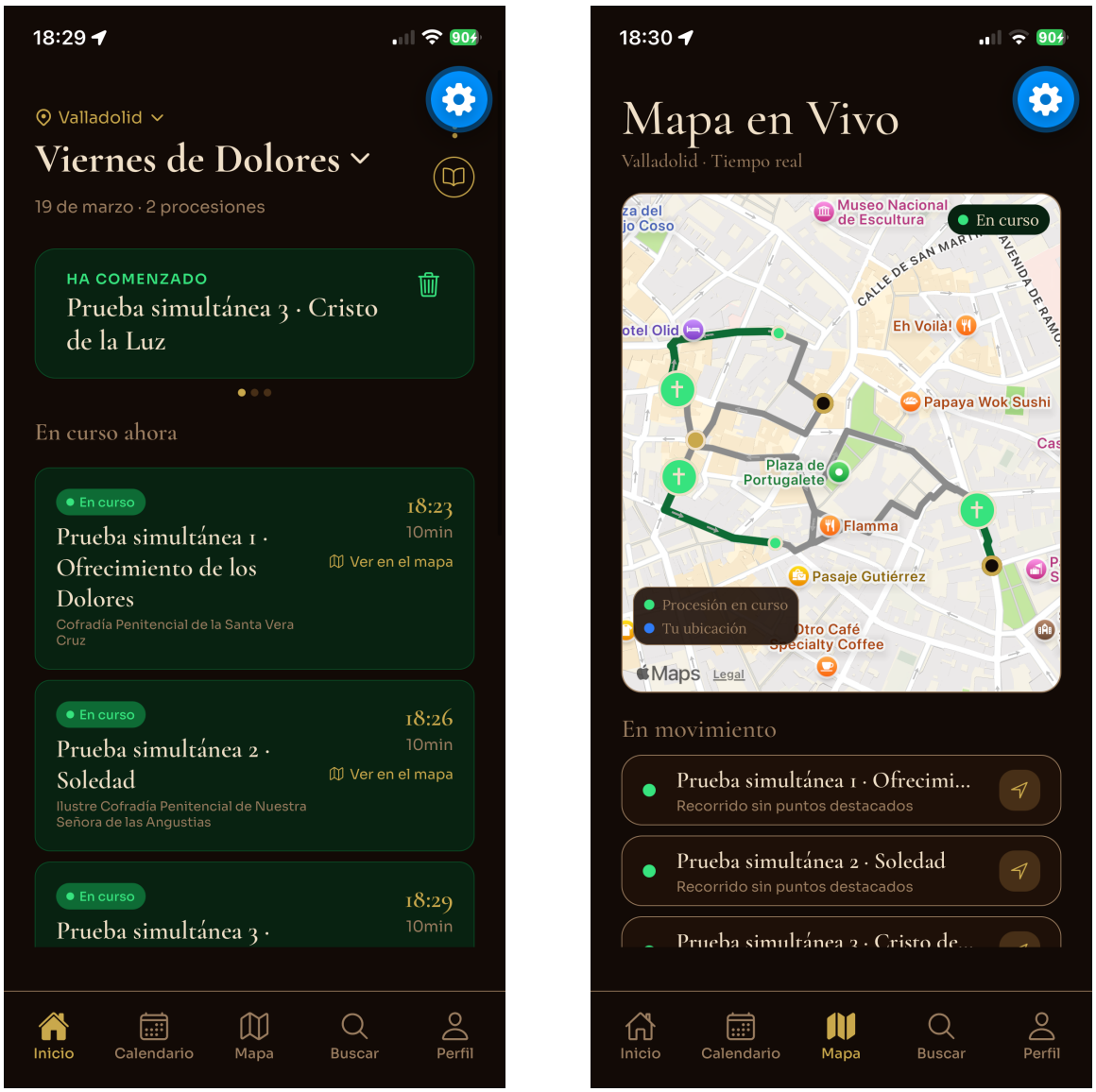
Tabla 5.4: Procesiones de la prueba de procesiones simultáneas

La prueba se preparó con otro programa auxiliar (`prueba-varias.mjs`) que, con las credenciales de la Junta de Cofradías, importa los tres recorridos, crea las procesiones para el día de la prueba, emite un código de acceso para cada cofradía y, cuando el backend pone cada procesión en curso, lanza un simulador para ella con cinco cofrades virtuales. Al terminar, el mismo programa elimina las procesiones y los recorridos de prueba. Se comprobó:

- que el backend ponía en curso y finalizaba cada procesión a su hora, de forma independiente, y que llegaban al dispositivo las seis notificaciones *push*, una de inicio y otra de fin por cada procesión (RF-34, RF-35, RNF-08);
- que el Mapa en Vivo mostraba las procesiones en curso a medida que salían, y las tres a la vez durante el tramo en que coincidían, cada una con su propia estela (RF-16, RF-18);
- que las ubicaciones de los cofrades de una procesión no alteraban la estela de las demás, aunque sus recorridos compartieran calles;
- que el código de una cofradía no permitía enviar ubicaciones a una procesión en la que esta no participa (RF-24).

Las tres procesiones se siguieron correctamente de principio a fin, sin que ninguna estela se mezclara con la de otra ni retrocediera, y cada una desapareció del Mapa en Vivo al finalizar.

La Figura **??** muestra el momento en que las tres procesiones estaban en curso a la vez: en la pantalla de inicio aparecen las tres en “En curso ahora”, junto al aviso de que acaba de comenzar la tercera, y en el Mapa en Vivo cada una se representa con su propia estela. La Figura **??** recoge la salida del programa durante ese mismo tramo, con las rondas de las tres procesiones intercaladas.



(a) Pantalla de inicio

(b) Mapa en Vivo

Figura 5.3: Las tres procesiones de prueba en curso a la vez

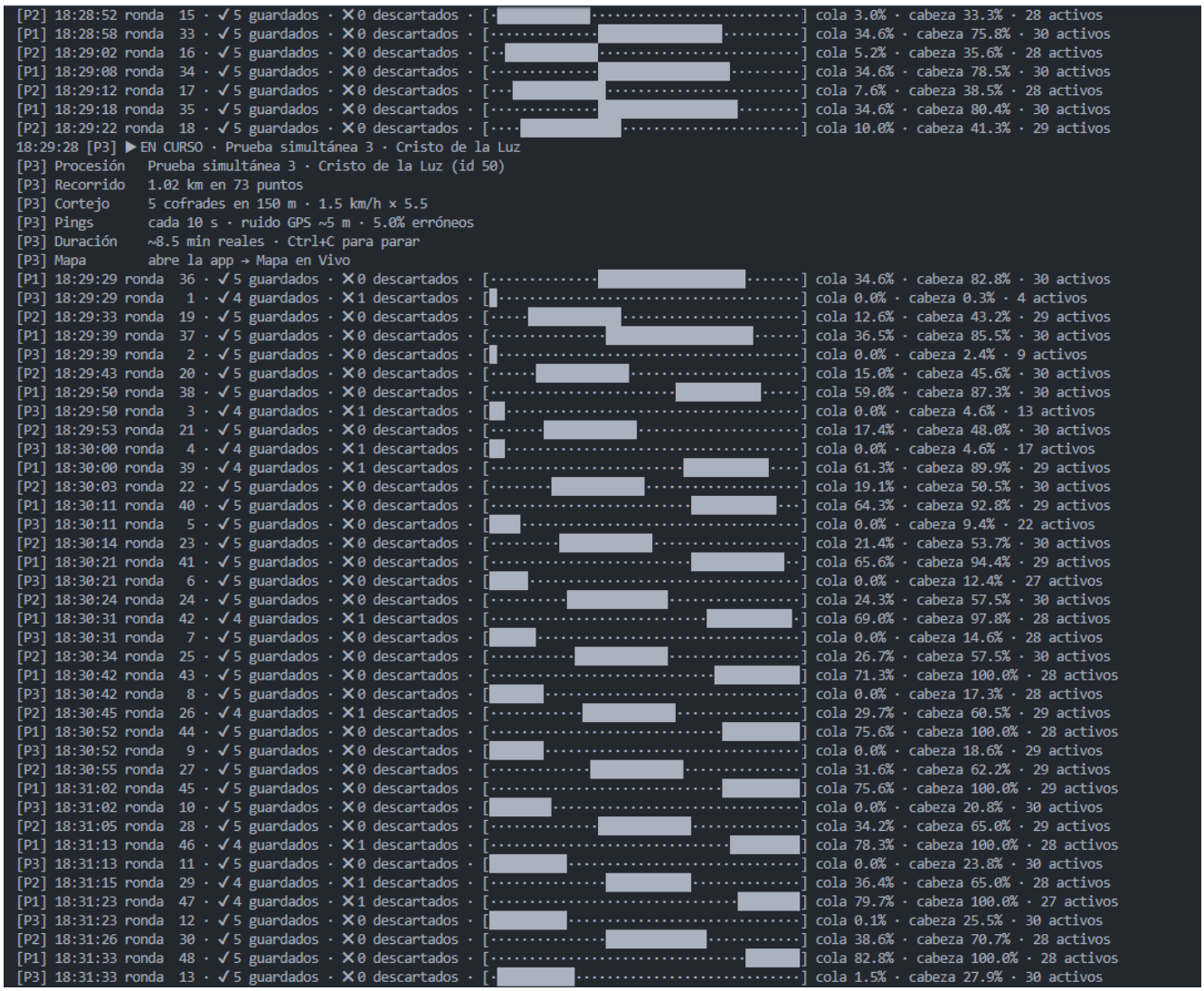


Figura 5.4: Salida de `prueba-varias.mjs` con las tres procesiones en curso

5.4. Pruebas manuales de aceptación

Por último, se ha recorrido cada caso de uso (Sección 3.3 y Apéndice **??**) sobre la aplicación, en Android y en iOS, con cada uno de los cuatro roles, comprobando la secuencia base y las secuencias alternativas. La Tabla **??** resume las pruebas agrupadas por rol.

Rol	Casos de uso probados	Resultado
Ciudadano	Seleccionar ciudad (con y sin permiso de ubicación), consultar información, listados, búsqueda y filtros, calendario, favoritos, estado de las notificaciones y Mapa en Vivo.	Superada
Cofrade	Iniciar sesión escaneando el QR y escribiendo el código, compartir ubicación (con la aplicación abierta y con la pantalla bloqueada), elegir procesión en curso y dejar de compartir.	Superada
Miembro de la Junta	Iniciar sesión, cambiar la contraseña provisional, solicitar la reactivación, gestionar cofradías, pasos, eventos y procesiones, importar el GPX, marcar puntos de interés, asignar pasos, crear notificaciones y gestionar los códigos QR.	Superada
Administrador	Gestionar ciudades, Juntas y miembros, reenviar la invitación, gestionar las solicitudes de reactivación y el propio perfil.	Superada

Tabla 5.5: Pruebas manuales de aceptación

Las capturas de cada pantalla de la aplicación se recogen en el apartado de implementación de la iteración en la que se desarrolló (Capítulo 4). Como ejemplo de las pruebas del rol de cofrade, la Figura **??** muestra la pantalla del cofrade compartiendo su ubicación en una de las procesiones de prueba. Al ejecutarse en Expo Go, que no permite la ubicación en segundo plano, la aplicación avisa de que solo se compartirá mientras esté abierta.



Figura 5.5: Cofrade compartiendo su ubicación durante una procesión de prueba

5.5. Errores detectados y corregidos

Los errores encontrados durante las pruebas se han registrado en un documento del repositorio (`bugs.md`), con su prioridad, la causa, la solución aplicada y, cuando procede, la prueba automática que lo comprueba. En total se registraron 45 errores, de los que la gran mayoría están corregidos. Entre los más relevantes:

- **Fallos de autorización**: cualquier usuario con sesión podía eliminar administradores; una Junta desactivada seguía pudiendo modificar datos; una Junta podía editar recorridos y ubicaciones de otra ciudad; y un código de acceso revocado seguía sirviendo hasta que caducaba la sesión del cofrade. Todos se corrigieron en el backend y se añadieron pruebas automáticas que los comprueban.
- **Seguimiento en tiempo real**: el cofrade dejaba de enviar su ubicación al bloquear el móvil, lo que se resolvió con una tarea en segundo plano; en los recorridos que salen y vuelven al mismo templo la estela aparecía completa desde la salida; solo se mostraba una procesión en curso aunque hubiera varias; y el cofrade no recibía ningún aviso cuando sus ubicaciones se descartaban por estar

fuera del recorrido.

- **Integridad de los datos**: editar una procesión con varias cofradías dejaba solo una y borraba su templo de salida, y eliminar un elemento con datos asociados fallaba sin explicar el motivo.

Los errores que quedan pendientes son de prioridad media o baja y se recogen como trabajo futuro (Sección **??**): el envío de notificaciones a más de 100 dispositivos de una misma ciudad, la consulta sin conexión del detalle de procesiones y eventos, la representación de los puntos de interés en el mapa y la limitación de intentos al introducir un código de acceso.

Capítulo 6

Conclusiones

6.1. Consecución de los objetivos

El objetivo principal del proyecto era desarrollar una aplicación móvil que reuniera en una sola plataforma la información de la Semana Santa de distintas ciudades y permitiera seguir las procesiones en tiempo real. El resultado es una aplicación para Android e iOS, respaldada por un backend propio desplegado en un servidor de la Universidad de Valladolid, que cubre ese objetivo. A continuación se repasan los objetivos específicos planteados en la Sección 1.4:

- **Elegir entre las distintas Semanas Santas.** El ciudadano elige la ciudad al entrar, o la aplicación selecciona automáticamente la más cercana a partir de su ubicación, y puede cambiarla en cualquier momento. Cada ciudad tiene su propia Junta de Cofradías, que gestiona su contenido.

- **Información general de la Semana Santa.** La aplicación muestra la historia y el patrimonio de cada ciudad, y la información de sus cofradías y pasos.

- **Información detallada de los actos.** Cada procesión y evento muestra su día, hora, duración, estado, cofradías y pasos, y su recorrido o ubicación sobre el mapa. Además, el ciudadano dispone de un calendario, de listados completos y de un buscador con filtros.

- **Personalización y notificaciones.** El ciudadano puede marcar como favoritos cofradías, procesiones, eventos y pasos, y recibe notificaciones *push* de su ciudad: automáticas, cuando una procesión o un evento comienza o termina, y manuales, cuando la Junta informa de un cambio, una incidencia o una cancelación.

- **Mapa con la evolución de las procesiones en tiempo real.** Es la funcionalidad más característica del sistema. Los cofrades comparten su ubicación de forma anónima, incluso con la pantalla bloqueada; el backend descarta las ubicaciones erróneas y calcula qué tramo del recorrido ocupa la procesión, y el ciudadano lo ve sobre el mapa sin necesidad de buscarla. Se ha comprobado con el simulador de procesiones y con una prueba en directo (Capítulo **??**).

- **Solución accesible desde dispositivos móviles.** La misma aplicación funciona en Android y en iOS gracias a React Native y Expo, con una interfaz basada en los *mockups* diseñados al inicio del proyecto.
- **Arquitectura escalable.** Añadir una ciudad nueva no exige ningún cambio en el sistema: basta con que el administrador la dé de alta y cree su Junta de Cofradías, que a partir de ahí gestiona su propio contenido. Como muestra, se han cargado las procesiones de la Semana Santa de Valladolid con sus recorridos oficiales.

No todos los requisitos se han llegado a implementar. Han quedado fuera, por falta de tiempo y por ser de prioridad baja, el cálculo de rutas hacia una procesión o que eviten las procesiones activas (RF-19 y RF-20) y la representación en el mapa de los puntos de interés del recorrido (RF-17). La consulta sin conexión (RNF-06) se ha limitado a los datos que ya se han visitado. Todos ellos se recogen como trabajo futuro.

Más allá del resultado, el proyecto ha permitido aplicar de principio a fin el proceso de desarrollo estudiado en el Grado: análisis de requisitos y casos de uso, diseño de una arquitectura en capas y de un modelo de datos relacional, implementación de una API propia con autenticación y autorización por roles, pruebas automáticas y despliegue en un servidor real. El cambio de una plataforma *Backend-as-a-Service* a un backend propio, decidido en la primera iteración, ha supuesto más trabajo, pero también el aprendizaje que se buscaba con él.

6.2. Trabajo futuro

Como líneas de trabajo futuro se plantean las siguientes ampliaciones del sistema.

En primer lugar, el desarrollo de una página web para la aplicación, que permita a los usuarios acceder a determinadas funcionalidades sin necesidad de instalar la aplicación móvil.

En segundo lugar, se plantea la posibilidad de generalizar el sistema desarrollado para dar soporte no solo a la Semana Santa, sino a cualquier tipo de evento o celebración tradicional (por ejemplo, ferias, romerías o fiestas patronales). Para ello, entre otros cambios, las entidades específicas del dominio actual deberían sustituirse por clases más genéricas: las cofradías o hermandades pasarían a modelarse como *organizaciones*, sus miembros o cofrades como *miembros*, y las Juntas de Cofradías como *conjuntos de organizaciones* encargados de coordinar a varias organizaciones dentro de un mismo evento. De esta forma, el resto del sistema (geolocalización en tiempo real, gestión de recorridos, notificaciones, etc.) podría reutilizarse prácticamente sin cambios, ya que estas funcionalidades no dependen de la naturaleza religiosa del evento, sino de su estructura organizativa. Esta generalización estaría alineada con el objetivo de escalabilidad planteado inicialmente para el proyecto (Sección 1.4), y permitiría ampliar el alcance de la aplicación mucho más allá del ámbito de la Semana Santa.

En tercer lugar, la imagen de un `Paso` se guarda hoy como una URL de texto libre (Apéndice **??**), sin ninguna infraestructura de subida de archivos: es la Junta de Cofradías quien aloja la imagen en otro

sitio y pega el enlace en el formulario, en lugar de seleccionarla directamente desde el propio dispositivo. Como mejora futura, esto podría resolverse de dos formas: guardando la imagen como un campo binario en la propia base de datos PostgreSQL –sin depender de ningún servicio externo, aunque no es la solución más eficiente para servir muchas imágenes pesadas a la vez–, o retomando el planteamiento inicial de usar **Firebase Storage** [**?**] –un servicio dedicado a este propósito, con mejor rendimiento a mayor escala, pero que exigiría gestionar las credenciales de un proyecto de Firebase además del backend propio ya existente–.

En cuarto lugar, la importación del recorrido desde un archivo GPX podría aprovechar más información del propio archivo. Hoy solo se leen las coordenadas de los puntos, y los puntos de interés se marcan después uno a uno desde la aplicación, lo que resulta lento en un recorrido largo. El formato GPX permite incluir puntos con nombre (`<wpt>`) junto al trazado, de modo que la Junta podría señalar en la propia herramienta con la que dibuja la ruta los puntos de interés que quiere marcar, y la aplicación crearlos automáticamente al importar el archivo, dejando la edición posterior solo para completar o corregir su información.

En quinto lugar, las notificaciones solo pueden activarse o desactivarse en bloque, desde el permiso de notificaciones del dispositivo. Una mejora natural sería permitir al ciudadano elegir desde la propia aplicación qué tipos de aviso quiere recibir (por ejemplo, solo los de procesiones o solo los de eventos). Esto exigiría guardar esas preferencias junto a cada dispositivo registrado y asociar cada notificación al tipo de acto del que informa, para que el backend pueda filtrar a quién se la envía.

En sexto lugar, quedan por completar los requisitos que no se han llegado a implementar: el cálculo de rutas a pie hacia los lugares más relevantes de una procesión, evitando las procesiones activas (RF-19 y RF-20), que podría apoyarse en un servicio de rutas sobre OpenStreetMap como el utilizado para preparar los recorridos; la representación en el mapa de los puntos de interés que ya marca la Junta (RF-17); y la consulta sin conexión del detalle de las procesiones y los eventos ya visitados (RNF-06).

En séptimo lugar, convendría reforzar varios aspectos técnicos detectados durante las pruebas (Sección **??**): enviar las notificaciones *push* en lotes de 100, que es el máximo que admite Expo en cada petición, para ciudades con muchos dispositivos registrados; limitar el número de intentos al introducir un código de acceso; y guardar el token de sesión en el almacenamiento cifrado del dispositivo.

Por último, sería interesante adaptar la interfaz en función de la Semana Santa de cada ciudad, asociando a cada una la terminología propia que se emplea localmente, de forma que la experiencia resulte más cercana y personalizada para cada usuario.

6.3. Puntos débiles

El sistema presenta una seguridad limitada en el acceso de los cofrades, que se realiza con un código de acceso compartido por toda la cofradía, ya sea escaneando su código QR o escribiéndolo a mano. Cualquiera que vea el QR o el código, por ejemplo en una fotografía, podría usarlo. No obstante, se ha priorizado la usabilidad frente a la seguridad, ya que el nivel de riesgo asociado a este método de acceso

se considera asumible dado el contexto de uso de la aplicación, la Junta de Cofradías puede revocar el código en cualquier momento, cada código caduca 24 horas después de emitirse, y solo permite compartir la ubicación mientras la procesión está en curso y cerca de su recorrido.

Además, la comunicación entre la aplicación y el servidor se realiza mediante HTTP, sin cifrar, porque el servidor de despliegue no dispone de un certificado TLS. En consecuencia, las credenciales de inicio de sesión, los tokens de sesión y las posiciones que comparten los cofrades viajan por la red sin cifrar y podrían ser interceptados en una red no segura. Para un uso real, sería imprescindible servir la API mediante HTTPS, con un dominio propio y un certificado TLS (por ejemplo, de Let's Encrypt), lo que además permitiría retirar de la aplicación las excepciones que hoy permiten el tráfico HTTP.

Por otro lado, la introducción de información a través del dispositivo móvil resulta costosa para el usuario. Una alternativa más adecuada sería ofrecer una página web con capacidad de edición tanto para las juntas de cofradías como para los administradores. Sin embargo, por motivos de acotación del alcance del proyecto y de la carga de trabajo disponible, esta funcionalidad no se ha llegado a implementar.

Asimismo, el uso de un backend propio con Spring Boot y PostgreSQL, en lugar de una plataforma *Backend-as-a-Service* como Firebase, traslada a la propia autora responsabilidades que antes asumía la plataforma de forma automática: el dimensionado y escalado de la infraestructura, las copias de seguridad de la base de datos, y la aplicación de parches de seguridad al propio servidor. Al desplegarse en un servidor virtual gestionado directamente (Apéndice **??**), estas tareas, junto con la actualización del servicio a cada nueva versión, recaen por completo en la propia autora, algo que con Firebase no era necesario.

Asimismo, la posición en vivo de una procesión (RNF-05) se resuelve mediante una API REST convencional consultada periódicamente por el cliente, en lugar de mediante los *listeners* en tiempo real que ofrecía Cloud Firestore de forma nativa. Esta solución cumple el requisito tal y como está definido (una actualización cada 30 segundos como mínimo), pero no ofrece la misma sensación de instantaneidad que un mecanismo de *push*, y su eficiencia se degrada más rápido al aumentar el número de usuarios simultáneos que la consultan, ya que cada consulta es una petición HTTP independiente en lugar de una única notificación que el servidor reenvía a quien esté suscrito. Del mismo modo, la disponibilidad sin conexión (RNF-06), que antes cubría de forma automática la caché local de Firestore, exige ahora una solución explícita en el cliente, que solo se ha aplicado a una parte de los datos.

También el desarrollo de un backend propio con una tecnología (Spring Boot) con la que la autora contaba con una experiencia previa limitada –un único proyecto de prácticas durante el Grado– ha supuesto una curva de aprendizaje que no era necesaria con el planteamiento inicial basado en Firebase, y que se ha gestionado mediante una prueba de concepto acotada antes de comenzar la implementación completa, tal y como recoge el riesgo correspondiente (Sección 2.3).

Por último, los códigos de acceso se asocian a una cofradía y no a una procesión concreta (Sección 4.1), por lo que un mismo código permite compartir la ubicación en cualquier procesión en curso de esa cofradía. Se considera una limitación menor, ya que una cofradía rara vez tiene varias procesiones en la calle a la vez, y en ese caso la aplicación pide al cofrade que elija en cuál participa.

Anexos

Apéndice A

Casos de uso adicionales

Este apéndice recoge los casos de uso adicionales del sistema que, por no considerarse esenciales para la comprensión general del proyecto, no se han incluido en el cuerpo principal de la memoria (Sección 3.3). Cada uno de ellos sigue la misma estructura que los presentados en dicha sección: resumen, actor, precondición, postcondición, secuencia base, secuencia alternativa y requisitos relacionados. Se agrupan por el rol que los realiza, en el mismo orden que los diagramas de casos de uso; los casos de uso que comparten el miembro de la Junta de Cofradías y el administrador (iniciar sesión y gestionar su propio perfil) se recogen una sola vez, junto a los del miembro de la Junta.

A.1. Ciudadano

A.2. Cofrade

A.3. Miembro de la Junta de Cofradías

A.4. Administrador

Caso de uso: Seleccionar ciudad	
Resumen	El usuario elige la ciudad de la que quiere consultar la Semana Santa.
Actor	Ciudadano
Precondición	Ninguna.
Postcondición	La ciudad elegida queda guardada en el dispositivo, y el dispositivo queda registrado para recibir las notificaciones de esa ciudad.
Secuencia Base	
[1] El sistema solicita permiso para acceder a la ubicación del dispositivo, la primera vez que se accede como ciudadano.	
[2] El usuario concede el permiso.	
[3] El sistema selecciona automáticamente la ciudad activa más cercana a la ubicación del usuario.	
[4] El sistema guarda la ciudad seleccionada y registra el dispositivo para recibir sus notificaciones.	
Secuencia Alternativa	
[2.a.1] El usuario deniega el permiso de ubicación: el sistema muestra las ciudades activas, ordenadas por nombre, y el usuario elige una; el caso de uso continúa en el paso 4.	
[3.a.1] No se puede obtener la ubicación o no hay ninguna ciudad cercana: el sistema muestra las ciudades activas y el usuario elige una.	
[3.a.2] El usuario busca una ciudad por su nombre: el sistema muestra solo las ciudades que coinciden con la búsqueda.	
[4.a.1] El usuario cambia más adelante de ciudad: el sistema muestra las ciudades activas, guarda la nueva y vuelve a registrar el dispositivo para las notificaciones de esa ciudad.	
[4.a.2] El usuario desiste de elegir ciudad la primera vez: el sistema olvida que había entrado como ciudadano y vuelve al inicio de la aplicación.	
Requisitos Relacionados	RF-02, RF-03 y RF-04.

Tabla A.1: Caso de uso: Seleccionar ciudad

Caso de uso: Consultar información de una cofradía, procesión, evento o paso	
Resumen	El usuario consulta la información detallada de una cofradía, procesión, evento o paso de la ciudad seleccionada.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad.
Postcondición	Se muestra la información detallada del elemento elegido.
Secuencia Base	
[1] El usuario elige una cofradía, procesión, evento o paso, ya sea desde un listado, desde una búsqueda, desde el calendario o desde sus favoritos.	
[2] El sistema muestra la información detallada del elemento: en una cofradía, su historia, su web, sus pasos y las procesiones y eventos en los que participa; en una procesión, su día, hora de salida, duración, estado, cofradías, pasos y el recorrido sobre el mapa; en un evento, su día, hora, duración, estado, cofradías, pasos y la ubicación sobre el mapa; en un paso, su historia, su análisis artístico, su imagen y su cofradía.	
[3] El usuario puede pasar de un elemento a otro relacionado con él (por ejemplo, de una procesión a uno de sus pasos).	
Secuencia Alternativa	
[2.a.1] La procesión todavía no tiene recorrido: el sistema indica que el recorrido está pendiente de confirmar.	
[2.a.2] La procesión está en curso: el sistema muestra además, sobre el recorrido, el tramo que ocupa en ese momento.	
[2.a.3] Sin conexión: el sistema muestra la última información consultada del elemento, si la hay, o informa de que no puede cargarla.	
Requisitos Relacionados	RF-05, RF-06, RF-07 y RF-08.

Tabla A.2: Caso de uso: Consultar información de una cofradía, procesión, evento o paso

Caso de uso: Acceder a listados de procesiones, pasos, eventos o cofradías	
Resumen	El usuario consulta el listado completo de procesiones, pasos, eventos o cofradías de la ciudad seleccionada.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad (caso de uso «Seleccionar ciudad»).
Postcondición	Se muestra el listado del tipo de elemento elegido.
Secuencia Base	
[1] El usuario solicita el listado de procesiones, pasos, eventos o cofradías.	
[2] El sistema muestra todos los elementos de ese tipo de la ciudad seleccionada, ordenados por nombre, indicando el estado de las procesiones y los eventos.	
[3] El usuario puede escribir un texto para reducir el listado a los elementos cuyo nombre lo contiene.	
[4] El usuario elige un elemento y se realiza el caso de uso «Consultar información de una cofradía, procesión, evento o paso».	
Secuencia Alternativa	
[2.a.1] No hay elementos de ese tipo en la ciudad: el sistema informa de que no hay elementos registrados.	
[2.a.2] Error de conexión: el sistema informa del error y permite volver a intentarlo.	
[3.a.1] Ningún elemento coincide con el texto: el sistema informa de que no hay resultados.	
Requisitos Relacionados	RF-11.

Tabla A.3: Caso de uso: Acceder a listados de procesiones, pasos, eventos o cofradías

Caso de uso: Buscar cofradías, eventos, procesiones o pasos	
Resumen	El usuario busca por su nombre cofradías, eventos, procesiones o pasos de la ciudad seleccionada.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad (caso de uso «Seleccionar ciudad»).
Postcondición	Se muestran los elementos que coinciden con la búsqueda.
Secuencia Base	
[1] El usuario solicita realizar una búsqueda.	
[2] El usuario introduce el texto que quiere buscar.	
[3] El sistema muestra las procesiones y los eventos cuyo nombre contiene el texto, ordenados por hora, junto con los pasos y las cofradías que también coinciden.	
[4] El usuario elige un resultado y se realiza el caso de uso «Consultar información de una cofradía, procesión, evento o paso».	
Secuencia Alternativa	
[2.a.1] El usuario no introduce ningún texto: el sistema muestra todos los elementos de la ciudad.	
[3.a.1] El usuario quiere acotar los resultados: se realiza el caso de uso «Filtrar elementos».	
[3.a.2] No hay resultados: el sistema informa de que no se han encontrado elementos.	
Requisitos Relacionados	RF-12.

Tabla A.4: Caso de uso: Buscar cofradías, eventos, procesiones o pasos

Caso de uso: Filtrar elementos	
Resumen	El usuario acota los resultados de una búsqueda según el día de la Semana Santa y el estado de las procesiones y los eventos.
Actor	Ciudadano
Precondición	El usuario está realizando una búsqueda (caso de uso «Buscar cofradías, eventos, procesiones o pasos»).
Postcondición	Los resultados de la búsqueda se limitan a los que cumplen los filtros elegidos.
Secuencia Base	
[1] El usuario solicita filtrar los resultados.	
[2] El sistema ofrece filtrar por día de la Semana Santa y por estado (en curso, programado, finalizado o cancelado).	
[3] El usuario elige un día, un estado o ambos.	
[4] El sistema muestra solo las procesiones y los eventos que cumplen los filtros elegidos.	
Secuencia Alternativa	
[3.a.1] El usuario quita los filtros: el sistema vuelve a mostrar todos los resultados de la búsqueda.	
[4.a.1] Hay algún filtro activo: el sistema deja de mostrar los pasos y las cofradías, que no tienen día ni estado.	
[4.a.2] Ningún elemento cumple los filtros: el sistema informa de que no hay resultados.	
Requisitos Relacionados	RF-13.

Tabla A.5: Caso de uso: Filtrar elementos

Caso de uso: Consultar calendario de Semana Santa	
Resumen	El usuario consulta en un calendario mensual qué días tienen procesiones o eventos programados.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad.
Postcondición	Se muestra el calendario del mes, indicando los días con procesiones o eventos.
Secuencia Base	
[1] El usuario solicita consultar el calendario.	
[2] El sistema muestra el mes de la Semana Santa e indica qué días tienen procesiones o eventos en la ciudad seleccionada.	
[3] El usuario puede avanzar o retroceder de mes.	
[4] El usuario elige un día y se realiza el caso de uso «Consultar procesiones y eventos de un día».	
Secuencia Alternativa	
[2.a.1] La ciudad no tiene procesiones ni eventos: el sistema muestra el calendario sin ningún día indicado.	
Requisitos Relacionados	RF-09.

Tabla A.6: Caso de uso: Consultar calendario de Semana Santa

Caso de uso: Consultar procesiones y eventos de un día	
Resumen	El usuario consulta las procesiones y los eventos programados en un día concreto.
Actor	Ciudadano
Precondición	El usuario ha elegido un día del calendario (caso de uso «Consultar calendario de Semana Santa»).
Postcondición	Se muestran las procesiones y los eventos de ese día.
Secuencia Base	
[1] El sistema muestra la semana del día elegido y el resumen del día: su nombre dentro de la Semana Santa, su fecha y cuántas procesiones y eventos tiene.	
[2] El sistema muestra las procesiones y los eventos de ese día, ordenados por hora, con sus cofradías, su hora y su duración.	
[3] El usuario puede pasar a otro día de la misma semana, o a la semana anterior o siguiente.	
[4] El usuario elige una procesión o un evento y se realiza el caso de uso «Consultar información de una cofradía, procesión, evento o paso».	
Secuencia Alternativa	
[2.a.1] El día no tiene procesiones ni eventos: el sistema informa de que no hay nada registrado ese día.	
[4.a.1] El usuario vuelve al calendario mensual: se realiza de nuevo el caso de uso «Consultar calendario de Semana Santa».	
Requisitos Relacionados	RF-10.

Tabla A.7: Caso de uso: Consultar procesiones y eventos de un día

Caso de uso: Añadir favoritos	
Resumen	El usuario marca como favorita una cofradía, procesión, evento o paso.
Actor	Ciudadano
Precondición	El elemento no está marcado como favorito.
Postcondición	El elemento queda guardado como favorito en el dispositivo.
Secuencia Base	
[1] El usuario indica que quiere marcar como favorito un elemento que está consultando.	
[2] El sistema guarda el elemento como favorito en el dispositivo.	
[3] El sistema indica que el elemento ya es favorito.	
Secuencia Alternativa	
No se contemplan secuencias alternativas.	
Requisitos Relacionados	RF-14.

Tabla A.8: Caso de uso: Añadir favoritos

Caso de uso: Eliminar favorito	
Resumen	El usuario deja de tener como favorito una cofradía, procesión, evento o paso.
Actor	Ciudadano
Precondición	El elemento está marcado como favorito.
Postcondición	El elemento deja de estar guardado como favorito.
Secuencia Base	
[1] El usuario indica que quiere quitar de favoritos un elemento que está consultando.	
[2] El sistema quita el elemento de los favoritos guardados en el dispositivo.	
[3] El sistema indica que el elemento ya no es favorito.	
Secuencia Alternativa	
No se contemplan secuencias alternativas.	
Requisitos Relacionados	RF-14.

Tabla A.9: Caso de uso: Eliminar favorito

Caso de uso: Consultar favoritos	
Resumen	El usuario consulta reunidos todos los elementos que ha marcado como favoritos.
Actor	Ciudadano
Precondición	Ninguna.
Postcondición	Se muestra el listado de favoritos del usuario.
Secuencia Base	
[1] El usuario solicita consultar sus favoritos.	
[2] El sistema muestra cuántos favoritos tiene el usuario y de qué tipo (cofradías, procesiones, eventos y pasos).	
[3] El sistema muestra el listado de todos los favoritos, ordenados por nombre.	
[4] El usuario elige uno y se realiza el caso de uso «Consultar información de una cofradía, procesión, evento o paso».	
Secuencia Alternativa	
[2.a.1] El usuario no tiene favoritos: el sistema informa de que todavía no ha marcado ninguno.	
[4.a.1] El usuario quita un elemento de favoritos desde el listado: se realiza el caso de uso «Eliminar favorito» y el sistema actualiza el listado.	
Requisitos Relacionados	RF-14.

Tabla A.10: Caso de uso: Consultar favoritos

Caso de uso: Consultar estado de las notificaciones	
Resumen	El usuario consulta si tiene activadas las notificaciones de su ciudad y, si quiere cambiarlo, accede a los ajustes del dispositivo.
Actor	Ciudadano
Precondición	El usuario ha seleccionado una ciudad.
Postcondición	El usuario conoce si recibe o no las notificaciones de su ciudad.
Secuencia Base	
[1] El usuario solicita consultar el estado de las notificaciones.	
[2] El sistema consulta el permiso de notificaciones del dispositivo e informa de si están activadas, indicando la ciudad de la que se reciben.	
[3] El usuario indica que quiere cambiarlo.	
[4] El sistema abre los ajustes de la aplicación en el dispositivo, donde el usuario activa o desactiva las notificaciones.	
[5] Al volver a la aplicación, el sistema consulta de nuevo el permiso y actualiza el estado mostrado.	
Secuencia Alternativa	
[2.a.1] Las notificaciones están desactivadas: el sistema informa de que no recibirá los avisos de su ciudad hasta que las active en los ajustes del dispositivo.	
[3.a.1] El usuario no quiere cambiar nada: el caso de uso termina.	
Requisitos Relacionados	RF-21 y RF-22.

Tabla A.11: Caso de uso: Consultar estado de las notificaciones

Caso de uso: Generar ruta hacia procesión	
Resumen	El usuario obtiene una ruta desde su ubicación hasta un lugar relevante de una procesión (su inicio, un punto de interés o su fin), evitando las procesiones activas. **Caso de uso no implementado en la versión actual** (ver Sección **??**).
Actor	Ciudadano
Precondición	El usuario ha concedido permiso de ubicación y la procesión tiene recorrido.
Postcondición	Se muestra sobre el mapa una ruta desde la ubicación del usuario hasta el lugar elegido.
Secuencia Base	
[1] El usuario elige una procesión y el lugar al que quiere llegar (inicio, punto de interés o fin).	
[2] El sistema obtiene la ubicación actual del usuario.	
[3] El sistema calcula una ruta a pie hasta ese lugar que evita los tramos ocupados por procesiones activas.	
[4] El sistema muestra la ruta sobre el mapa.	
Secuencia Alternativa	
[2.a.1] No se puede obtener la ubicación del usuario: el sistema informa de que no puede calcular la ruta.	
[3.a.1] No existe ninguna ruta que evite las procesiones activas: el sistema informa de ello y ofrece la ruta más corta.	
Requisitos Relacionados	RF-19 y RF-20.

Tabla A.12: Caso de uso: Generar ruta hacia procesión

Caso de uso: Elegir procesión en curso	
Resumen	El cofrade elige en qué procesión quiere compartir su ubicación cuando su cofradía tiene varias en curso a la vez.
Actor	Cofrade
Precondición	El cofrade ha solicitado compartir su ubicación y su cofradía tiene más de una procesión en curso (caso de uso «Compartir ubicación durante procesión»).
Postcondición	Queda elegida la procesión en la que se compartirá la ubicación.
Secuencia Base	
[1] El sistema muestra las procesiones en curso de la cofradía.	
[2] El cofrade elige una de ellas.	
[3] El caso de uso «Compartir ubicación durante procesión» continúa con la procesión elegida.	
Secuencia Alternativa	
[2.a.1] El cofrade no elige ninguna: no se comparte la ubicación.	
Requisitos Relacionados	RF-24.

Tabla A.13: Caso de uso: Elegir procesión en curso

Caso de uso: Dejar de compartir ubicación	
Resumen	El cofrade deja de compartir su ubicación en cualquier momento.
Actor	Cofrade
Precondición	El cofrade está compartiendo su ubicación (caso de uso «Compartir ubicación durante procesión»).
Postcondición	El sistema deja de enviar la ubicación del cofrade.
Secuencia Base	
[1] El cofrade indica que quiere dejar de compartir su ubicación.	
[2] El sistema detiene el envío de la ubicación, también el que se realiza con la pantalla bloqueada.	
[3] El sistema informa de que ya no se está compartiendo la ubicación.	
Secuencia Alternativa	
[1.a.1] El cofrade cierra la sesión mientras comparte su ubicación: el sistema deja de compartirla igualmente.	
Requisitos Relacionados	RF-24.

Tabla A.14: Caso de uso: Dejar de compartir ubicación

Caso de uso: Iniciar sesión	
Resumen	Un miembro de la Junta de Cofradías o un administrador accede a la aplicación con su correo electrónico y su contraseña.
Actor	Miembro de la Junta de Cofradías, Administrador
Precondición	El usuario tiene una cuenta dada de alta en el sistema.
Postcondición	El usuario queda identificado con su rol y accede a las funcionalidades de gestión que le corresponden.
Secuencia Base	
[1] El usuario solicita iniciar sesión e indica si es miembro de una Junta de Cofradías o administrador.	
[2] El sistema solicita el correo electrónico y la contraseña.	
[3] El usuario introduce sus credenciales.	
[4] El sistema comprueba las credenciales y que la cuenta corresponde al tipo de usuario indicado.	
[5] El sistema inicia la sesión, válida durante 24 horas, y da acceso al usuario a sus funcionalidades de gestión.	
Secuencia Alternativa	
[3.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[4.a.1] Credenciales incorrectas: el sistema informa de que el correo o la contraseña no son correctos y no inicia la sesión.	
[4.a.2] La cuenta no corresponde al tipo de usuario indicado: el sistema lo informa y no inicia la sesión.	
[5.a.1] El miembro de la Junta todavía usa la contraseña provisional que recibió por correo: se realiza el caso de uso «Cambiar contraseña provisional» antes de darle acceso.	
[5.a.2] La cuenta del miembro de la Junta está desactivada: el sistema le informa de que no puede realizar cambios y le ofrece el caso de uso «Solicitar reactivación de cuenta».	
[5.a.3] La sesión caduca mientras se usa la aplicación: el sistema la cierra e informa al usuario de que debe volver a iniciar sesión.	
Requisitos Relacionados	RF-26.

Tabla A.15: Caso de uso: Iniciar sesión

Caso de uso: Cambiar contraseña provisional	
Resumen	El miembro de la Junta de Cofradías sustituye, la primera vez que inicia sesión, la contraseña provisional que recibió por correo por una propia.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión con la contraseña provisional que el sistema le envió al darle de alta o al reenviarle la invitación.
Postcondición	La contraseña provisional queda sustituida por la nueva, el alta del miembro queda completada y el usuario accede a sus funcionalidades de gestión.
Secuencia Base	
[1] El sistema informa al usuario de que debe elegir una contraseña propia antes de continuar, y le impide realizar cualquier otra operación hasta hacerlo.	
[2] El sistema solicita la contraseña provisional y la contraseña nueva, por duplicado.	
[3] El usuario introduce las contraseñas.	
[4] El sistema comprueba que la contraseña provisional es correcta, que las dos nuevas coinciden, que la nueva es distinta de la provisional y que tiene al menos 8 caracteres, una mayúscula, un número y un carácter especial.	
[5] El sistema guarda la contraseña nueva, marca el alta del miembro como completada y le da acceso a sus funcionalidades de gestión.	
Secuencia Alternativa	
[3.a.1] El usuario decide cerrar la sesión: el sistema la cierra sin cambiar la contraseña, y la próxima vez que inicie sesión se le volverá a pedir.	
[4.a.1] Contraseña provisional incorrecta: el sistema lo informa y no cambia la contraseña.	
[4.a.2] Las contraseñas nuevas no coinciden, la nueva es igual a la provisional o no cumple las reglas: el sistema indica qué debe corregir y no cambia la contraseña.	
Requisitos Relacionados	RF-26.

Tabla A.16: Caso de uso: Cambiar contraseña provisional

Caso de uso: Solicitar reactivación de cuenta	
Resumen	Un miembro de la Junta de Cofradías cuya cuenta ha sido desactivada pide al administrador que la reactive.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión y su cuenta está desactivada.
Postcondición	Queda registrada una solicitud de reactivación pendiente, que el administrador puede aceptar o rechazar.
Secuencia Base	
[1] El sistema informa al usuario de que su cuenta está desactivada y de que no puede realizar cambios.	
[2] El usuario solicita la reactivación de su cuenta.	
[3] El sistema registra la solicitud de reactivación como pendiente.	
[4] El sistema informa al usuario de que se ha avisado al administrador y de que debe esperar a que la revise.	
Secuencia Alternativa	
[2.a.1] El usuario no la solicita y cierra la sesión: el caso de uso termina sin registrar ninguna solicitud.	
[3.a.1] La cuenta ya ha sido reactivada entretanto: el sistema informa de que la cuenta ya está activa.	
Requisitos Relacionados	RF-41.

Tabla A.17: Caso de uso: Solicitar reactivación de cuenta

Caso de uso: Gestionar su propio perfil	
Resumen	El usuario consulta y modifica los datos de su propio perfil y, si lo desea, cambia su contraseña.
Actor	Miembro de la Junta de Cofradías, Administrador
Precondición	El usuario ha iniciado sesión. Si es miembro de la Junta de Cofradías, su cuenta está activa y ya ha sustituido su contraseña provisional.
Postcondición	Los datos del perfil quedan actualizados.
Secuencia Base	
[1] El usuario solicita consultar su perfil.	
[2] El sistema muestra su nombre, su correo electrónico y su teléfono.	
[3] El usuario modifica su nombre o su teléfono y, opcionalmente, introduce una contraseña nueva.	
[4] El sistema solicita la contraseña actual para confirmar los cambios.	
[5] El usuario introduce su contraseña actual y confirma.	
[6] El sistema comprueba la contraseña actual y guarda los cambios.	
Secuencia Alternativa	
[3.a.1] El usuario intenta cambiar su correo electrónico: el sistema no lo permite, ya que es su identificador para iniciar sesión.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[6.a.1] Contraseña actual incorrecta: el sistema lo informa y no guarda los cambios.	
[6.a.2] La contraseña nueva no tiene al menos 8 caracteres, una mayúscula, un número y un carácter especial: el sistema indica qué debe corregir y no guarda los cambios.	
Requisitos Relacionados	RF-27 y RF-40.

Tabla A.18: Caso de uso: Gestionar su propio perfil

Caso de uso: Editar información de la ciudad	
Resumen	El miembro de la Junta de Cofradías actualiza la información general sobre la Semana Santa de su ciudad.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión como miembro activo de la Junta de Cofradías y ya ha sustituido su contraseña provisional.
Postcondición	La historia y el patrimonio de la ciudad quedan actualizados.
Secuencia Base	
[1] El usuario solicita editar la información de su ciudad.	
[2] El sistema muestra la historia y el patrimonio actuales de la ciudad.	
[3] El usuario modifica la información y confirma.	
[4] El sistema guarda la información de la ciudad.	
Secuencia Alternativa	
[3.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
Requisitos Relacionados	RF-29.

Tabla A.19: Caso de uso: Editar información de la ciudad

Caso de uso: Gestionar cofradía	
Resumen	El miembro de la Junta de Cofradías da de alta, edita o da de baja las cofradías de su ciudad.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión como miembro activo de la Junta de Cofradías y ya ha sustituido su contraseña provisional.
Postcondición	Las cofradías de la ciudad quedan actualizadas.
Secuencia Base	
[1] El usuario solicita gestionar las cofradías de su ciudad.	
[2] El sistema muestra las cofradías de la ciudad.	
[3] El usuario indica si quiere dar de alta una cofradía nueva, editar una existente o darla de baja.	
[4] El sistema solicita los datos de la cofradía: nombre, historia, web oficial, imagen y si está activa (visible para los ciudadanos) o todavía en preparación.	
[5] El usuario introduce o modifica los datos y confirma.	
[6] El sistema valida los datos y guarda la cofradía en la ciudad del usuario.	
Secuencia Alternativa	
[3.a.1] Dar de baja: el sistema pide confirmación y, si el usuario confirma, elimina la cofradía.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[6.a.1] Datos incompletos o no válidos: el sistema indica qué datos debe corregir el usuario y no guarda la cofradía.	
Requisitos Relacionados	RF-28 y RF-29.

Tabla A.20: Caso de uso: Gestionar cofradía

Caso de uso: Gestionar pasos	
Resumen	El miembro de la Junta de Cofradías da de alta, edita o da de baja los pasos de las cofradías de su ciudad.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión como miembro activo de la Junta de Cofradías y ya ha sustituido su contraseña provisional. Existe al menos una cofradía en su ciudad.
Postcondición	Los pasos de la cofradía quedan actualizados.
Secuencia Base	
[1] El usuario solicita gestionar los pasos.	
[2] El sistema muestra los pasos de las cofradías de la ciudad.	
[3] El usuario indica si quiere dar de alta un paso nuevo, editar uno existente o darlo de baja.	
[4] El sistema solicita los datos del paso: nombre, cofradía a la que pertenece, historia, análisis artístico e imagen.	
[5] El usuario introduce o modifica los datos y confirma.	
[6] El sistema valida los datos y guarda el paso.	
Secuencia Alternativa	
[3.a.1] Dar de baja: el sistema pide confirmación y, si el usuario confirma, elimina el paso.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[6.a.1] Datos incompletos o no válidos: el sistema indica qué datos debe corregir el usuario y no guarda el paso.	
Requisitos Relacionados	RF-30.

Tabla A.21: Caso de uso: Gestionar pasos

Caso de uso: Gestionar eventos	
Resumen	El miembro de la Junta de Cofradías da de alta, edita, cambia de estado o da de baja los eventos de su ciudad.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión como miembro activo de la Junta de Cofradías y ya ha sustituido su contraseña provisional. Existe al menos una cofradía en su ciudad.
Postcondición	Los eventos de la ciudad quedan actualizados.
Secuencia Base	
[1] El usuario solicita gestionar los eventos de su ciudad.	
[2] El sistema muestra los eventos de la ciudad con su estado.	
[3] El usuario indica si quiere dar de alta un evento nuevo, editar uno existente o darlo de baja.	
[4] El sistema solicita los datos del evento: nombre, cofradía organizadora, día de la Semana Santa, hora de inicio, duración, ubicación (latitud, longitud y, opcionalmente, dirección), historia y web oficial; y, al editar, también su estado.	
[5] El usuario introduce o modifica los datos y confirma.	
[6] El sistema valida los datos y guarda el evento; al darlo de alta, queda en estado programado.	
[7] El sistema confirma los cambios y ofrece notificarlos a los ciudadanos.	
Secuencia Alternativa	
[3.a.1] Dar de baja: el sistema pide confirmación y, si el usuario confirma, elimina el evento.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[5.a.2] El usuario quiere elegir qué pasos participan en el evento: se realiza el caso de uso «Asignar pasos».	
[6.a.1] Datos incompletos o no válidos: el sistema indica qué datos debe corregir el usuario y no guarda el evento.	
[6.a.2] El cambio de estado contradice el horario (volver a programado una vez pasada la hora de inicio, o a en curso una vez pasada la de fin): el sistema no lo guarda e indica que se modifique la hora en lugar del estado.	
[6.a.3] El nuevo estado es en curso o finalizado: el sistema genera y envía automáticamente la notificación correspondiente a los ciudadanos de la ciudad.	
[7.a.1] El usuario quiere notificar los cambios, o ha cambiado el estado a cancelado: se realiza el caso de uso «Crear notificación».	
Requisitos Relacionados	RF-28, RF-29, RF-34 y RF-35.

Tabla A.22: Caso de uso: Gestionar eventos

Caso de uso: Gestionar procesión	
Resumen	El miembro de la Junta de Cofradías da de alta, edita, cambia de estado o da de baja las procesiones de su ciudad.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión como miembro activo de la Junta de Cofradías y ya ha sustituido su contraseña provisional. Existe al menos una cofradía en su ciudad.
Postcondición	Las procesiones de la ciudad quedan actualizadas.
Secuencia Base	
[1] El usuario solicita gestionar las procesiones de su ciudad.	
[2] El sistema muestra las procesiones de la ciudad con su estado.	
[3] El usuario indica si quiere dar de alta una procesión nueva, editar una existente o darla de baja.	
[4] El sistema solicita los datos de la procesión: nombre, cofradía, día de la Semana Santa, hora de salida, duración, web oficial, historia y tradición; y, al editar, también su estado.	
[5] El usuario introduce o modifica los datos y confirma.	
[6] El sistema valida los datos y guarda la procesión; al darla de alta, queda en estado programado.	
[7] El sistema confirma los cambios y ofrece notificarlos a los ciudadanos.	
Secuencia Alternativa	
[3.a.1] Dar de baja: el sistema pide confirmación y, si el usuario confirma, elimina la procesión junto con su recorrido.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[5.a.2] El usuario quiere definir el recorrido de la procesión: se realiza el caso de uso «Importar recorrido (GPX)».	
[5.a.3] El usuario quiere elegir qué pasos desfilan en la procesión: se realiza el caso de uso «Asignar pasos».	
[6.a.1] Datos incompletos o no válidos: el sistema indica qué datos debe corregir el usuario y no guarda la procesión.	
[6.a.2] El cambio de estado contradice el horario (volver a programada una vez pasada la hora de inicio, o a en curso una vez pasada la de fin): el sistema no lo guarda e indica que se modifique la hora en lugar del estado.	
[6.a.3] El nuevo estado es en curso o finalizada: el sistema genera y envía automáticamente la notificación correspondiente a los ciudadanos de la ciudad.	
[7.a.1] El usuario quiere notificar los cambios, o ha cambiado el estado a cancelada: se realiza el caso de uso «Crear notificación».	
Requisitos Relacionados	RF-28, RF-29, RF-34 y RF-35.

Tabla A.23: Caso de uso: Gestionar procesión

Caso de uso: Asignar pasos	
Resumen	El miembro de la Junta de Cofradías elige qué pasos de la cofradía participan en una procesión o un evento.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario está gestionando una procesión o un evento ya dado de alta (casos de uso «Gestionar procesión» y «Gestionar eventos»).
Postcondición	Quedan actualizados los pasos que participan en la procesión o el evento.
Secuencia Base	
[1] El usuario solicita elegir los pasos que participan.	
[2] El sistema muestra los pasos de la cofradía, indicando cuáles participan ya.	
[3] El usuario marca los pasos que deben participar y desmarca los que no.	
[4] El usuario confirma la selección.	
[5] El sistema guarda los pasos que participan en la procesión o el evento.	
Secuencia Alternativa	
[2.a.1] La cofradía no tiene pasos: el sistema informa de que antes hay que darlos de alta (caso de uso «Gestionar pasos»).	
[4.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
Requisitos Relacionados	RF-29 y RF-30.

Tabla A.24: Caso de uso: Asignar pasos

Caso de uso: Importar recorrido (GPX)	
Resumen	El miembro de la Junta de Cofradías define el recorrido de una procesión a partir de un archivo GPX preparado con una herramienta externa de trazado de rutas.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario está gestionando una procesión (caso de uso «Gestionar procesión»).
Postcondición	La procesión queda asociada a un recorrido formado por la secuencia ordenada de puntos del archivo.
Secuencia Base	
[1] El usuario solicita importar el recorrido de la procesión.	
[2] El sistema solicita el archivo GPX.	
[3] El usuario elige el archivo desde su dispositivo.	
[4] El sistema lee del archivo la secuencia ordenada de coordenadas del trazado, sin ningún nombre ni información adicional, tomando primero las del recorrido grabado (*track*), si no las de la ruta planificada (*route*) y, como último recurso, los puntos sueltos (*waypoints*).	
[5] El sistema crea el recorrido con esos puntos, calcula su distancia total y lo asocia a la procesión.	
[6] El sistema informa de que el recorrido se ha importado e indica su distancia.	
Secuencia Alternativa	
[3.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[4.a.1] El archivo no es un GPX válido: el sistema informa del error y no importa ningún recorrido.	
[4.a.2] El archivo tiene menos de dos puntos: el sistema informa de que no hay puntos suficientes para formar un recorrido.	
[4.a.3] El archivo tiene más de 300 puntos: el sistema conserva 300 repartidos de forma uniforme, incluidos siempre el primero y el último.	
[6.a.1] El usuario quiere destacar lugares del recorrido: se realiza el caso de uso «Marcar puntos de interés».	
Requisitos Relacionados	RF-32.

Tabla A.25: Caso de uso: Importar recorrido (GPX)

Caso de uso: Marcar puntos de interés	
Resumen	El miembro de la Junta de Cofradías destaca lugares concretos del recorrido de una procesión, como un encuentro, la entrada en una iglesia o una parada para una oración.
Actor	Miembro de la Junta de Cofradías
Precondición	La procesión tiene un recorrido importado (caso de uso «Importar recorrido (GPX)»).
Postcondición	Los puntos elegidos quedan guardados como puntos de interés del recorrido, con su información.
Secuencia Base	
[1] El usuario solicita revisar el recorrido de la procesión.	
[2] El sistema muestra los puntos del recorrido en orden, indicando cuáles son ya puntos de interés.	
[3] El usuario elige un punto del recorrido.	
[4] El sistema solicita el tipo de punto de interés (monumento, iglesia, encuentro, oración, entrada de procesión o salida de procesión), su nombre y, opcionalmente, una descripción y una imagen.	
[5] El usuario introduce los datos y confirma.	
[6] El sistema guarda el punto como punto de interés del recorrido.	
Secuencia Alternativa	
[3.a.1] El punto elegido ya es un punto de interés: el sistema muestra su información para que el usuario la modifique.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[6.a.1] Faltan el tipo o el nombre: el sistema indica que son obligatorios y no guarda el punto.	
Requisitos Relacionados	RF-33.

Tabla A.26: Caso de uso: Marcar puntos de interés

Caso de uso: Gestionar códigos de acceso	
Resumen	El miembro de la Junta de Cofradías emite y revoca los códigos de acceso con los que los cofrades de una cofradía acceden al modo cofrade.
Actor	Miembro de la Junta de Cofradías
Precondición	El usuario ha iniciado sesión como miembro activo de la Junta de Cofradías y ya ha sustituido su contraseña provisional.
Postcondición	Los códigos de acceso de la cofradía quedan actualizados.
Secuencia Base	
[1] El usuario elige una cofradía de su ciudad y solicita gestionar sus códigos de acceso.	
[2] El sistema muestra los códigos emitidos para la cofradía, con su estado (emitido, ya usado, revocado o caducado) y su fecha de caducidad.	
[3] El usuario solicita emitir un código nuevo.	
[4] El sistema genera automáticamente un código único, lo asocia a la cofradía y fija su caducidad 24 horas después.	
[5] El sistema muestra el código como código QR, junto al texto del código y su fecha de caducidad, para que los cofrades lo escaneen o lo escriban.	
Secuencia Alternativa	
[3.a.1] El usuario solicita revocar un código: el sistema pide confirmación y, si el usuario confirma, lo revoca; los cofrades que lo estén usando dejan de poder compartir su ubicación.	
[3.a.2] El usuario no quiere hacer ningún cambio: el caso de uso termina.	
Requisitos Relacionados	RF-31.

Tabla A.27: Caso de uso: Gestionar códigos de acceso

Caso de uso: Gestionar una Junta de Cofradías	
Resumen	El administrador da de alta, edita o da de baja las Juntas de Cofradías del sistema.
Actor	Administrador
Precondición	El usuario ha iniciado sesión como administrador. Existe al menos una ciudad.
Postcondición	Las Juntas de Cofradías quedan actualizadas en el sistema.
Secuencia Base	
[1] El usuario solicita gestionar las Juntas de Cofradías.	
[2] El sistema muestra las Juntas registradas, con la ciudad que gestiona cada una.	
[3] El usuario indica si quiere dar de alta una Junta nueva, editar una existente o darla de baja.	
[4] El sistema solicita los datos de la Junta: nombre, correo electrónico, teléfono, ciudad que gestiona y si está activa.	
[5] El usuario introduce o modifica los datos y confirma.	
[6] El sistema valida los datos y guarda la Junta.	
Secuencia Alternativa	
[3.a.1] Dar de baja: el sistema pide confirmación y, si el usuario confirma, elimina la Junta.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[6.a.1] La ciudad elegida ya tiene una Junta de Cofradías: el sistema informa de que cada ciudad solo puede tener una y no guarda la Junta.	
[6.a.2] Datos incompletos o no válidos: el sistema indica qué datos debe corregir el usuario y no guarda la Junta.	
[6.a.3] La Junta se desactiva: sus miembros dejan de poder realizar cambios hasta que se vuelva a activar.	
Requisitos Relacionados	RF-38.

Tabla A.28: Caso de uso: Gestionar una Junta de Cofradías

Caso de uso: Gestionar a los miembros de las Juntas de Cofradías	
Resumen	El administrador da de alta, edita, desactiva o da de baja a los miembros de las Juntas de Cofradías.
Actor	Administrador
Precondición	El usuario ha iniciado sesión como administrador. Existe al menos una Junta de Cofradías.
Postcondición	Los miembros de las Juntas quedan actualizados en el sistema.
Secuencia Base	
[1] El usuario elige una Junta de Cofradías y solicita gestionar sus miembros.	
[2] El sistema muestra los miembros de la Junta con su situación: activo, desactivado o con la invitación pendiente (todavía no ha cambiado su contraseña provisional).	
[3] El usuario indica si quiere dar de alta un miembro nuevo, editar uno existente o darlo de baja.	
[4] El sistema solicita los datos del miembro: nombre, correo electrónico y teléfono; y, al editar, también si está activo o desactivado.	
[5] El usuario introduce o modifica los datos y confirma.	
[6] El sistema valida los datos y guarda el miembro.	
[7] Al dar de alta al miembro, el sistema genera una contraseña provisional y se la envía por correo electrónico junto a un mensaje de bienvenida.	
Secuencia Alternativa	
[3.a.1] Dar de baja: el sistema pide confirmación y, si el usuario confirma, elimina al miembro.	
[3.a.2] El miembro tiene la invitación pendiente y no encuentra el correo: se realiza el caso de uso «Reenviar invitación».	
[4.a.1] Al editar, el correo electrónico no se puede modificar, ya que es el identificador con el que el miembro inicia sesión.	
[5.a.1] Cancelación de la operación: el usuario desiste y el sistema no guarda ningún cambio.	
[6.a.1] El correo electrónico ya pertenece a otra cuenta: el sistema lo informa y no guarda el miembro.	
[6.a.2] Datos incompletos o no válidos: el sistema indica qué datos debe corregir el usuario y no guarda el miembro.	
[7.a.1] Error al enviar el correo: el sistema informa del error y no da de alta al miembro, para no dejar una cuenta cuya contraseña nadie ha recibido.	
Requisitos Relacionados	RF-39.

Tabla A.29: Caso de uso: Gestionar a los miembros de las Juntas de Cofradías

Caso de uso: Reenviar invitación	
Resumen	El administrador vuelve a enviar a un miembro de la Junta de Cofradías el correo con una contraseña provisional, porque no lo recibió o lo ha perdido.
Actor	Administrador
Precondición	El usuario ha iniciado sesión como administrador. El miembro tiene la invitación pendiente: todavía no ha sustituido su contraseña provisional.
Postcondición	El miembro recibe por correo una contraseña provisional nueva, y la anterior deja de ser válida.
Secuencia Base	
[1] El usuario solicita reenviar la invitación a un miembro con la invitación pendiente.	
[2] El sistema genera una contraseña provisional nueva, que invalida la anterior.	
[3] El sistema envía de nuevo el correo de bienvenida con la contraseña provisional nueva.	
[4] El sistema informa de que la invitación se ha reenviado.	
Secuencia Alternativa	
[3.a.1] Error al enviar el correo: el sistema informa del error y mantiene la contraseña provisional anterior.	
Requisitos Relacionados	RF-39.

Tabla A.30: Caso de uso: Reenviar invitación

Caso de uso: Gestionar solicitudes de reactivación	
Resumen	El administrador revisa las solicitudes de los miembros desactivados que piden reactivar su cuenta, y las acepta o las rechaza.
Actor	Administrador
Precondición	El usuario ha iniciado sesión como administrador.
Postcondición	Las solicitudes revisadas dejan de estar pendientes; las aceptadas reactivan la cuenta del miembro.
Secuencia Base	
[1] El usuario solicita consultar las solicitudes de reactivación.	
[2] El sistema muestra las solicitudes pendientes, con el nombre, el correo electrónico y la Junta de cada miembro.	
[3] El usuario elige una solicitud y decide aceptarla.	
[4] El sistema reactiva la cuenta del miembro y marca la solicitud como resuelta.	
[5] El sistema actualiza las solicitudes pendientes.	
Secuencia Alternativa	
[2.a.1] No hay solicitudes pendientes: el sistema informa de ello.	
[3.a.1] El usuario decide rechazar la solicitud: el sistema la marca como resuelta y la cuenta sigue desactivada.	
Requisitos Relacionados	RF-41.

Tabla A.31: Caso de uso: Gestionar solicitudes de reactivación

Apéndice B

Esquema de la base de datos

Este apéndice recoge, a nivel de detalle de campo, el esquema completo de las tablas de PostgreSQL [**?**] introducidas en el apartado Diseño de la base de datos (Sección 4.1), incluyendo los tipos de dato de cada campo y las claves primarias y foráneas. Los tipos de dato mostrados corresponden a tipos nativos de PostgreSQL (`BIGINT`, `VARCHAR`, `TEXT`, `BOOLEAN`, `TIMESTAMP`, `DOUBLE PRECISION`) o a tipos enumerados propios (`ENUM`) definidos para este esquema.

B.1. Ciudad (`ciudades`)

Requisito: RI-01

Agrupa las procesiones y actos de cada localidad.

Campo	Tipo	Nota
`id`	`BIGINT`	PK, autogenerado
`nombre`	`VARCHAR`	
`comunidad_autonoma`	`VARCHAR`	
`descripcion`	`TEXT`	
`activa`	`BOOLEAN`	visible para el ciudadano o no
`latitud`	`DOUBLE PRECISION`	opcional, para preseleccionar la ciudad más cercana por GPS (RF-04)
`longitud`	`DOUBLE PRECISION`	opcional, para preseleccionar la ciudad más cercana por GPS (RF-04)

Tabla B.1: Campos de `ciudades`

B.2. Junta de Cofradías (`juntas_cofradias`)

Requisito: RI-02

Junta que gestiona una ciudad. Relación 1:1 con ciudades.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
nombre	VARCHAR	
email	VARCHAR	
telefono	VARCHAR	
ciudad_id	BIGINT	FK → ciudades(id), UNIQUE

Tabla B.2: Campos de juntas_cofradias

B.3. Cofradía / Hermandad (cofradias)

Requisito: RI-04

Asociación religiosa que organiza eventos y procesiones. Pertenece a la ciudad –nunca a la Junta de Cofradías que la gestiona–: Ciudad es quien agrupa estructuralmente a sus cofradías (relación 1:N), mientras que JuntaCofradias es solo la cuenta de gestión de esa ciudad, sustituible sin afectar a los datos de las cofradías que agrupa.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
nombre	VARCHAR	
historia	TEXT	
web	VARCHAR	URL del sitio propio de la cofradía
fecha_creacion	TIMESTAMP	
activa	BOOLEAN	visible para el ciudadano, o todavía en preparación
ciudad_id	BIGINT	FK → ciudades(id)

Tabla B.3: Campos de cofradias

B.4. Paso (pasos)

Requisito: RI-10 Pasos que posee la cofradía. A diferencia del planteamiento inicial, en el que la relación N:M era solo con procesiones, "los eventos también pueden tener pasos": la relación se generalizó a eventos (Sección **??**), del que procesiones hereda, resuelta mediante la tabla intermedia pasos_eventos.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
nombre	VARCHAR	
historia	TEXT	
analisis_artistico	TEXT	
imagen	VARCHAR	URL de texto libre, pegada a mano por la Junta (sin infraestructura de subida propia; ver trabajo futuro)
cofradia_id	BIGINT	FK → cofradias(id)

Tabla B.4: Campos de pasos

B.4.1. Paso-Evento (pasos_eventos) — tabla intermedia N:M

Campo	Tipo	Nota
paso_id	BIGINT	PK compuesta, FK → pasos(id)
evento_id	BIGINT	PK compuesta, FK → eventos(id)

Tabla B.5: Campos de pasos_eventos

B.5. Código de Acceso (codigos_acceso)

Requisito: RI-13 Códigos que emite la Junta de Cofradías en nombre de una cofradía de su ciudad (Cofradia no es un actor con cuenta propia, no puede emitir nada por sí misma). Quien lo valida obtiene un token identificado con esa cofradía y el rol de cofrade, sin que ello genere ningún registro de usuario –ver la explicación completa en la Sección **??**–. El código no se invalida al usarse por primera vez: sigue sirviendo para volver a autenticarse mientras no esté revocado ni haya caducado. Cada código caduca 24 horas después de emitirse (migración V44), pensado para que la Junta lo genere el mismo día de la procesión.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
codigo	VARCHAR	UNIQUE, generado por el servidor
estado	ENUM	EMITIDO, VALIDADO, REVOCADO
cofradia_id	BIGINT	FK → cofradias(id)
fecha_emision	TIMESTAMP	Momento en que se emite el código
fecha_expiracion	TIMESTAMP	24 horas después de fecha_emision; pasado ese momento el código deja de servir

Tabla B.6: Campos de codigos_acceso

B.6. Ubicación (ubicaciones)

Punto geográfico reutilizable (latitud, longitud y dirección textual), referenciado por eventos y por puntos_ruta. Se modela como tabla independiente, en lugar de como columnas repetidas en cada entidad que necesita una posición, para poder compartir la misma ubicación entre varias entidades –por ejemplo, la sede de una cofradía y un punto de interés del recorrido que coincidan en el mismo templo– sin duplicar datos.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
latitud	DOUBLE PRECISION	
longitud	DOUBLE PRECISION	
direccion	VARCHAR	

Tabla B.7: Campos de ubicaciones

B.7. Evento (eventos)

Requisito: RI-05

Acto que puede tener más de una cofradía participando (por ejemplo, un Vía Crucis conjunto o una procesión con pasos de varias hermandades): la relación con cofradias es N:M, no 1:N, resuelta mediante la tabla intermedia eventos_cofradias. Evento es además la tabla raíz de la que procesiones hereda sus columnas comunes (Sección **??**).

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
nombre	VARCHAR	
historia	TEXT	
tradicion	TEXT	
fecha	TIMESTAMP	
estado	ENUM	PROGRAMADO, EN_CURSO, FINALIZADO, CANCELADO
ubicacion_id	BIGINT	FK → ubicaciones(id)

Tabla B.8: Campos de eventos

B.7.1. Evento-Cofradía (eventos_cofradias) — tabla intermedia N:M

Campo	Tipo	Nota
evento_id	BIGINT	PK compuesta, FK → eventos(id)
cofradia_id	BIGINT	PK compuesta, FK → cofradias(id)

Tabla B.9: Campos de eventos_cofradias

B.8. Procesión (procesiones)

Requisito: RI-09, RI-06

A diferencia del planteamiento inicial, en el que procesiones referenciaba a eventos mediante una clave foránea 1:N, la traducción final a PostgreSQL usa **herencia real de tablas** (estrategia JOINED de JPA): Procesion *es* un Evento –hereda nombre, historia, tradicion, fecha, estado, ubicacion_id y sus cofradías participantes (eventos_cofradias)– y añade únicamente sus columnas propias en esta tabla. El id de procesiones es a la vez su clave primaria y una clave foránea a eventos(id): al guardar una procesión, el motor inserta primero la fila común en eventos y reutiliza el identificador generado para la fila de procesiones, en lugar de generar uno nuevo. Esta decisión evita mantener columnas duplicadas (cofradia_id, estado...) sincronizadas manualmente entre las dos tablas.

Campo	Tipo	Nota
id	BIGINT	PK y FK → eventos(id)
fecha_inicio	TIMESTAMP	
fecha_fin	TIMESTAMP	
recorrido_id	BIGINT	FK → recorridos(id), UNIQUE (1:1), opcional –puede programarse una procesión antes de tener ruta definida–

Tabla B.10: Campos de procesiones

B.8.1. Posiciones actuales (posiciones_actuales) — histórico de pings anónimos

A diferencia del planteamiento inicial, esta tabla no guarda una única fila por procesión que se sobrescribe con cada actualización (UPSERT), sino un **histórico de lecturas GPS sueltas** (*pings*): cada vez que un cofrade comparte su ubicación se inserta una fila nueva, sin sobrescribir ninguna anterior. Cada fila es además **anónima**: no lleva ninguna referencia a la persona que la envió, solo a la procesión (Sección **??** explica por qué). La "posición actual" de una procesión ya no es un dato guardado como tal, sino un valor **calculado** en el momento de la consulta: el promedio de latitud y longitud de los *pings* recibidos en los últimos 60 segundos, siendo el número de *pings* recientes una aproximación al número de cofrades compartiendo ubicación en ese instante. La aplicación cliente consulta este valor calculado mediante la API propia (GET /procesiones/{id}/ubicacion) con la cadencia que exige el requisito RNF-05, tal y como se explica en el apartado Problemas planteados y alternativas valoradas.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
latitud	DOUBLE PRECISION	
longitud	DOUBLE PRECISION	
timestamp	TIMESTAMP	instante de la lectura
procesion_id	BIGINT	FK → procesiones(id)

Tabla B.11: Campos de posiciones_actuales

B.9. Recorrido (recorridos)

Requisito: RI-07, RI-11

Itinerario compuesto por puntos de ruta ordenados. Sin campo `nombre`: un recorrido se identifica por la procesión a la que pertenece, no por un nombre propio.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
distancia_total	DOUBLE PRECISION	metros
tiempo_estimado	INTEGER	minutos

Tabla B.12: Campos de recorridos

B.9.1. Punto de Ruta (puntos_ruta)

A diferencia del planteamiento inicial, un mismo punto de ruta puede formar parte de **varios** recorridos –varias procesiones pueden compartir un mismo tramo de calle o pasar por el mismo templo–, así que `puntos_ruta` ya no referencia directamente a un `recorrido`: la relación es N:M y se resuelve en la tabla intermedia `recorrido_puntos_ruta` (Sección **??**). `PuntoRuta` es además la tabla raíz de una segunda jerarquía de herencia real de tablas: un punto "de paso" simple se guarda directamente aquí, mientras que un punto "de interés" con detalle añade sus columnas propias en `puntos_de_interes` (Sección **??**).

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
ubicacion_id	BIGINT	FK → ubicaciones(id)

Tabla B.13: Campos de puntos_ruta

B.9.2. Punto de Interés (puntos_de_interes)

Subtipo de `PuntoRuta` con detalle: nombre, descripción, imagen y una categoría (RI-11). El tipo enumerado se llama `TipoPuntoInteres` –es una categoría de punto de interés, no de cualquier punto de ruta– e incluye tanto categorías de interés turístico o religioso como puntos operativos del propio recorrido de una procesión (inicio y fin de la salida procesional, o el lugar donde se celebra el evento asociado).

Campo	Tipo	Nota
id	BIGINT	PK y FK → puntos_ruta(id)
tipo	ENUM	MONUMENTO, IGLESIA, ENCUENTRO, ORACCION, ENTRADAPROCESION, SALIDAPROCESION, UBICACIONEVENTO
nombre	VARCHAR	
descripcion	TEXT	
imagen	VARCHAR	URL de texto libre, pegada a mano por la Junta (sin infraestructura de subida propia; ver trabajo futuro)

Tabla B.14: Campos de puntos_de_interes

B.9.3. Recorrido-Punto de Ruta (recorrido_puntos_ruta) — tabla intermedia N:M

El orden de un punto dentro de un recorrido y la hora prevista de paso ya no son propiedades del punto en sí (un mismo punto puede estar en la posición 3 de un recorrido y en la posición 7 de otro, a horas distintas), sino de **esta relación**. Lleva un identificador propio en lugar de una clave primaria compuesta –a diferencia de pasos_eventos o eventos_cofradias– porque, además de las dos claves foráneas, tiene columnas propias; dos restricciones UNIQUE garantizan que un punto no se repita dentro del mismo recorrido y que dos puntos no compartan posición.

Campo	Tipo	Nota
id	BIGINT	PK, autogenerado
recorrido_id	BIGINT	FK → recorridos(id)
punto_ruta_id	BIGINT	FK → puntos_ruta(id)
orden	INTEGER	UNIQUE junto con recorrido_id
hora_prevista	TIMESTAMP	hora prevista de paso

Tabla B.15: Campos de recorrido_puntos_ruta

B.10. Usuario (usuarios)

Requisito: RI-03

A diferencia del planteamiento inicial, en el que usuarios era una única tabla con una columna rol que distinguía entre cofrade, junta de cofradías y administrador, la traducción final a PostgreSQL usa de nuevo **herencia real de tablas**: usuarios es la tabla raíz, con las columnas que de verdad comparten sus subtipos, y cada subtipo añade sus propias columnas en su propia tabla. Sin columna rol: con herencia real el tipo de usuario lo determina en qué tabla de subtipo tiene fila, no un valor guardado.

El cambio más relevante respecto al planteamiento inicial es que **el cofrade ha dejado de ser un** Usuario: ya no se guarda ninguna fila –ni en usuarios ni en ninguna tabla propia– para las personas que comparten ubicación durante una procesión. Validar un código de acceso (Sección **??**, codigos_acceso) no registra a nadie: el servidor comprueba que el código no está revocado y devuelve directamente

un token JWT identificado con la **cofradía** cuyo código se validó, no con una persona. Esta decisión responde a que la aplicación no necesita conocer la identidad de cada cofrade para cumplir su función –agregar y mostrar la posición de una procesión–, y evita cargar el sistema con el registro de personas cuya participación es, además, estacional (solo durante los días de procesión). `Cofrade` sigue existiendo como clase en el modelo de dominio (Figura 3.5) y en el código del backend, pero como una representación construida en memoria a partir del token en cada petición, nunca persistida.

Por tanto, `usuarios` solo contiene los perfiles con credenciales propias –junta de cofradías y administrador–, que sí se autentican con `email` y una contraseña almacenada como *hash*[1] mediante BCrypt, el algoritmo utilizado por defecto por Spring Security [**?**]. El ciudadano tampoco se registra, igual que en el planteamiento inicial.

Campo	Tipo	Nota
`id`	`BIGINT`	PK, autogenerado
`email`	`VARCHAR`	`UNIQUE`
`password_hash`	`VARCHAR`	*hash* BCrypt
`fecha_ingreso`	`TIMESTAMP`	

Tabla B.16: Campos de `usuarios`

B.10.1. Miembro de Junta de Cofradías (`miembros_junta_cofradia`)

Persona con credenciales que pertenece a la Junta de Cofradías de una ciudad –una Junta puede tener varios miembros, cada uno con su propio inicio de sesión–. La referencia es a la `JuntaCofradias`, no directamente a la `Ciudad`: así se reutiliza la relación 1:1 ya existente entre `JuntaCofradias` y `Ciudad` en lugar de duplicarla, y de ahí –`miembro.juntaCofradias.ciudad`– es de donde la capa de servicio de la API obtiene qué ciudad gestiona cada miembro a la hora de comprobar la autorización por rol (RNF-03). `activo` lo alterna el Administrador: desactivado no significa sin acceso –el *login* le sigue funcionando–, sino que el backend le rechaza cualquier escritura. `password_provisional` vale `TRUE` hasta que el miembro cambia la contraseña que se le generó y envió por correo al darlo de alta, como señal de que la invitación sigue pendiente. `solicitud_reactivacion_pendiente` la activa un miembro desactivado que pide volver a estar activo, y la resuelve el Administrador aceptándola o rechazándola.

Campo	Tipo	Nota
`id`	`BIGINT`	PK y FK → `usuarios(id)`
`nombre`	`VARCHAR`	
`telefono`	`VARCHAR`	opcional
`junta_cofradias_id`	`BIGINT`	FK → `juntas_cofradias(id)`
`activo`	`BOOLEAN`	
`password_provisional`	`BOOLEAN`	
`solicitud_reactivacion_pendiente`	`BOOLEAN`	

Tabla B.17: Campos de `miembros_junta_cofradia`

[1] Hash: transformación irreversible de una contraseña en una cadena de caracteres, de forma que el sistema pueda comprobar que una contraseña es correcta sin necesidad de guardarla en texto plano.

B.10.2. Administrador (`administradores`)

Rol global del sistema, sin referencia a `Ciudad`: cualquier administrador puede gestionar cualquier ciudad, no tiene sentido atarlo de antemano a una que todavía no exista. `nombre` y `telefono` son opcionales –no se piden al crear la cuenta, solo se completan cuando el propio administrador edita su perfil–.

Campo	Nota
`id`	PK y FK → `usuarios(id)`
`nombre`	opcional
`telefono`	opcional

Tabla B.18: Campos de `administradores`

B.11. Notificación (`notificaciones`)

Requisito: RI-08

A diferencia del planteamiento inicial, no hay una jerarquía Aviso/Alerta: una única clase cubre los dos casos, con un tipo y una prioridad en vez de una distinción de subtipos –al revisar qué se usaba de verdad se vio que el cliente pinta cada notificación solo a partir de su prioridad, nunca de un subtipo más fino, así que esa jerarquía no se estaba ganando su sitio–. Pertenece a una `Ciudad`: es el ámbito con el que el ciudadano filtra qué notificaciones ver, y con el que la Junta que las crea comprueba su autorización.

Campo	Tipo	Nota
`id`	`BIGINT`	PK, autogenerado
`titulo`	`VARCHAR`	
`mensaje`	`TEXT`	opcional
`fecha_creacion`	`TIMESTAMP`	
`ciudad_id`	`BIGINT`	FK → `ciudades(id)`
`tipo`	`ENUM`	`INICIO`, `FIN` (los genera el propio sistema al cambiar el estado de un `Evento`), `INCIDENCIA`, `CAMBIO_HORARIO`, `CANCELACION` (los crea la Junta a mano)
`prioridad`	`ENUM`	`BAJA`, `MEDIA`, `ALTA`; obligatoria salvo en `INICIO/FIN`
`fecha_expiracion`	`TIMESTAMP`	opcional, para cualquier tipo

Tabla B.19: Campos de `notificaciones`

Sin tabla de notificación entregada: no se registra si un ciudadano concreto ha leído cada notificación o no –no tiene cuenta con la que asociar esa lectura (Sección **??**)–, simplemente se consulta la lista de notificaciones vigentes de su ciudad.

B.12. Dispositivo Push (dispositivos_push)

Requisito: RI-12

Dispositivo del ciudadano registrado para recibir notificaciones *push* de una ciudad. Sin usuario al que pertenecer –el ciudadano nunca se autentica (Sección **??**)–: la única pertenencia real es a qué ciudad le interesa a ese token, que puede cambiar si el ciudadano cambia de ciudad seleccionada (el mismo token se reregistra, actualizando `ciudad_id` en vez de crear una fila nueva). El `token` es de Expo Push (`ExponentPushToken[...]`), no un token FCM/APNs en crudo: el backend solo habla con la API de Expo Push (`exp.host`), nunca directamente con Firebase Cloud Messaging ni con APNs.

Campo	Tipo	Nota
`id`	`BIGINT`	PK, autogenerado
`token`	`VARCHAR`	`UNIQUE`, token de Expo Push
`ciudad_id`	`BIGINT`	FK → `ciudades(id)`
`fecha_registro`	`TIMESTAMP`	
`fecha_actualizado`	`TIMESTAMP`	

Tabla B.20: Campos de `dispositivos_push`

B.13. Relaciones entre tablas

La Tabla **??** recoge las relaciones entre tablas de este esquema, todas ellas garantizadas por claves foráneas con restricción de integridad referencial.

De	A	Relación	Cardinalidad
`ciudades`	`cofradias`	agrupa	1:N
`ciudades`	`juntas_cofradias`	gestionada por	1:1
`juntas_cofradias`	`miembros_junta_cofradia`	tiene	1:N
`cofradias`	`eventos`	participa en	N:M
`cofradias`	`pasos`	posee	1:N
`cofradias`	`codigos_acceso`	emite	1:N
`eventos`	`procesiones`	es extendido por (herencia)	1:1
`eventos`	`ubicaciones`	se celebra en	N:1
`procesiones`	`recorridos`	tiene	1:1
`procesiones`	`posiciones_actuales`	recibe	1:N
`recorridos`	`puntos_ruta`	compuesto por	N:M
`puntos_ruta`	`puntos_de_interes`	es extendido por (herencia)	1:1
`puntos_ruta`	`ubicaciones`	está en	N:1
`pasos`	`eventos`	participa en	N:M
`miembros_junta_cofradia`	`usuarios`	es extendido por (herencia)	1:1
`administradores`	`usuarios`	es extendido por (herencia)	1:1
`ciudades`	`notificaciones`	recibe	1:N
`ciudades`	`dispositivos_push`	recibe registros de	1:N

Tabla B.21: Relaciones entre tablas de PostgreSQL

Apéndice C

Manual de despliegue

Este apéndice describe los pasos necesarios para desplegar el backend propio (API Spring Boot y base de datos PostgreSQL) en el servidor virtual proporcionado por la Escuela de Ingeniería Informática de la Universidad de Valladolid, así como la configuración necesaria en el cliente React Native para que la aplicación completa funcione en un entorno nuevo. El proyecto no depende de ningún servicio de Firebase (Sección 4.1.1): las notificaciones *push* se envían a través de Expo Push Notifications, sin necesidad de credenciales propias.

C.1. Requisitos previos

- Acceso por SSH al servidor virtual (`virtual.lab.inf.uva.es`), con un usuario con permisos de `sudo`. El servidor es una máquina virtual Linux cuyos puertos de entrada asignados están en el rango 20101–20109: el 20101 se usa para SSH y el 20102 para la API.
- Java 17 o superior, Git y PostgreSQL instalados en el servidor.
- Repositorio del proyecto en GitHub [**?**], desde el que se descarga el código en el servidor.
- Cuenta de Expo [**?**] con un proyecto EAS creado, para generar el ejecutable de la aplicación móvil y para que `expo-notifications` pueda emitir tokens de Expo Push válidos.
- Clave de API de Google Maps Platform [**?**] para el SDK de Maps en Android, restringida al paquete de la aplicación y a la huella SHA-1 del certificado con el que EAS firma el ejecutable. En iOS los mapas usan Apple Maps y no necesitan clave.
- Java 17 o superior y Node.js instalados en el entorno de desarrollo local.

C.2. Despliegue del backend en el servidor

1. Conectarse al servidor por SSH:

```
ssh -p 20101 usuario@virtual.lab.inf.uva.es
```

2. Crear en PostgreSQL la base de datos y el usuario que utilizará la API. No es necesario crear ninguna tabla: al arrancar, Spring Boot ejecuta automáticamente las migraciones versionadas del esquema (Apéndice **??**) antes de empezar a atender peticiones.

3. Clonar el repositorio y compilar el backend, lo que genera el ejecutable `.jar` de la API:

```
git clone <URL del repositorio> ~/TFG
cd ~/TFG/backend && ./mvnw package -DskipTests
```

4. Crear el fichero de configuración `/etc/semanasanta/backend.env`, legible solo por el administrador del servidor, con las variables de entorno de la API:

   - `PORT`: puerto en el que escucha la API (`20102`).
   - `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME` y `SPRING_DATASOURCE_PASSWORD`: conexión con la base de datos PostgreSQL creada en el paso anterior.
   - `JWT_SECRET`: clave secreta utilizada para firmar los tokens JWT (Sección 4.1). Debe ser una cadena aleatoria de suficiente longitud y no debe compartirse ni subirse al repositorio.
   - `JWT_EXPIRATION`: tiempo de validez del token, en milisegundos.
   - `ADMIN_BOOTSTRAP_SECRET`: secreto que permite crear el primer Administrador del sistema (`POST /administradores/bootstrap`) cuando todavía no existe ninguno.
   - `MAIL_USERNAME` y `MAIL_PASSWORD`: cuenta SMTP de Gmail (con una contraseña de aplicación) desde la que se envía el correo de bienvenida a los miembros de las Juntas de Cofradía.

   No hace falta configurar ninguna credencial para las notificaciones *push*: el backend envía cada notificación con una petición HTTP corriente al servicio de Expo Push (`https://exp.host/-/api/v2/push/send`), sin autenticación propia.

5. Registrar la API como servicio del sistema (`semanasanta`) mediante systemd, con `backend.env` como fichero de entorno, para que arranque sola al iniciar el servidor y se reinicie automáticamente si se detiene por un error.

6. Arrancar el servicio y comprobar su estado:

```
sudo systemctl start semanasanta
systemctl status semanasanta
sudo journalctl -u semanasanta -f
```

   Una vez arrancada, la API queda accesible en `http://virtual.lab.inf.uva.es:20102`, que es la URL base que consume la aplicación cliente. El servidor no dispone de un certificado TLS, por lo que la comunicación se realiza mediante HTTP; por este motivo el cliente permite expresamente el tráfico HTTP hacia ese dominio (véase la sección de configuración del cliente).

7. Crear el primer Administrador llamando, desde el propio servidor, al *endpoint* de *bootstrap* con el secreto configurado en `ADMIN_BOOTSTRAP_SECRET`. A partir de ahí, el resto de cuentas se crean desde la propia aplicación.

C.2.1. Actualización a una nueva versión

El despliegue de una nueva versión se realiza manualmente desde el servidor: se descarga el código más reciente del repositorio, se vuelve a compilar el backend y se reinicia el servicio. Si la nueva versión incluye migraciones del esquema, se aplican automáticamente al arrancar.

```
cd ~/TFG && git pull
cd backend && ./mvnw package -DskipTests
sudo systemctl restart semanasanta
```

C.3. Documentación de la API

Una vez desplegado el backend, la documentación interactiva de la API, generada automáticamente por springdoc-openapi [**?**] a partir del propio código, queda disponible en `/swagger-ui.html` sobre la URL del backend, y permite consultar y probar cada *endpoint* sin necesidad de herramientas adicionales.

C.4. Configuración del cliente React Native

1. La URL base de la API se fija con la variable de entorno `EXPO_PUBLIC_API_URL`. En los ejecutables generados con EAS la define `eas.json` en los perfiles `preview` y `production` (`http://virtual.lab.inf.uva.es`); durante el desarrollo puede fijarse en un fichero `.env.local` para trabajar contra el servidor, o dejarse sin definir para trabajar contra un backend local.

2. Al usar HTTP, el cliente debe permitir expresamente ese tráfico: en Android mediante la opción `usesCleartextTraffic` del plugin `expo-build-properties`, y en iOS mediante una excepción de App Transport Security para `virtual.lab.inf.uva.es` en `ios.infoPlist`, ambas en `app.json`.

3. Comprobar que `app.json` tiene configurado el `projectId` del proyecto EAS (campo `extra.eas.projectId`) y el plugin `expo-notifications`: son los que permiten que el dispositivo obtenga un token de Expo Push válido y lo registre en la API propia (`POST /dispositivos-push`).

4. Registrar la clave de Google Maps como variable de entorno de EAS (`GOOGLE_MAPS_API_KEY`) para los entornos `preview` y `production`. El fichero `app.config.js` la lee y la pasa al plugin de `react-native-maps`, de modo que la clave nunca se guarda en el repositorio.

C.5. Generación del ejecutable de la aplicación móvil

El paquete de instalación se genera en los servidores de Expo mediante EAS Build, sin necesidad de compilar en local. El perfil `preview` produce un `.apk` de Android que puede instalarse directamente en el dispositivo a partir del enlace que devuelve la compilación:

```
eas build --profile preview --platform android
```

El perfil `production` genera el paquete destinado a su publicación en las tiendas de aplicaciones. En el caso de iOS, instalar el ejecutable en un dispositivo físico requiere además una cuenta del Apple Developer Program.

Apéndice D

Manual de uso

Apéndice E

Contenido del TFG